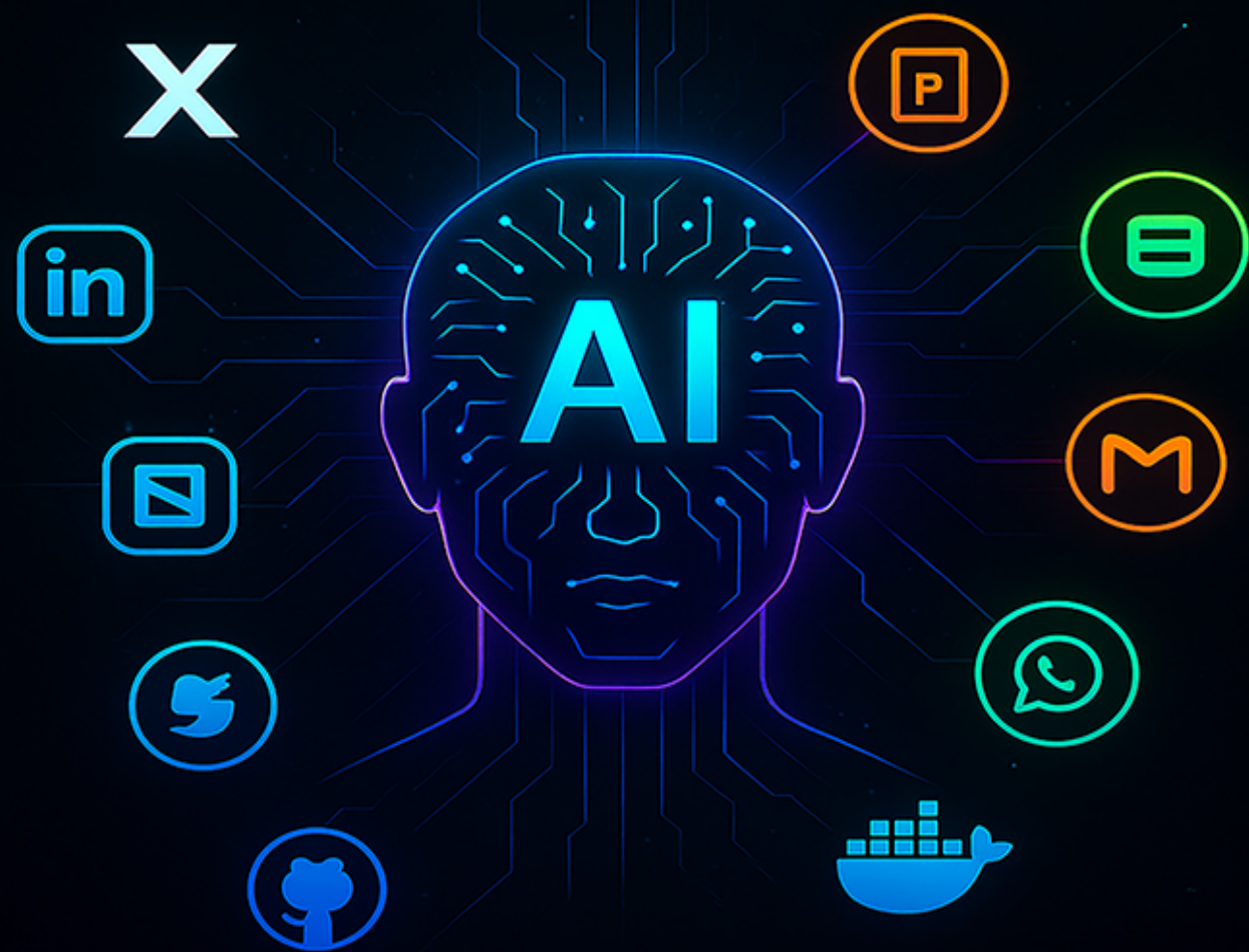


MODEL CONTEXT PROTOCOL

Advanced AI Agents for Beginners



MEHUL GUPTA & NILADRI SEN

Model Context Protocol

Advanced AI Agents for Beginners

For the love of AI

Copyrights

Model Context Protocol: Advanced AI Agents for Beginners

Copyright © 2025 by Mehul Gupta, Niladri Sen

All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means—including photocopying, recording, or other electronic or mechanical methods—without prior written permission from the author, except in cases of brief quotations for critical reviews and certain noncommercial uses permitted by copyright law.

Disclaimer:

The information in this book is for general informational purposes only. The author is not liable for any errors, omissions, or damages resulting from its use.

Edition: First Edition

ISBN: **978-93-342-8471-3**

For permissions or inquiries, contact:

mehulgupta2016154@gmail.com

me.niladri.sen@gmail.com

Preface

Dear Reader,

Hope you're doing great. Welcome to our new book, ***Model Context Protocol : Advanced AI Agents for Beginners***. If you've picked this up, chances are you're curious about what comes after chatbots and LLMs—and you're definitely not alone. The past couple of years were all about crafting clever prompts and watching LLMs spit out magic. But 2025? *This is the year of AI agents*. And this book is your entry point into that world.

So, what's all the buzz about AI agents?

Here's the simple version: instead of relying on a single smart assistant to answer your questions, imagine creating a **team of helpful bots** that can **think, plan, and work together** to solve real problems. Whether it's automating a task, researching a topic, reviewing resumes, translating content, or writing code—AI agents are quietly becoming the real workers behind the scenes.

Rather than spitting tokens, AI can now execute stuff!

So, are we talking about Crewai, LangChain, LangGraph, or Autogen?

Well... not exactly. Those frameworks/tools were great for getting us started. Think of them as **version 1.0** of AI agents. But now, we've moved into the next phase. With the introduction of **Model Context Protocol (MCP)** by Anthropic last year, the way we think about agents has completely changed. It's no longer just about connecting prompts and tools. MCP brings in structure, memory, communication, and true collaboration. And this book is all about helping you build with that—without breaking a sweat.

Why this book?

We wrote this book to help beginners take the next step in their AI journey. One of us had previously written *LangChain in Your Pocket (published by Packt)*, which introduced readers to building AI-powered apps. But in the past year, we both saw a shift. People want more than chat tools—they want systems that can **understand goals, make decisions, and take action**. That's what AI agents do—and this book will show you how to build them using MCP.

We've kept it all **hands-on, beginner-friendly, and jargon-free**. You don't need a PhD or even a background in AI to follow along. We'll walk you through the basics, introduce key concepts with plain English, and give you working code examples you can run, tweak, and learn from. Think of this book as your personal guide to building smart, practical, and useful AI agents—without the overwhelm.

We've both spent years in the AI space—writing blogs, building projects, creating tutorials, and helping people get started. And we saw the same thing again and again: lots of curiosity, but not enough clear guidance. So, we combined our experiments—and turned them into a simple, focused roadmap anyone can follow.

Why should you read this book?

Because AI agents aren't just a trend—they're the future of how software works.

Whether you're a developer, student, or just someone excited about where AI is headed, this book will help you understand and build the systems that are powering the next generation of tools. And trust us—they're not as hard as they sound.

Finally, a huge thank you to everyone who's supported us—especially our families, for letting us chase wild ideas, debug at midnight, and write chapters fueled by coffee and ambition. And to you, dear reader: thanks for joining us. We hope this book answers your questions, sparks your creativity, and helps you build something incredible. A special mention to Nitya Pydipati (our colleague) for thoroughly reviewing the book. Let's dive in.

Happy building,
Mehul & Niladri

Table Of Contents

[Copyrights](#)

[Preface](#)

[Table Of Contents](#)

[Chapter 1: Introduction](#)

[1.1 What is Generative AI?](#)

[1.1.1 How does Generative AI work?](#)

[1.1.2 Generative AI vs Traditional AI](#)

[1.1.3 Generative AI use cases](#)

[1.1.4 Why is Generative AI Revolutionary?](#)

[1.2 What are LLMs?](#)

[1.2.1 How LLMs Work](#)

[1.2.2 LLMs Limitations](#)

[Chapter 2: What are AI Agents](#)

[2.1 What are Agents?](#)

[2.1.1 AI Agents vs LLMs?](#)

[2.1.2 How Do AI Agents Execute Tasks?](#)

[2.1.3 Why Are AI Agents Important?](#)

[2.2 Popular AI Agent Frameworks](#)

[2.3 Potential Challenges with AI Agents](#)

[Chapter 3: What is Model Context Protocol?](#)

[3.1 What is a Protocol?](#)

[3.2 What is Model Context Protocol?](#)

[3.2.1 The Need for MCP](#)

[3.2.2 How MCP Works](#)

[3.3 Key Components of MCP](#)

[3.3.1 Resources: What the AI Knows](#)

[3.3.2 Tools: What the AI Can Do](#)

[3.3.3 Prompts: What the AI Should Say](#)

[3.3.4 Real-World Use Cases](#)

[3.4 MCP = AI Agents 2.0?](#)

[3.5 Basic Setup](#)

[Chapter 4: GSuite MCP server](#)

[4.1 Introduction](#)

[4.1.1 GSuite MCP tools \(Gmail\)](#)

[4.1.1 Real-World Applications](#)

[4.2 Step-by-Step Installation](#)

[4.2.1 Pre-requisites](#)

[4.2.2 Steps](#)

[Chapter 5: Excel MCP server](#)

[5.1 Introduction](#)

[5.1.1 Excel MCP Tools](#)

[5.1.2 Real-World Applications](#)

[5.2 Step-by-Step Installation](#)

[5.2.1 Steps](#)

[Chapter 6: PowerPoint MCP server](#)

[6.1 Introduction](#)

[6.1.1 PowerPoint MCP Tools](#)

[6.1.2 Real-World Applications](#)

[6.2 Step-by-Step Installation](#)

[6.2.1 Pre-requisites](#)

[6.2.2 Steps](#)

[Chapter 7: Notion MCP server](#)

[7.1 Introduction](#)

[7.1.1 Notion MCP Tools](#)

[7.1.2 Real-World Applications](#)

[7.2 Step-by-Step Installation](#)

[7.2.1 Pre-requisites](#)

[7.2.2 Steps](#)

[Chapter 8: WhatsApp MCP server](#)

[8.1 Introduction](#)

[8.1.1 WhatsApp MCP Tools](#)

[8.1.2 Real-World Applications](#)

[8.2 Step-by-Step Installation](#)

[8.2.1 Pre-requisites](#)

[8.2.2 Steps](#)

[Chapter 9: YouTube MCP Server](#)

[9.1 Introduction](#)

[9.1.1 YouTube MCP Tools](#)

[9.1.2 Real-World Applications](#)

[9.2 Step-by-Step Installation](#)

[9.2.1 Pre-requisites](#)

[9.2.2 Steps](#)

[Chapter 10: Slack MCP Server](#)

[10.1 Introduction](#)

[10.1.1 Slack MCP Tools](#)

[10.1.2 Real-World Applications](#)

[10.2 Step-by-Step Installation](#)

[10.2.1 Pre-requisites](#)

[10.2.2 Steps](#)

[Chapter 11: Discord MCP server](#)

[11.1 Introduction](#)

[11.1.1 Discord MCP Tools](#)

[11.1.2 Real-World Applications](#)

[11.2 Step-by-Step Installation](#)

[11.2.1 Prerequisites](#)

[11.2.2 Steps](#)

[Chapter 12: Twitter MCP Server](#)

[12.1 Introduction](#)

[12.1.1 Twitter MCP Tools](#)

[12.1.2 Real-World Applications](#)

[12.2 Step-by-Step Installation](#)

[12.2.1 Pre-requisites](#)

[12.2.2 Steps](#)

[Chapter 13: LinkedIn Automation using Zapier MCP](#)

[13.1 Introduction](#)

[13.1.1 Zapier MCP Tools \(LinkedIn\)](#)

[13.1.2 Real-World Applications](#)

[13.2 Step-by-Step Installation](#)

[13.2.1 Pre-requisites](#)

[13.2.2 Steps](#)

[Chapter 14: SQL MCP server](#)

[14.1 Introduction](#)

[14.1.1 SQL MCP Tools](#)

[14.1.2 Real-World Applications](#)

[14.2 Step-by-Step Installation](#)

[14.2.2 Steps](#)

[Chapter 15: GitHub MCP server](#)

[15.1 Introduction](#)

[15.1.1 GitHub MCP Tools](#)

[15.1.2 Real-World Applications](#)

[15.2 Step-by-Step Installation](#)

[15.2.1 Pre-requisites](#)

[15.2.2 Steps](#)

[Chapter 16: Docker MCP server](#)

[16.1 Introduction](#)

[16.1.1 Docker MCP Tools](#)

[16.1.2 Real-World Applications](#)

[16.2 Step-by-Step Installation](#)

[16.2.1 Pre-requisites](#)

[16.2.2 Steps](#)

[Chapter 17: Jupyter MCP server](#)

[17.1 Introduction](#)

- [17.1.1 Jupyter MCP Tools](#)
 - [17.1.2 Real-World Applications](#)
- [17.2 Step-by-Step Installation](#)
 - [17.2.1 Pre-requisites](#)
 - [17.2.2 Steps](#)
- [Chapter 18: Blender MCP server](#)
 - [18.1 Introduction](#)
 - [18.1.1 Blender MCP Tools](#)
 - [18.1.2 Real-World Applications](#)
 - [18.2 Step-by-Step Installation](#)
 - [18.2.1 Pre-requisites](#)
 - [18.2.2 Steps](#)
- [Chapter 19: Figma MCP server](#)
 - [19.1 Introduction](#)
 - [19.1.1 Figma MCP Tools](#)
 - [19.1.2 Real-World Applications](#)
 - [19.2 Step-by-Step Installation](#)
 - [19.2.1 Pre-requisites](#)
 - [19.2.2 Steps](#)
- [Chapter 20: Filesystem MCP Server](#)
 - [20.1 Introduction](#)
 - [20.1.1 Filesystem MCP Tools](#)
 - [20.1.2 Real-World Applications](#)
 - [20.2 Step-by-Step Installation](#)
 - [20.2.1 Steps](#)
- [Chapter 21: Puppeteer MCP Server](#)
 - [21.1 Introduction](#)
 - [21.1.1 Puppeteer MCP Tools](#)
 - [21.1.1 Real-World Applications](#)
 - [21.2 Step-by-Step Installation](#)
 - [21.2.1 Prerequisites](#)
 - [21.2.2 Steps](#)
- [Chapter 22: MCP using Local LLMs with Ollama](#)
 - [22.1 Introduction](#)
 - [22.1.1 Task Automation with MCPHost](#)
 - [22.2 Step-by-Step Installation](#)
 - [22.2.1 Prerequisites](#)
 - [22.2.2 Steps](#)
 - [22.3 Usage tips](#)
- [Chapter 23: MCP tools using LangChain](#)
 - [23.1 Introduction](#)
 - [23.2 Step-by-Step Installation](#)

[23.2.1 Prerequisites](#)
[23.2.2 Steps](#)
[Chapter 24: Any LLM - Any MCP](#)
[24.1 Introduction](#)
[24.1.1 Features](#)
[24.2 Step-by-Step Installation](#)
[24.2.1 Pre-requisites](#)
[24.2.2 Steps](#)
[Chapter 25: Building Your Own MCP Server](#)
[25.1 Introduction](#)
[25.1.1 Features](#)
[25.2 Step-by-Step Installation](#)
[25.2.1 Pre-requisites](#)
[25.2.2 Steps](#)
[Chapter 26: MCP might be risky](#)
[26.1 Why are MCP Servers unsafe?](#)
[26.2 How to Stay Safe](#)
[Chapter 27 : The Future of MCP and AI Agents](#)
[EndNotes](#)
[About the Authors](#)

Part 1: Foundations

Chapter 1: Introduction

AI will probably most likely lead to the end of the world, but in the meantime, there'll be great companies

- Sam Altman

ChatGPT dropped in November 2022, and life hasn't been the same since. In just 2.5–3 years, AI has completely changed the game. At first, everyone was obsessed with making better language models (LLMs)—that's why we saw so many releases like Llama, Claude, Mistral, and new versions of GPT. But by late 2024, things started shifting. Now, it's not just about chatbots—it's about AI agents.

What's an AI agent?

How do you build one?

And can we actually trust them in real life?

We'll delve into all of that soon. But first, this chapter is simply about setting the stage—we won't go into too much depth yet; just cover the basics so you're ready for what's coming next.

Let's start with the first and foremost question ...

1.1 What is Generative AI?

Generative AI is a type of AI that produces content by learning from existing examples. Think of it like a really smart digital artist, writer, or musician. Instead of just analyzing data, it takes what it has learned and produces fresh content — like writing a story, generating an image, composing music, or even creating computer code. If you've heard of ChatGPT, DALL·E, or Midjourney, those are examples of generative AI in action.

1.1.1 How does Generative AI work?

We won't deep dive into the world of Transformers and GPTs but let me give you a shallow explanation so that you have an overview how this entire thing works.

1. **Learn from a Lot of Data:** Generative AI starts by learning from a huge amount of content—books, websites, images, videos, code, you name it. This stage is called

training. The AI looks at all this data and figures out how humans typically write, draw, or speak. It's like the AI is binge-reading the internet, soaking up patterns and styles without really *understanding* anything—it's *still outputting tokens it doesn't understand* !

2. **Spot Patterns:** After all that training, the AI becomes a pattern expert. It can look at a sentence and guess what word should come next, or take a drawing prompt and figure out what shapes and colors match it. Think of it like a very advanced autocomplete—it doesn't think like we do, but it's scarily good at predicting what *sounds right* based on what it's seen before.
3. **Generate New Content:** When you give the AI a prompt—like “write a bedtime story” or “draw a robot riding a unicorn”—it uses what it learned to generate something brand new. It's not just copying from its training data; it's mixing, matching, and reassembling familiar pieces into something original. That's what makes it generative—it *creates* rather than just repeats.
4. **Important to Know:** Even though it looks smart, the AI doesn't truly understand anything. It doesn't know facts, feel emotions, or have opinions. It just predicts what sounds plausible. Because it learned from real-world data (which can be biased or flawed), it sometimes makes mistakes or produces weird, unhelpful, or even incorrect results.
5. **Quick Analogy:** Picture a talented chef who has studied thousands of recipes. When you ask for a new dish, they don't just copy one recipe—they combine ingredients and techniques in fresh ways to create something that feels new. But everything they know comes from what they've learned before; they're not inventing flavors out of nowhere, just remixing what works

Like a chef who reworks recipes, generative AI remixes data patterns—it doesn't invent or think, just reshapes what it knows

1.1.2 Generative AI vs Traditional AI

AI isn't a one-size-fits-all thing. In fact, there's a huge difference between traditional models—like logistic regression and random forests—and the newer, flashier generative models like GPT-4. To really understand how AI agents work, you need to get a feel for how these two branches of AI go about solving problems very differently.

A. Traditional AI: Prediction, Classification, and Clear Rules

Let's start with the old-school models. Traditional AI models are built to solve structured problems. These include well-known algorithms like logistic regression, decision trees, and random forests. Their goal is usually to predict an outcome or classify data based on clear patterns in structured inputs—though with proper feature engineering, they can also work effectively with unstructured data, such as text, images, or audio.

Imagine you're working in a bank. You might use logistic regression to predict whether a loan applicant is likely to default. You feed it data like income, credit score, and employment history, and it gives you a score—maybe a 72% chance of default. Similarly, a random forest might be used to flag fraudulent transactions by analyzing how much was spent, where, and how often.

These models follow strict rules. If the input is the same, the output will always be the same. They're great for tasks where the right answer is well-defined, like "spam or not spam," "will churn or won't churn," "positive review or negative review". Alternatively, clustering models—often considered the flag bearers of unsupervised learning—group similar data without predefined labels, offering insights into areas like customer segmentation and topic discovery.

B. Generative AI: Creating New Ideas from Patterns

Now let's talk about the new kid on the block: **Generative AI**. This type of AI doesn't just classify data—it **creates** new content based on what it's learned. Rather than answering a question like "Is this email spam?" It can write an email from scratch. Rather than identifying a cat in a photo, it can generate a brand-new picture of a cat flying through space in a superhero cape.

Models like GPT-4, DALL·E, and Stable Diffusion are trained on **massive amounts of unstructured data**—everything from Wikipedia articles to Reddit threads, code repositories, product reviews, and more. The magic happens in how they use that data: they learn patterns, styles, and relationships between words, phrases, and images, and then use those patterns to generate something entirely new.

C. Deterministic vs. Probabilistic Behavior

Another key difference is **predictability**. Traditional models are **deterministic**: feed in the same input, get the same output every time. They're consistent, but inflexible. Generative models, on the other hand, are **probabilistic**: they calculate many possible next words or images and choose the most likely—but with some randomness thrown in.

This is why asking ChatGPT the same question twice might get you two slightly different, but equally good, answers. That kind of variability makes generative models feel more conversational and human—but it also means they're not always reliable for tasks that require precision.

D. Use Cases: When to Use What

So when should you use traditional models, and when should you go for generative models?

1. Use Traditional AI when you want solid, explainable answers: credit scoring, fraud detection, customer churn prediction, recommendation systems.

2. Use Generative AI when you need flexible, human-like output: writing emails, generating images, building chatbots, summarizing documents, answering open-ended questions.

1.1.3 Generative AI use cases

Generative AI isn't just for tech demos or sci-fi fantasies anymore—it's already woven into our daily routines and powering serious business tools behind the scenes. Let's look at some of the most common ways it's being used today.

1. **Content Creation:** Whether you're crafting emails, social media posts, or blog articles, generative AI tools like ChatGPT, Jasper, and Notion AI can turn rough ideas into polished drafts in seconds. They help break through writer's block, suggest alternative phrasings, and even tailor tone and style depending on the audience—perfect for marketers, students, and busy professionals alike. If you're eager to dive deeper, Chapter 7 explores how you can use Notion to take notes effectively.
2. **Personalized Learning and Tutoring:** Students and self-learners use generative AI for on-demand tutoring, getting instant explanations for tough concepts or step-by-step walkthroughs of math problems. Platforms like Khan Academy's Khanmigo and language apps like Duolingo use these models to simulate real-time conversations and adapt lessons to each learner's pace and level.
3. **Smarter Customer Service Bots:** Many companies are replacing rigid chatbots with conversational AI assistants that understand natural language and respond with helpful, personalized answers. For example, instead of offering just menu options, an AI-powered bot can handle specific questions like "Where's my refund?" or "Can I change my flight?"—making support faster and more human-like.
4. **Brainstorming and Creative Collaboration;** Designers, writers, and musicians are turning to AI as a creative partner—using tools like DALL·E for visual concepts, Sudowrite for novel writing, or AI music generators to spark melodies and lyrics. It's not about replacing human creativity, but enhancing it with new perspectives and quick idea generation.
5. **Programming Help and Automation:** Developers save time by using tools like GitHub Copilot or Replit Ghostwriter, which suggest code snippets, fix bugs, and even explain tricky syntax. Even non-coders can benefit—just describe what you want (e.g., "Rename files with today's date"), and AI can generate the script for you in Python, JavaScript, or whatever language you need.
6. **Personalised Recommendations and Experiences:** Streaming platforms, shopping sites, and fitness apps are using generative AI to create more tailored user experiences. For instance, Netflix may generate customized thumbnails, Spotify might create a playlist for your Monday blues, and fitness apps can design workouts based on your energy level or goals.
7. **Everyday Assistant Tasks:** From generating travel checklists to summarizing meeting notes, generative AI can handle dozens of tiny but time-consuming tasks. It's like having a digital assistant who's great with research, writing, and organization—always ready to help you phrase an email, translate a document, or prep for a presentation.

1.1.4 Why is Generative AI Revolutionary?

The benefits of Generative AI and its profound influence on various domains underscore its role as a major milestone in technological advancement.

1. **Democratization of Creativity:** Generative AI removes the barriers to creation. You no longer need to be an expert to design graphics, write stories, compose music, or build apps. It empowers anyone—regardless of skill level—to turn ideas into tangible outputs, shifting creativity from an exclusive club to a global playground.
2. **Blazing Fast Productivity:** What used to take hours—editing photos, writing drafts, generating reports—now takes seconds. Whether it's translating a document, writing code, or generating branding assets, Generative AI compresses timelines drastically, making workflows smoother and faster across industries.
3. **Massive Accessibility Gains:** Generative AI puts powerful capabilities in the hands of anyone with a browser and a prompt. Students, freelancers, small business owners—people who never had access to top-tier tools or resources—can now compete with larger players, leveling the playing field like never before.
4. **Everyday Problem-Solving Made Simple:** Need a travel itinerary? A personalized workout plan? Dinner recipes based on your fridge contents? Generative AI handles small, routine tasks with surprising depth and variety, becoming a daily sidekick for personal and professional life alike.
5. **Constantly Evolving and Improving:** It's still early days for Generative AI, but the rate of progress is mind-blowing. Models are becoming more accurate, nuanced, and context-aware with every iteration. As the tech matures, it unlocks even more complex, domain-specific use cases that were unimaginable a year ago.
6. **Revolutionizing Entire Industries:** From drafting medical documentation in healthcare, to rapid prototyping in design, to virtual tutors in education—Generative AI is reshaping how professionals work. It's not just enhancing productivity—it's redefining job roles and workflows.
7. **Creative Collaboration Between Human and Machine:** The leap from “machines that detect” to “machines that create” is huge. This isn't automation as we knew it—this is augmentation. Generative AI acts as a co-pilot, helping humans ideate, iterate, and innovate more effectively than ever before.
8. **A Paradigm Shift in Human Expression:** More than a tool, Generative AI is a new medium for human expression. It enables people to communicate ideas visually, textually, musically—whatever their creative flavor—with unprecedented ease and impact.

9. **Fueling Innovation at the Edges:** Generative AI isn't just helping big corporations—it's enabling the garage hacker, the indie creator, the solo entrepreneur. The long tail of innovation is getting longer, and we're seeing new ideas bloom from corners of the world that were previously underrepresented.
10. **It's More Than a Trend—It's a Movement:** This isn't just another flashy wave of tech hype. Generative AI represents a foundational change in how we think, work, learn, and create. It's changing the *who*, *how*, and *how fast* of creation—and that's revolutionary.

That's too much flattery about Generative AI (it deserve it though). Now we will be taking a step deeper into generative ways, and let's talk about ...

1.2 What are LLMs?

Large Language Models (LLMs) are a special kind of machine learning model built to understand and generate human-like language. Unlike traditional models that are trained to do one specific task, LLMs are generalists—they can perform a wide range of tasks like summarizing text, translating languages, answering questions, or classifying documents, all with the same model.

LLMs are a type of model in Generative AI. Do remember that it's not the only model in the generative AI space!

These models are called “large” because they're trained on enormous datasets—think of billions of words—and have millions (sometimes even hundreds of billions) of parameters. This massive scale allows them to capture deep patterns, relationships, and meanings in text that make them appear surprisingly fluent and intelligent.

1.2.1 How LLMs Work

At the heart of every LLM is a **neural network**—a kind of algorithm loosely inspired by how the human brain works. The job of the model is to predict the next word (or more precisely, the next “token”) in a sentence based on what it has already seen.

Let's say you type:

“The weather today is really...”

The model might guess that the next word is “nice,” “hot,” or “cold” based on what it has learned from its training data.

This sequential prediction is what powers everything from writing poetry to answering questions. It doesn't “know” what's correct in the human sense—it simply makes the most statistically likely guess based on patterns in the data.

A. The Role of Attention

To make better predictions, LLMs use something called **attention mechanisms**, especially a version called **multi-head attention**, which was popularized in the research paper titled “*Attention Is All You Need*.” This technique helps the model figure out which words in a sentence are the most important, so it can weigh them more heavily in its predictions. For example, in the sentence:

“The book, which I borrowed from the library, was fascinating,”

Attention allows the model to link “book” and “fascinating,” even though they’re separated by a bunch of words. Attention helps models not just understand individual words, but also how they relate to one another across long sentences.

B. Token-by-Token Thinking

One thing to keep in mind is that LLMs generate their output **one token at a time**, without seeing the final answer in advance. That means they don’t “know” where a sentence is going—they’re just predicting each next word based on what’s already been written. This is a fundamental difference between how LLMs work and how traditional rule-based systems operate.

C. The Magic of Zero-Shot Learning

One of the coolest things about LLMs is their **zero-shot ability**. This means they can do tasks they’ve never been explicitly trained for. For instance, you might ask a language model to translate a sentence into a new language or write an email apology—and it can do it, even without having been directly trained for that exact job.

This happens because the model has seen such a diverse range of language patterns during training that it can generalize to new tasks. It’s like reading a ton of books in different genres and being able to write one yourself just by picking up on the patterns.

D. Transformers Under the Hood

Most LLMs are built using a special architecture called the **Transformer**, or a subset of it like the **GPT (Generative Pretrained Transformer)** model. Transformers are what make it possible to train these models at scale and process huge amounts of text efficiently. If neural networks are the brain, Transformers are the nervous system routing all the signals intelligently.

E. Pre-Trained, Not Perfect

LLMs are often trained in advance on a general-purpose dataset—this is called **pre-training**. These models are good at a wide range of tasks but might not be highly accurate for specific problems. Think of them as generalists who can do a lot of things “pretty well” but not necessarily with expert precision.

So if you want extremely accurate results—like 99% accuracy on a medical diagnosis—you might need to go a step further and **fine-tune** the model. This means giving the LLM more training on task-specific data to make it an expert in one area.

F. Size, Scale, and Parameters

LLMs come in many sizes—small, large, extra-large, or named with a number (like LLaMA-70B). The “B” here stands for **billion parameters**, which are the adjustable parts of the model that learn patterns. More parameters usually mean more power, but also more computing resources needed to run them.

That’s why loading these massive models directly into your computer’s memory can be a challenge. Some models can be hundreds of gigabytes in size! To deal with this, developers often use **APIs**—you send a request to a remote server, and it gives you the output. In this book, we’ll use OpenAI’s API for most hands-on examples.

1.2.2 LLMs Limitations

LLMs come with a few quirks and limitations:

- **They can be unpredictable.** Since the model generates text probabilistically, you might get slightly different responses for the same question each time you ask.
- **They need a prompt.** To perform any task, you have to give the model a clear instruction in natural language. This is called “prompting.”
- **They have word limits.** Each model has a limit on the maximum number of tokens (a mix of words and punctuation) it can process in one go.
- **Tendency to Hallucinate:** LLMs can generate factually incorrect or fabricated content, especially when extrapolating beyond their training data or handling out-of-distribution queries. Gaps or inconsistencies in the training corpus may lead the model to produce confident but misleading responses.

As we are now covered with the basics, Let's jump onto understanding AI Agents in detail in the next chapters.

Chapter 2: What are AI Agents

The IT department of every company is going to be the HR department of AI agents in the future

- Jensen Huang

As major companies like Meta and Salesforce have noted, 2025 is shaping up to be the era of AI agents. Building on this momentum, this chapter explores what AI agents are, how they function, and their growing impact across industries.

2.1 What are Agents?

In Generative AI, an agent is like a virtual assistant or a digital entity designed to perform specific tasks, make decisions, or solve problems. It usually uses a combination of LLM + some third-party APIs, databases, or even special functions to complete a task. It can also be enabled for task execution and decision-making capabilities.

2.1.1 AI Agents vs LLMs?

To better understand the distinction between AI agents and large language models (LLMs), let's break it down with a practical example: email generation.

Email Generation Using an LLM

When you use an LLM to generate an email, here's what happens:

You Provide a Prompt:

For example, you might ask the LLM:

"Write an email thanking a client for a meeting and attach a proposal."

LLM Generates Text:

The LLM produces the following output:

```
Subject: Thank You for the Meeting

Dear [Client Name],

I wanted to take a moment to thank you for the time you took to meet with us today. It was a
pleasure discussing [specifics of the meeting], and we're excited about the opportunity to w
ork together.

As promised, please find attached the proposal for your review. If you have any questions or
need further clarification, don't hesitate to reach out.

Looking forward to hearing from you.

Best regards,
[Your Name]
```

Now, as you must have noticed, it just generates the text. A lot of work is still pending for you to

1. *Copy the email text.*
2. *Open your email client.*
3. *Fill in the recipient's details.*
4. *Attach the proposal manually.*
5. *Click send.*

Using Email Agent : The email agent will not just output the text but also do the remaining steps (if enabled) to complete the task.

That's not the case with AI Agents ...

AI agents go beyond just generating content—they execute tasks by interacting with the tools they're connected to. For example, a scheduling agent doesn't just suggest meeting times; it connects with your calendar API, finds available slots, and books the meeting automatically. The real power of AI agents lies in their ability to bridge the gap between generating ideas and turning them into actions, all without human intervention.

2.1.2 How Do AI Agents Execute Tasks?

While AI agents internally use LLMs, their ability to interact with APIs, databases, or other tools is enabled through customized code. Here's how this interaction typically works:

Example: *Weather Agent*

User Query: *"How is the weather in New Delhi?"*

Agent's Workflow:

1. **Step 1:** The LLM within the agent analyzes whether a tool (e.g., weather API) is required. It may use a system prompt like: *"Check whether the user is asking about the weather of a specific place."*

2. **Step 2:** If a tool is required, the agent extracts the relevant entity (e.g., “New Delhi”) from the user’s prompt.
3. **Step 3:** A request is sent to the weather API (tool) with the extracted entity as a parameter.
4. **Step 4:** The API returns data (e.g., temperature, humidity), which the LLM uses as context to generate a final answer.
5. **Step 5:** The agent delivers the response: *“The weather in New Delhi is 28°C with moderate humidity.”*

This process of letting the LLM decide when to use a specific function or service is known as **tool calling**. It’s like giving the agent access to a toolbox and letting it pick the right tool based on the task at hand—whether it’s calling a weather API, searching the web, sending an email, or querying a database. Tool calling is what allows the agent to move from just generating ideas to actually performing real actions.

Some examples of AI agents

Agent Type	Description
Weather Agent	LLM + Weather API to fetch real-time weather data.
Perplexity AI	LLM + Internet access for real-time information retrieval.
Customer Support	LLM + Database access to provide personalized responses.
Spanish Agent	LLM + Special internal prompt to answer queries only in Spanish.

2.1.3 Why Are AI Agents Important?

AI agents significantly enhance the utility of LLMs, transforming them from text-generation tools into intelligent assistants. Here’s why they matter:

1. **Task Automation:** Agents automate complex, multi-step workflows, going beyond simple content generation.
2. **System Integration:** They connect AI to external tools (e.g., APIs, calendars) to perform real-world actions.
3. **Decision-Making:** Agents can autonomously choose and execute tasks without constant user input.
4. **Real-Time Action:** Agents interact with live data, making dynamic decisions and updating in real time.
5. **Personalization:** Agents remember user preferences and adapt based on past interactions.
6. **Reduced Human Intervention:** Agents handle tasks end-to-end, minimizing manual effort.

7. **Scalability:** Agents can process repetitive, large-scale tasks efficiently, freeing up human resources.

2.2 Popular AI Agent Frameworks

Since the AI agent hype, there have been a number of AI agent frameworks that have been released on the internet, which you can't even trace out. To ease things up for you, these are the famous and the most popular AI agent frameworks that you should know:

1. **LangGraph:** Developed by LangChain, this framework enables building stateful, graph-based AI agent systems. It specializes in handling cyclic workflows and multi-agent coordination with persistent memory. Ideal for complex applications like automated research, customer service bots, and long-running decision processes where agents need to maintain context.
2. **CrewAI:** A Python-based framework focused on creating specialized AI teams where each agent has defined roles (e.g., analyst, writer, reviewer). It features built-in task delegation, parallel processing, and easy integration with external tools. Perfect for content creation pipelines, data analysis workflows, and other structured multi-step processes.
3. **AutoGen:** Microsoft's framework for creating conversational AI systems with advanced multi-agent capabilities, specific to coding tasks. Supports customizable LLM backends, tool integration, and complex dialog patterns. Commonly used for developing sophisticated chatbots, coding assistants, and collaborative problem-solving systems.
4. **Google-ADK (Agent Development Kit):** Google's comprehensive toolkit for building enterprise-grade, multimodal AI agents. Offers tight integration with Google Cloud services and specialized APIs for handling real-time data streams. Designed for large-scale deployments in business automation, smart assistants, and data processing applications.
5. **OpenAI Agent SDK:** A lightweight development kit for creating function-calling AI agents powered by OpenAI models. Simplifies connecting LLMs to external APIs, databases, and tools. Best for building responsive agents handling tasks like information retrieval, data processing, and simple automation workflows.
6. **Microsoft TinyTroop & Magentic-One:** Two specialized frameworks - TinyTroop simulates interactions between multiple AI personas for testing scenarios, while Magentic-One focuses on reliable code generation. Together, they support development of simulation environments and AI-powered developer tools with an emphasis on stability.
7. **HuggingFace smolAgents:** A collection of lightweight, modular agent components optimized for small-scale AI applications. Enables rapid prototyping and experimentation with HuggingFace models. Particularly useful for researchers and developers working on constrained hardware or needing simple agent implementations.
8. **AWS Multi-Agent Orchestrator:** Amazon's cloud-native solution for coordinating distributed AI agents at scale. Integrates seamlessly with AWS services like Lambda and Bedrock to manage complex workflows. Tailored for enterprise applications in logistics, IoT systems, and large-scale automation projects.

We won't be discussing these frameworks in detail in this book. Why?

Think of them as early versions—AI agents 1.0. With the emergence of the Model Context Protocol (MCP), we're on the cusp of a new wave of AI agent development. This book aims to prepare readers for the next generation of AI agents—those powered by MCP—by building a strong foundational understanding of the evolving landscape.

2.3 Potential Challenges with AI Agents

While AI agents offer incredible potential, some challenges come with their implementation.

1. First, agent systems depend heavily on the ***reliability of external tools***, such as APIs or databases, which means any downtime or error from those tools can cause the agent to fail.
2. Secondly, agents often need ***careful design*** to ensure that their decision-making abilities align with user expectations, particularly in critical tasks like financial transactions or sensitive data handling.
3. Lastly, maintaining ***security and privacy*** when agents access multiple systems is another important consideration.
4. A ***lot of coding*** and usage of multiple frameworks is required to build out complex systems, making it completely unreachable for non-programming folks.
5. ***No standardization*** is present if you are using AI agent frameworks. Every framework works in a different manner, and hence a lot of segmentation.

Concluding, AI agents look to be a promising concept, and if you've been listening to news, you might have heard of this concept way before. Though there are certain lacunae present in this concept, it's still very useful. With the incoming Model Context Protocol (MCP) that we'll discuss in the next chapter, you'll know how MCP is the AI agent 2.0.

Chapter 3: What is Model Context Protocol?

To MCP Or Not To MCP, That Is The Question

- Sundar Pichai

Now that we've warmed you up with the essentials, we're diving into the core of this book—the Model Context Protocol (MCP). This is what the book was built around: a topic that's gaining traction now and is likely to remain central throughout the year, or maybe decades to come.

Before exploring MCP, it's important to understand what a protocol is.

3.1 What is a Protocol?

Think of protocols as the digital equivalent of social etiquette—like saying "hello" before a conversation or shaking hands when you meet someone. Computers also need a common set of rules to communicate properly. These rules are called **protocols**.

A. Everyday Examples of Protocols

1. **HTTP/HTTPS**: This is how your browser talks to websites. HTTP loads the page, while HTTPS adds encryption for safety—perfect for online banking or shopping.
2. **TCP/IP**: The postal service of the internet. It breaks up your data, sends it across the world, and reassembles it correctly at the other end.
3. **SMTP**: The protocol behind sending emails. When you hit "Send" in Gmail, SMTP makes sure your message gets to the right inbox.
4. **Wi-Fi (IEEE 802.11)**: The rules your phone and router follow to connect wirelessly.

Each of these keeps our online experiences running smoothly—whether we're browsing memes or sending mission-critical emails.

B. Why Protocols Matter

Without protocols, digital communication would be like two people yelling in different languages across a noisy room. But with protocols, we get:

1. **Consistency:** Devices know what to expect.
2. **Reliability:** Messages get through in one piece.
3. **Security:** Encryption keeps hackers at bay.
4. **Efficiency:** No guessing games—data moves fast and clean.

Think of it like driving: everyone follows the same traffic rules, so there are fewer accidents and less chaos.

C. What If Protocols Didn't Exist?

Now let's play doomsday for a sec. What happens if we pull out protocols out of the equation?

1. **No HTTP?** Websites wouldn't load. Typing "google.com" would do absolutely nothing.
2. **No SMTP?** Emails vanish into the void—or worse, show up as unreadable nonsense.
3. **No TCP/IP?** Say goodbye to stable video calls, smooth downloads, or messages that make sense.
4. **No Wi-Fi/Bluetooth?** Your phone and router would be total strangers.
5. **No HTTPS/SSL?** Every password or credit card number you send would be out in the open like writing your bank PIN on a billboard.

It would be like driving on roads with no signs, no lanes, no traffic lights. Total digital anarchy.

You might have understood how important protocols are, but unfortunately, the world of AI agents was operating without protocols before Model Context Protocol came in. Hence, you might have understood why there was such a ruckus all over the place.

MCP is the protocol for AI agents to connect with external tools and APIs.

3.2 What is Model Context Protocol?

Imagine trying to plug a USB device into a port that doesn't fit—frustrating, right? Before MCP, integrating AI models with various tools and databases felt just like that. Each connection required custom code, making the process cumbersome and error-prone. MCP,

introduced by Anthropic in late 2024, serves as the **"USB-C" for AI integrations**—a universal, standardized protocol that simplifies and streamlines these connections.

3.2.1 The Need for MCP

Before MCP came onto the scene, integrating AI agents with external systems was a manual and often cumbersome process. Developers had to:

1. **Build Custom Connectors:** Each external tool or API required its own bespoke integration. This meant writing specific code to handle the communication between the AI agent and the external service.
2. **Handle Authentication Individually:** Every service had its own authentication mechanism, necessitating separate handling for API keys, OAuth tokens, and other security protocols.
3. **Manage Data Formats and Protocols:** Different services used various data formats (like JSON, XML) and communication protocols (like REST, SOAP), requiring additional code to parse and format data appropriately.

This approach was not only time-consuming but also led to a maintenance nightmare, especially as the number of integrations grew.

And that's why model context protocol is gaining traction all over the internet.

This is also the reason why we didn't discuss frameworks like Langgraph, Crew-AI, etc., because in that case, you need to customize every AI tool depending on the framework, and there's no standardisation. With MCP coming in now, the interaction between AI agents and tools has become standardised, and you don't need to learn multiple frameworks or standards to integrate third-party extensions to LLMs.

3.2.2 How MCP Works

At its heart, MCP operates on a **client-server model**, which might sound techy—but it's actually pretty intuitive. Think of it like a waiter (client) and a kitchen (server). The waiter takes your order and brings it to the kitchen, where the magic happens. Then they bring your meal (response) back to you. MCP works the same way for AI systems.

A. MCP Clients

These are the AI applications or agents—think ChatGPT, a code assistant, or a customer support bot.

1. **Their Role:** Clients are the ones *requesting* data or actions.

2. **What They Do:** They ask servers for things like, “Hey, can I get all unresolved GitHub issues?” or “Can you run this SQL query?”
3. **Why It Matters:** The client acts as the bridge between the AI model and the real world—it knows what it needs, but it depends on the server to get it.

B. MCP Servers

Now, here’s where the tools and data live. MCP servers are like plug-and-play modules that expose:

1. **Tools:** These are functions the AI can invoke (e.g., `send_email`, `list_calendar_events`, or `fetch_customer_data`).
2. **Resources:** Static or dynamic data like CSV files, emails, vector databases, or even codebases.
3. **Prompts:** Templates that help guide how the AI should interpret or present the information.

In short, MCP servers *serve* the stuff the AI needs to get work done.

C. Standardised Communication

Here’s where the real magic of MCP kicks in.

1. **Uniform Language:** The client and server don’t just shout at each other in random formats. They use a standardized protocol—typically something like JSON-RPC. This ensures both sides “speak the same language.”
2. **No Guesswork:** This standardization means tools and data can be discovered, used, and invoked by the AI dynamically without developers writing messy, one-off integrations.
3. **Plug-and-Play Vibes:** You can switch out one MCP-compatible server for another (e.g., swap GitHub for GitLab) and the client won’t even flinch.

What is JSON-RPC?

JSON-RPC stands for **JavaScript Object Notation - Remote Procedure Call**. It’s a lightweight remote procedure call (RPC) protocol encoded in JSON. Translation? It’s a way for one program to call functions on another system (usually over a network) using plain ol’ JSON to pass data back and forth.

D. Putting It All Together — A Real-World Example

Let's say you're using an AI assistant to help with project management.

1. **Connection:** The AI app (client) connects to an MCP server set up for GitHub.
2. **Tool Use:** The model uses the `list_issues` tool to fetch open issues in a repo.
3. **Data Return:** The server grabs the issue data and returns it as a resource.
4. **Prompt Time:** The AI uses a predefined prompt to analyze and prioritize the issues.
5. **Follow-Up Action:** It invokes another tool (like `create_task`) to push top issues into work management platforms like Asana.

You didn't write a single line of integration code. The client and server simply followed the MCP playbook.

E. Why This Architecture Rocks

1. **Modularity:** Build Once, Use Everywhere.
The architecture allows you to develop tools once and deploy them across multiple applications and environments. Whether you're adding new services or switching clients, the modular design ensures minimal disruption and maximum reusability.
2. **Scalability:** Grow Without Growing Pains
Multiple teams can independently manage their own MCP servers—for example, separate ones for data, DevOps, or CRM—without stepping on each other's toes. Meanwhile, any AI application can seamlessly connect to and use these services as needed.
3. **Security & Control:** Keep It Tight and Traceable
Each server maintains full control over which tools and data it exposes to the outside world. This ensures strong security boundaries and makes all interactions auditable, giving organizations better governance over AI-driven workflows.
4. **Future-Proof:** Designed for What's Next
As the AI landscape evolves and new tools emerge, this architecture ensures you won't need to rebuild integrations from scratch. You can plug in innovations without overhauling your existing systems, saving time and resources.

Now you see how important MCP is, and that's why this book has been written. The moment we read about Model Context Protocol, we were just blown away.

3.3 Key Components of MCP

In this subsection, we will be discussing the key components of the Model Context Protocol Server.

3.3.1 Resources: What the AI Knows

Resources are the data the AI can “read” and use for reasoning. Think of these as the AI’s memory or context—it could be a CSV file of sales numbers, a batch of emails, or the result of a database query. Whether it’s a static document like a PDF or a dynamic stream like a live API response, if it provides information, it’s a resource.

For example, imagine you’re building an AI assistant for recruiters. The resumes your team uploads? Those become resources. Now the model can comb through them to answer questions like, “Who has experience with Kubernetes and Python?” without you lifting a finger.

This shift—from manual copy-pasting into the prompt to structured, queryable context—is one of the game-changers MCP brings to the table.

3.3.2 Tools: What the AI Can Do

Resources are external knowledge bases that give the AI something to refer—but what if you want it to act? That’s where **Tools** come in. These are the model’s arms and legs, allowing it to trigger real-world functions like creating a task, sending an email, querying a database, or even executing shell commands.

So instead of just saying, “This bug looks critical,” your AI assistant could take it a step further and say, “I’ve filed the bug report in GitHub for you.” That’s the magic of tools.

Each tool is defined by developers and made discoverable through an MCP server. Once that tool is exposed, any compatible AI agent can find it, understand how to use it, and call it dynamically—no custom code required. You can think of tools as the AI’s built-in “API menu”—ready to be used whenever it makes sense.

3.3.3 Prompts: What the AI Should Say

Now, just because the AI has access to data (resources) and actions (tools), that doesn’t mean it’ll know what to do with them. That’s where **Prompts** come in.

Prompts are like the AI’s script. They tell the model how to behave, what task to perform, and how to structure the output. MCP takes prompts beyond one-off manual instructions by turning them into reusable templates that can be filled in dynamically.

For example, instead of crafting a new prompt every time, you could have one that says:
“Summarize the following customer support tickets and tag each as a bug, feature request,

or question.”

Then, at runtime, the model gets the actual tickets (as a resource), plugs them into the prompt, and runs the analysis.

Prompts bring consistency, modularity, and intelligence to AI workflows—especially when paired with tools. You could summarize GitHub issues, then use another tool to automatically file tasks based on the model’s suggestions—all in one smooth flow.

3.3.4 Real-World Use Cases

Let’s say you’re using an AI-powered productivity agent at work. You ask it, *“Can you clean up my project backlog and assign tasks to the right people?”*

Behind the scenes, here’s what happens:

1. It pulls all your unassigned GitHub issues (resources).
2. It uses a prompt to prioritize and categorize them based on urgency and owner.
3. Then it runs a few tools to create tickets in Asana, notify your team on Slack, and close out stale issues.

All of that happens because the AI has the context (resources), the intelligence (prompts), and the capability (tools) to get it done.

3.4 MCP = AI Agents 2.0?

Before MCP came along, building AI agents felt like assembling IKEA furniture blindfolded—***technically possible, but frustrating*** and definitely not for the faint of heart. You had to hardwire tools into your model pipeline, design complex JSON schemas, maintain flaky integrations, and handle a mess of edge cases. Every new API? A new round of glue-code nightmares.

Even worse, it was mostly ***developer-only territory***. If you weren’t comfortable with Python, Docker, and Postman, good luck trying to wire your AI agent to pull data from your CRM or run database queries.

So yeah—tool calling was cool when it first dropped. But let’s be real: it wasn’t scalable, it wasn’t user-friendly, and it definitely wasn’t democratized.

...here comes MCP

Now imagine someone walked in and said,

"Hey, what if we just built a universal plug-and-play system for AI tools and data? One where models could discover resources, invoke APIs, and generate dynamic prompts without you lifting a finger every time?"

That's **MCP** in a nutshell.

MCP makes AI agents more modular, scalable, and composable. It gives us **AI Agents 2.0**—where:

1. Tools aren't hardcoded, they're *discoverable*.
2. Prompts aren't duplicated, they're *centralized and templated*.
3. Context isn't copy-pasted, it's *streamed in real-time*.
4. And agents aren't dev-locked—they're *user-accessible*.

In other words, the barrier to entry drops from DevOps engineer to curious knowledge worker with a laptop.

Here's the magic: MCP servers can run **locally**. That means no cloud costs, no creepy data surveillance, and no monthly subscriptions. Want your AI agent to pull up your meeting notes, summarize last week's emails, and generate a to-do list from a PDF? You can do all of that with MCP—on your laptop, for free.

This flips the entire AI dev ecosystem on its head.

Before: "Let's spin up an OpenAI function-calling endpoint, wrap it in LangChain, and deploy it to a web dev framework like Vercel."

After: "Let's run a local MCP server and just let the agent figure it out."

See the difference?

1. **From Code to Clicks:** No more coding every tool. Just expose it via MCP, and your agent can use it.
2. **From Static to Dynamic:** Agents can discover tools and prompts at runtime—no rebuild needed.
3. **From Silos to Ecosystems:** Anyone can build, share, or remix MCP-compatible components.

4. **From Experts to Everyone:** You no longer need to be a dev to run advanced AI workflows.

So yeah—MCP is more than just a spec. It's the **platform shift** that makes AI agents mainstream, modular, and accessible. Just like how HTML + HTTP made the internet usable, MCP is the protocol layer that turns AI into a real platform—not just a playground.

Wanna build your first MCP agent or see what a local setup looks like? Let's do it.

3.5 Basic Setup

Before we jump onto installing different MCP servers, a basic setup is mandatory

1. Install Claude Desktop app and sign in : <https://claude.ai/download>
2. Install Python 3.10 or above : <https://www.python.org/downloads/>
3. Install Docker : <https://www.docker.com/products/docker-desktop/>
4. pip install uv (a python package)
5. Node.js : <https://nodejs.org/en/download>
6. Path to Claude's config.json: **Claude Desktop App→File (top left)-->Settings→Developer→Edit Config**

Don't feel overwhelmed by names like Docker or UV. You won't require in depth knowledge of these tools

Also, a few things to note as well:

1. This entire book assumes that you are **using MCP with Claude AI Desktop app** (free to use for some hits) **in Windows**. MCP Servers can also be used with other AI clients like Cursor AI, Windsurf etc. and even with local LLMs. In that case, just the location for the config.json file (that we would be configuring for MCP servers) will change. Rest of the process remains the same.
2. As the world of Generative AI is evolving fast, it may be the case that **some MCP servers may go obsolete or the installation guide may change** while we publish this book (Around June"2025). In case of any errors, kindly drop a mail to the authors of the book, or refer to the git repo link mentioned for every MCP server.
3. If any of the MCP servers doesn't work, kindly retry with providing full path for commands in any system, be it windows, linux or Mac.

The rest of the book will deal with how to get started with Model Context Protocol, running them locally and using APIs and different useful MCP servers that everyone should know. Get your backpack ready for this crazy adventure. We'll begin by exploring how MCP servers are accessible to anyone and how they enhance productivity by integrating seamlessly with familiar tools like Gmail, PowerPoint, and more in the next section

Part 2: Productivity MCPs

Chapter 4: GSuite MCP server

Work Smarter, Not Harder: AI Brings GSuite to Life!

4.1 Introduction

The GSuite Model Context Protocol (MCP) is a convenient tool that allows your AI assistant to handle your Gmail, Google Calendar, Google Maps, etc through simple, natural language commands. Talking about the Gmail automation specifically, whether you need to check your inbox, send emails, or organize your schedule, Gmail MCP makes it easy without the need for manual navigation.

Though in this chapter, we will demonstrate just Gmail integration. Rest of the integrations are also similar.

Using GSuite MCP, your assistant can perform tasks like listing recent emails, conducting advanced searches, sending new messages, modifying email labels, viewing upcoming calendar events, creating new events with attendees, and updating existing ones.

4.1.1 GSuite MCP tools (Gmail)

GSuite MCP for Gmail offers a suite of tools that empower your AI assistant to manage your Gmail inbox efficiently using natural language commands. Here's a concise overview of the available tools:

1. **list_emails** : Retrieve recent emails from your inbox with optional filtering.
Example: "List the latest 5 unread emails."
2. **search_emails** : Perform advanced email searches using Gmail's query syntax.
Example: "Search for emails from 'client@example.com' with attachments."
3. **send_email**: Compose and send new emails, including support for CC and BCC fields.
Example: "Send an email to 'team@example.com' with the subject 'Meeting Update' and CC 'manager@example.com'."
4. **modify_email**: Update email labels to archive, trash, or mark emails as read/unread.
Example: "Mark the email from 'newsletter@example.com' as read and archive it."

4.1.1 Real-World Applications

Amit, a customer support lead, starts his day by checking for any urgent client emails that need immediate attention. To streamline his workflow, he turns to GSuite MCP for Gmail.

He prompts his assistant:

- *"List the latest emails from the inbox."*
- *"Search for emails from 'client@example.com' received this week."*
- *"Send an email to 'client@example.com' with the subject 'Follow-up on Support Ticket' and include the latest update in the body. CC 'manager@example.com'."*
- *"Mark the email from 'client@example.com' with the subject 'Issue Resolved' as read and archive it."*

Within minutes, the assistant retrieves the necessary emails, sends the follow-up message, and organizes the inbox accordingly. Amit efficiently manages his communications, ensuring prompt responses and a well-organized inbox.

4.2 Step-by-Step Installation

4.2.1 Pre-requisites

1. Install Node.js version 14 or higher.
2. Go to the Google Cloud Console (<https://console.cloud.google.com/>).
3. Create a new project or select an existing one.

Select a project

 [New project](#)

 Search projects and folders

RecentStarredAll

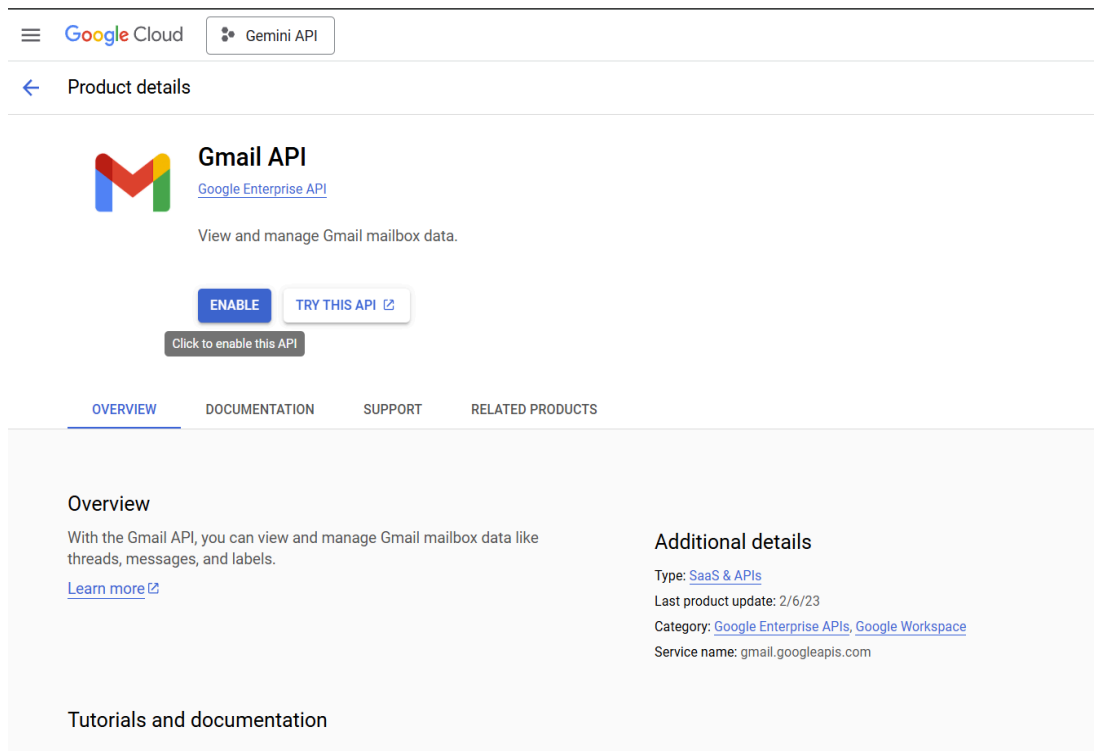
	Name	ID
✓ ☆ 	Gemini API 	gen-lang-client-0959198452
☆ 	ProfessorBot 	professorbot-xhqqwr

Cancel

Project list in Console

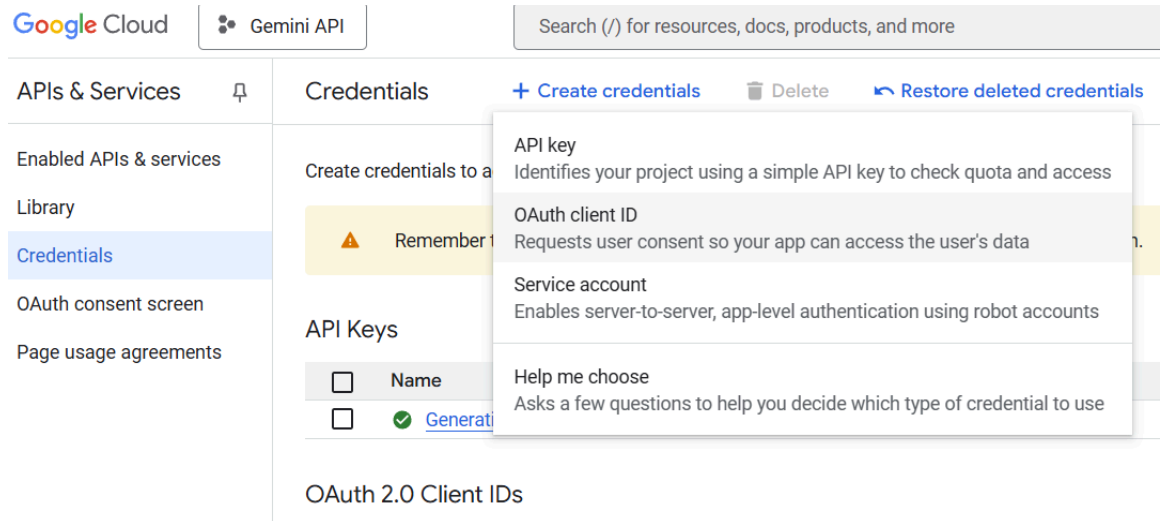
4. Navigate to **"APIs & Services" > "Library"**. Search for and enable the Gmail API.

Note : *If you wish to use other GSuite MCPs like Google Calendar or Google Maps, search for those APIs as well and enable similarly*



Enable Gmail API

5. **Setup OAuth 2.0 credentials:** Go to the Credentials page in the API library & click on *Create Credentials and select OAuth client ID*



Create OAuth client ID

Google Cloud My Project Search (/) for resources, docs, products, and more

Google Auth Platform / Clients / Create client

Overview
Branding
Audience
Clients
Data Access
Verification Center

Create OAuth client ID

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

- Web application
- Android
- Chrome Extension
- iOS
- TVs and Limited Input devices
- Desktop app
- Universal Windows Platform (UWP)

Select Web Application from the dropdown.

- Set **"Authorized redirect URIs"** to include: <http://localhost:4100/code>

[←](#) Create OAuth client ID



The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins [?](#)

For use with requests from a browser

[+ Add URI](#)

Authorized redirect URIs [?](#)

For use with requests from a web server

URIs 1 *

[+ Add URI](#)

Note: It may take 5 minutes to a few hours for settings to take effect

[Create](#)

[Cancel](#)

Redirect URI

7. Note down the Client ID and Client Secret. We will need these details in the later stage.

4.2.2 Steps

1. Clone and Install the MCP Server (run the below commands in Command-Line, cmd)

```
git clone https://github.com/epaproditus/google-workspace-mcp-server.git
cd google-workspace-mcp-server
npm install
```

2. **Create OAuth Credentials** - In the root directory, create a file named `credentials.json` with the following content:

```
{
  "web": {
    "client_id": "YOUR_CLIENT_ID",
    "client_secret": "YOUR_CLIENT_SECRET",
    "redirect_uris": ["http://localhost:4100/code"],
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token"
  }
}
```

`credentials.json`

3. Obtain Refresh Token by running the below command inside the repo

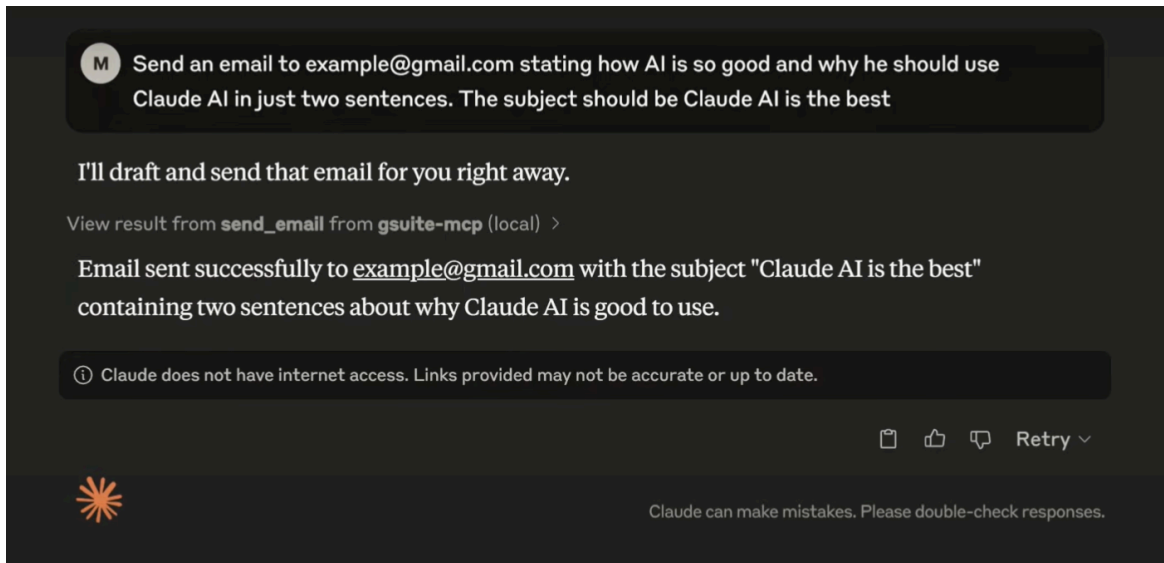
```
node get-refresh-token.js
```

4. This will open your browser for Google OAuth authentication and request for some permissions. Approve them and save the credentials to **token.json** :
5. Add the server to Claude config json file

```
{
  "mcpServers": {
    "google-workspace": {
      "command": "node",
      "args": ["/path/to/google-workspace-server/build/index.js"],
      "env": {
        "GOOGLE_CLIENT_ID": "your_client_id",
        "GOOGLE_CLIENT_SECRET": "your_client_secret",
        "GOOGLE_REFRESH_TOKEN": "your_refresh_token"
      }
    }
  }
}
```

`claude_desktop_config.json`

6. Restart Claude (end from Task Manager). On restarting, you should the MCP tools in Claude message section



And that's it, you've now added Gmail MCP in your Claude AI. Similarly, you can add these settings to Cursor.ai, Windsurf or Cline and let AI operate your Gmail account. Also, if you wish to use other GSuite services (like Google Calendar etc.), this MCP server should support it too.

Chapter 5: Excel MCP server

Your Spreadsheet Pain Ends Here!

5.1 Introduction

The Excel Model Context Protocol (MCP) enables your AI assistant to create, edit, and analyze Excel spreadsheets through natural language commands. Whether you're building a new budget sheet, generating pivot tables, or formatting data, Excel MCP allows your assistant to handle these tasks seamlessly—**no manual input required**.

In the previous chapter, we explored how Gmail MCP enables your assistant to manage emails efficiently. Now, with Excel MCP, your assistant becomes a proficient spreadsheet manager, capable of handling complex data tasks with ease.

5.1.1 Excel MCP Tools

Excel MCP isn't just about managing spreadsheets—it comes equipped with smart tools that let your assistant take over the tedious parts of Excel work. Here's what it can do:

1. **read_sheet_names** : Quickly lists all the sheet names in an Excel file. Perfect for getting an overview of your workbook's structure.
2. **read_sheet_data**: Reads data from a specified range in an Excel sheet, supporting pagination for large datasets. Ideal for analyzing specific sections without loading the entire sheet.
3. **read_sheet_formula**: Extracts formulas from a specified range in an Excel sheet, also with pagination support. Useful for auditing calculations or understanding complex formula setups.
4. **read_sheet_image (Windows only)**: Captures data as an image from a specified range in an Excel sheet. Handy for visual references or sharing snapshots of your data.
5. **write_sheet_data**: Writes data to a specified range in an Excel sheet. Great for automating data entry or updating reports with new information.
6. **write_sheet_formula**: Inserts formulas into a specified range in an Excel sheet. Perfect for setting up calculations without manual input.

5.1.2 Real-World Applications

Imagine you're working on a time-sensitive report and need to quickly update an Excel file with new data, reorganize the content, and add key calculations. Instead of doing it manually, you open up the Excel MCP server and let your assistant handle the heavy lifting.

You start by giving a few simple commands:

- *"List all sheet names in 'Q2_Sales_Report.xlsx'."*
- *"Read the data from the 'SalesData' sheet, cells A1 to D50."*
- *"Write the updated sales numbers to the 'Summary' sheet, starting at A1."*
- *"Add formulas in the 'Summary' sheet to calculate total sales by region."*

The assistant springs into action, pulling the required data, updating the right sections, and inserting the necessary formulas — all in just a few minutes. You quickly review the final output, satisfied with the accuracy and formatting, and head into your meeting feeling prepared and confident.

5.2 Step-by-Step Installation

5.2.1 Steps

1. Node.js 20.x or later installed in your system.
2. Set the following config in your claude config json file.

For Windows platform:

```
{
  "mcpServers": {
    "excel": {
      "command": "cmd",
      "args": ["/c", "npx", "--yes", "@negokaz/excel-mcp-server"],
      "env": {
        "EXCEL_MCP_PAGING_CELLS_LIMIT": "4000"
      }
    }
  }
}
```

Claude_desktop_config.json

For Other platforms:

```
{
  "mcpServers": {
    "excel": {
      "command": "npx",
      "args": ["--yes", "@negokaz/excel-mcp-server"],

```

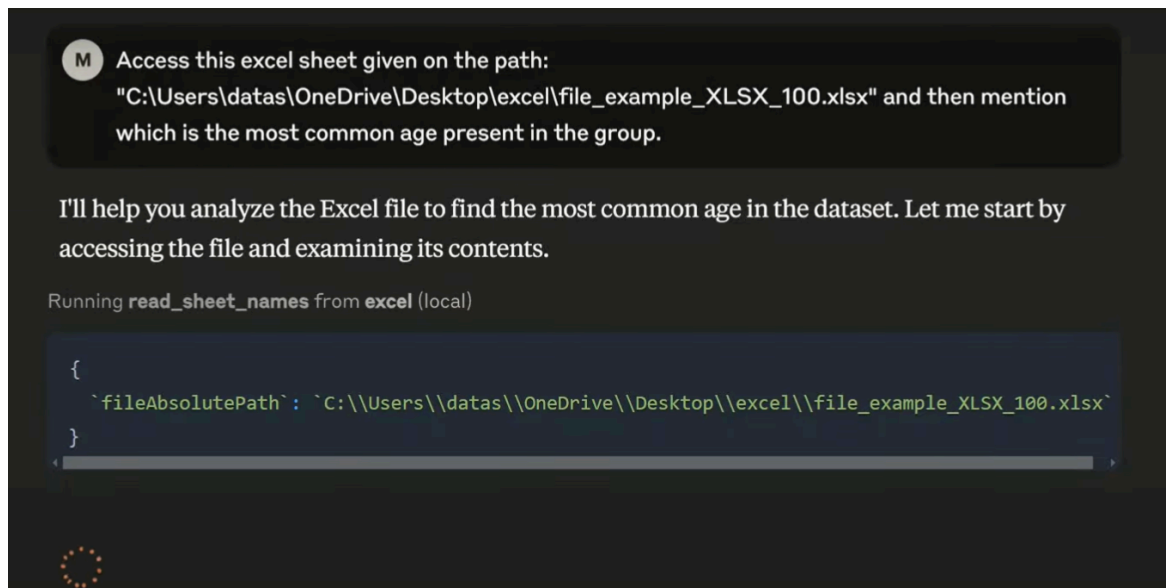
```

    "env": {
      "EXCEL_MCP_PAGING_CELLS_LIMIT": "4000"
    }
  }
}

```

claude_desktop_config.json

After Gmail, now you should have enabled Excel MCP as well. Good job !



Definitely give this a try—you'll thank us for this AI trick once you see how much smoother Excel becomes. We've already automated Gmail and Excel. What's next? it's **PowerPoint for your presentations!**

Chapter 6: PowerPoint MCP server

No More Slide Stress—MCP Brings your PPTs to life!

6.1 Introduction

PowerPoint Model Context Protocol (MCP) is a handy tool that lets your AI assistant build and edit PowerPoint slides for you—just by asking. Whether it's starting a brand-new presentation, adding charts, dropping in section headers, or even generating images for your slides, PowerPoint MCP helps you get it all done without the hassle of manual edits.

Your assistant can create slides with titles and content, tables, charts based on your data, or even picture slides with captions. It can also open existing decks, make updates, and save everything neatly when you're done.

In the last section, we explored how Excel MCP can help crunch numbers and manage data smarter. Now we're stepping into the world of presentations—with PowerPoint MCP, your assistant becomes your personal slide designer, turning your ideas into clean, clear presentations in no time.

6.1.1 PowerPoint MCP Tools

PowerPoint MCP isn't just about making slides—it comes packed with smart tools that let your assistant take over the boring parts of presentation-making. Here's what it can do:

1. create-presentation

Kick off a brand-new PowerPoint presentation by simply giving it a name. Great for starting fresh projects without opening PowerPoint manually.

2. add-slide-title-only

Add a simple title slide to your deck. Just say what the title should be, and it drops it in as your opening slide.

3. add-slide-section-header

Want to break your slides into neat sections? This tool creates a slide with a big bold section header (and an optional subtitle if you want).

4. add-slide-title-content

Perfect for content-heavy slides. Add a title and some body text, and this tool builds a clean layout for it automatically.

5. add-slide-title-with-table

Need to show tabular data? This adds a slide with a title and a table that's built

from your provided data—no manual formatting needed.

6. add-slide-title-with-chart

Visualize your data by adding a chart. Just give it a title and the data, and the assistant figures out the best chart type to use.

7. add-slide-picture-with-caption

Insert an image with a caption under it. You give the title, caption, and image path, and it makes a slide with everything laid out nicely.

8. open-presentation

Want to continue editing a previous deck? This opens up an existing presentation and auto-saves a backup copy to keep things safe.

9. save-presentation

Once all your slides are in place, this tool saves your presentation file with all updates. Just provide the name and you're done.

10. generate-and-save-image

Need visuals? This tool uses AI to generate an image based on your prompt and saves it for later use in a slide. Super handy for custom graphics.

6.1.2 Real-World Applications

Ananya is a project manager who just got a last-minute request from her boss—she needs to present a quarterly update in less than 30 minutes.

With no time to waste, Ananya turns to PowerPoint MCP. She quickly prompts her assistant:

- *“Create a presentation called ‘Q1 Project Summary’.*
- *Add a title slide. Add a section header ‘Progress Overview’.*
- *Then add a slide with the title ‘Milestones Completed’ and list out key achievements.*
- *Create a chart from this progress data.*
- *And finally, add a picture slide with our team photo and a short thank-you note.”*

Within minutes, the assistant builds the full deck. Ananya just reviews it and walks confidently into the meeting—presentation ready, stress-free.

6.2 Step-by-Step Installation

6.2.1 Pre-requisites

1. You need to set the following environment variable for the [ForgeUI API](#) to generate images. *This is not necessary though. If not used, AI won't be able to add any custom images.*

```
"env": {  
  "SD_WEBUI_URL": "http://your-sd-webui-url:7860",  
  "SD_AUTH_USER": "your-username", // Optional: if authentication is enabled  
  "SD_AUTH_PASS": "your-password", // Optional: if authentication is enabled  
}
```

What is ForgeUI you ask?

Think of ForgeUI as a smart design assistant that helps build and customize user interfaces using AI. It taps into powerful image generation tools like Stable Diffusion —a system that creates high-quality images from simple text descriptions. With ForgeUI, you can turn plain text into visuals like buttons, layouts, or diagrams, making it easier to design polished interfaces without starting from scratch.

Just remember to put the credentials in the `claude_desktop_config.json` file as shown in the next section.

2. Setup uv package installer. This is a simple ***pip install uv***
3. You need to specify a folder where all presentations and images will be saved. This can be any folder in your computer.

6.2.2 Steps

1. Clone the git repo:

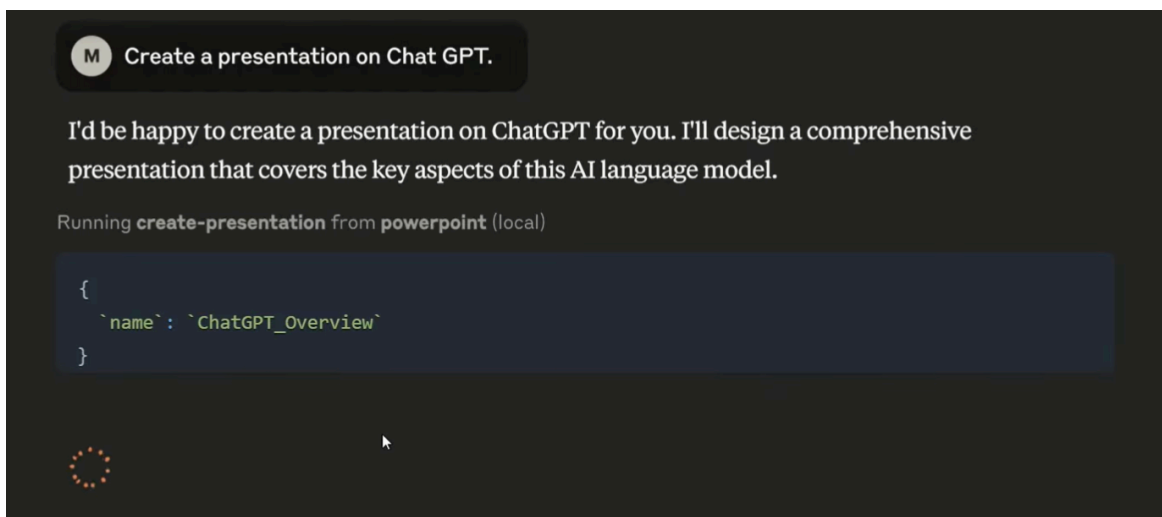
```
git clone https://github.com/lchigo3766/powerpoint-mcp.git
```

2. Add the following entry to your `claude_desktop_config.json` file. Provide the full path to `uv` (run **where uv**) if you face any error. Also, this applied to other paths provided in any config.

```
"mcpServers": {
  "powerpoint": {
    "command": "uv",
    "env": {
      "SD_WEBUI_URL": "http://your-sd-webui-url:7860",
      "SD_AUTH_USER": "your-username", // Optional: if authentication is enabled
      "SD_AUTH_PASS": "your-password", // Optional: if authentication is enabled
    },
    "args": [
      "--directory",
      "/path/to/powerpoint",
      "run",
      "powerpoint",
      "--folder-path",
      "/path/to/decks_folder"
    ]
  }
}
```

claude_desktop_config.json

And Voila!!!



You can now use AI to handle Gmail, Excel, and PowerPoint with ease. I used to hate putting together presentations—now AI does the heavy lifting for me! So I started wondering: Can it also help automate my **documentation writing**? Turns out, the answer is a pretty exciting yes!

Chapter 7: Notion MCP server

Notion x MCP? Yes, it's possible

7.1 Introduction

Notion Model Context Protocol (MCP) is one of my favourite MCP servers out there that connects your AI assistant directly to your Notion workspace. It enables seamless interaction with your notes, databases, and tasks through natural language commands. Whether you're organizing project plans, updating meeting notes, or querying databases, Notion MCP simplifies these tasks, allowing your assistant to handle them effortlessly.

7.1.1 Notion MCP Tools

Here's a concise list of tools available in the Notion MCP server, each designed to help your assistant interact with Notion workspaces:

1. **notion_append_block_children**
Add content blocks under a specific parent block. Useful for organizing and structuring content within a Notion page.
2. **notion_retrieve_block**
Fetch details about a particular block. Ideal for retrieving specific information or metadata about a block.
3. **notion_retrieve_block_children**
Get the child blocks of a specified parent block. Helps in navigating hierarchical structures within Notion pages.
4. **notion_delete_block**
Remove a specific block from a page. Perfect for cleaning up or reorganizing content.
5. **notion_retrieve_page**
Access information about a specific page. Great for retrieving metadata or other details about a Notion page.
6. **notion_update_page_properties**
Modify the properties of a page. Useful for updating titles, tags, or other attributes of a page.

7. **notion_create_database**
Set up a new database within Notion. Essential for creating structured data collections like tables or lists.
8. **notion_query_database**
Search and retrieve entries from a database. Perfect for filtering and finding specific records based on criteria.
9. **notion_retrieve_database**
Obtain details about a specific database. Helps in understanding the structure and properties of a database.
10. **notion_update_database**
Update the schema or properties of a database. Ideal for modifying the structure of an existing database.
11. **notion_create_database_item**
Add a new entry to a database. Great for appending new records or rows to a database.
12. **notion_search**
Look for pages or databases by title. Perfect for quickly locating specific content within your workspace.
13. **notion_list_all_users**
List all users in the Notion workspace. Useful for managing team members or collaborators.
14. **notion_retrieve_user**
Get information about a specific user. Ideal for retrieving details about a collaborator or workspace member.

You might not even need to touch Notion ever in life after this

7.1.2 Real-World Applications

A. Personal Use: Rahul's Streamlined Notion Experience

Rahul, a freelance photographer, often struggled to juggle his projects, client interactions, and personal responsibilities. His Notion workspace had become cluttered, making it hard to stay on top of everything. To simplify his routine, he integrated an AI assistant into his Notion system. Now, he simply describes what he needs, and the assistant handles the rest.

For instance, in the morning, he might say, "Pull up all tasks due this week," and the assistant retrieves the relevant entries from his task database. If a client requests a change, he can say, "Mark the 'Wedding Shoot' project as 'In Progress,'" and the update is made instantly. When a new assignment comes in, he says, "Create a task: 'Edit photos for the travel blog' with a deadline of Friday," and it's automatically added to his list.

This seamless process allows Rahul to dedicate more time to his creative work, with the confidence that his assistant is maintaining an organized and updated Notion workspace.

B. Business Use: Anjali's Efficient Project Management

Anjali, an operations lead at a logistics company, handles a wide range of daily responsibilities, from vendor coordination to internal process improvements. Her Notion workspace, packed with timelines, task lists, and meeting notes, was becoming too complex to manage effectively.

To simplify her workflow, Anjali connected an AI assistant to her Notion setup. Now, she simply instructs the assistant, and it takes care of the details. For example, she might start her day by saying, "Fetch all tasks due this week," and the assistant pulls the relevant data from her task database. If there's an update from a vendor, she can say, "Change the status of the 'Warehouse Expansion' project to 'In Progress,'" and the adjustment is made immediately. When a new task arises, she says, "Add a task: 'Assess new inventory management software' due next Monday," and it's promptly added to her schedule.

This effortless approach enables Anjali to concentrate on high-level planning and team collaboration, knowing her assistant ensures her Notion workspace remains organized and current.

7.2 Step-by-Step Installation

7.2.1 Pre-requisites

Before you dive in, here are a few things you need to set up for Notion MCP:

1. **Notion Account:** Ensure you have an active Notion account.
2. **Internal Integration Token:** This token authenticates your integration with Notion. To get this token, you can create a integration by visiting <https://www.notion.so/profile/integrations>

Integrations > New Integration

Integration Name

Add integration name

Associated workspace *

Select a workspace to install the integration to. Workspace owners will be able to manage the integration as well. You can upgrade the integration to use OAuth later

Type *

Internal

You can change the type of your integration at any time. Please note that switching to a public integration may require additional fields.

Logo

Upload
512 x 512

By submitting, you agree to Notion's [Developer Terms](#).

Cancel Save

Integration page for Notion

3. Copy the token once the integration is created

Profile
Templates
Integrations
Analytics
Coupons
Learn

Upload
512 x 512

Integration Name
i1

Configuration Basic information Webhooks

Internal Integration Secret
..... Refresh Show

Only works with the Nilladri Sen's Notion workspace. To change workspace, [create another integration](#).

Capabilities

These requested capabilities will be displayed to users when they authorize your integration. See [developer docs](#) for more help.

Content Capabilities

- ☒ Read content
Request to read content.
- ☒ Update content
Request to update existing content.
- ☒ Insert content

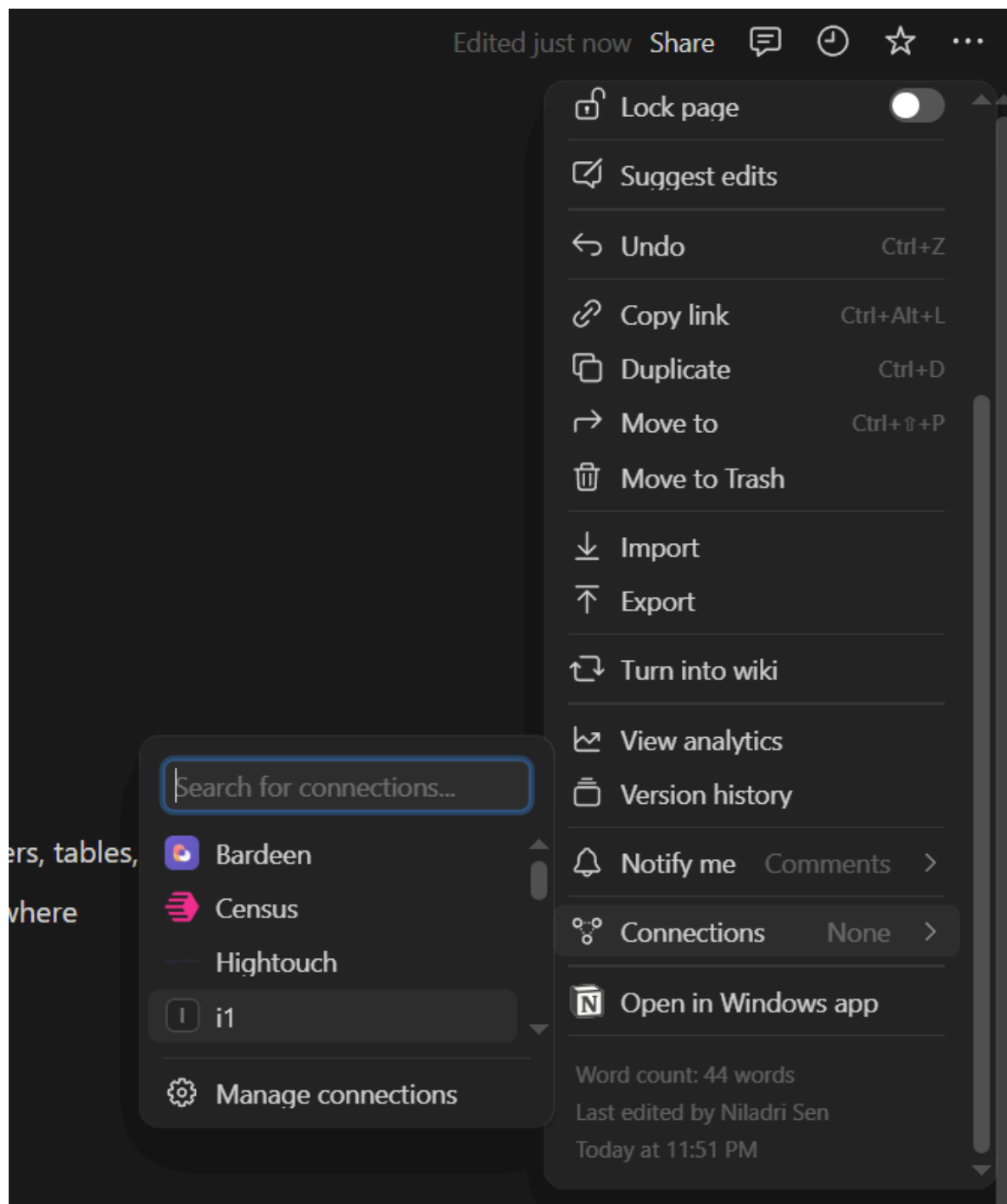
Integration token for Notion integration

4. **Node.js and npm Installed:** Required to run the MCP server.

7.2.2 Steps

1. **Add the Integration to Your Workspace:**

- Open the Notion page or database you want the integration to access.
- Click the "... " button in the top right corner.
- Select the integration from the connections options and proceed to integrate it to the workspace.

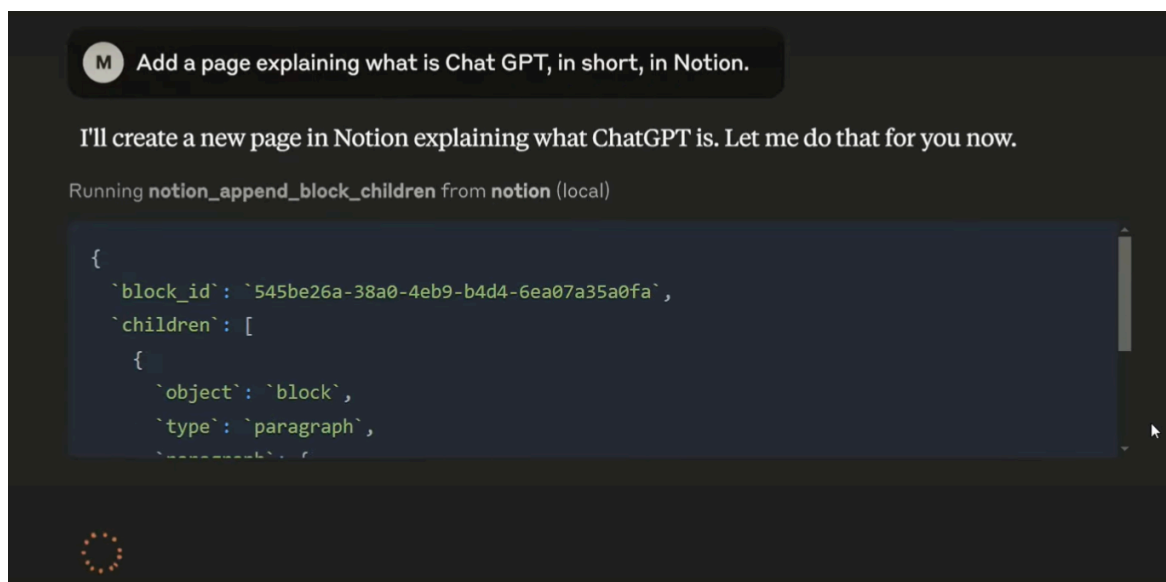


Add integration to the workspace connections

2. Add the following to the claude config json.

```
{
  "mcpServers": {
    "notion": {
      "command": "npx",
      "args": ["-y", "@suekou/mcp-notion-server"],
      "env": {
        "NOTION_API_TOKEN": "your-integration-token"
      }
    }
  }
}
```

Claude_desktop_config.json



When everything is set up, you can interact with your assistant using everyday language. Just type out what you need done—no need for complicated commands. Here are some examples of what you can ask:

- *"Create a new page titled 'Weekly Goals' and list my top 5 priorities."*
- *"Set a reminder to look over the 'Marketing Strategy' document tomorrow at 10 AM."*
- *"Give me a summary of the key points from the 'Client Feedback' database."*

Your assistant will handle the rest—creating pages, setting reminders, summarizing data, updating statuses, fetching information, deleting pages, searching for content, and listing team members. It's really that straightforward

Enough of Productivity gains, we will now move onto automating some Social Media posting !

Part 3: Social Media MCPs

Chapter 8: WhatsApp MCP server

From Messages to Management—AI Supercharges Your WhatsApp Chats!

8.1 Introduction

WhatsApp Model Context Protocol (MCP) is a simple tool that helps you connect WhatsApp with an AI assistant. It makes it easier for the assistant to understand your messages, remember past chats, and reply in a smart, helpful way. Whether you're building a personal helper or a business chatbot, WhatsApp MCP keeps things organized and easy to manage.

When it comes to social messaging, WhatsApp is often the first app that comes to mind. Known for its strong end-to-end encryption, it ensures that your conversations stay private and secure. So why not bring AI into the mix? With the WhatsApp MCP server, you can now send messages, read texts, and do much more—all with the power of AI behind you.

8.1.1 WhatsApp MCP Tools

Not just sending or receiving messages, WhatsApp MCP has some very interesting tools that you can enable to automate your WhatsApp.

1. **search_contacts**
Search for WhatsApp contacts using their name or phone number. Perfect for quickly finding people without scrolling through your contact list.
2. **list_messages**
Retrieve past messages with optional filters like date, sender, or keyword—great for reviewing conversations or getting context for replies.
3. **list_chats**
View all ongoing chats, along with helpful metadata like contact names and last message time. Useful for managing multiple conversations.
4. **get_chat**
Access detailed information about a specific chat, including participants and timestamps. Ideal for understanding who's involved in a conversation.
5. **get_direct_chat_by_contact**
Find a one-on-one chat with a specific contact. Helps the assistant focus only on personal messages, not group threads.

6. **get_contact_chats**
List all chats that include a particular contact, whether individual or group. Helpful for tracking interactions across different contexts.
7. **get_last_interaction**
Quickly grab the most recent message from a specific contact. Perfect for picking up right where the conversation left off.
8. **get_message_context**
Fetch the surrounding messages of a specific message to understand its context—crucial for generating accurate replies.
9. **send_message**
Send a text message to any contact or group. This is the core tool for keeping communication flowing.
10. **send_file**
Share images, videos, documents, or raw audio with WhatsApp contacts. Useful for sending attachments like receipts, notes, or media files.
11. **send_audio_message**
Send an audio file as a playable WhatsApp voice message. Automatically converts formats if FFmpeg is installed for better compatibility.
12. **download_media**
Download media from a WhatsApp message using its ID and chat info. Lets you store or reuse media received in chats.

8.1.2 Real-World Applications

A. Personal Use: Maya's AI Assistant for Everyday Life

Maya is a university student who relies on WhatsApp for everything—class discussions, group project coordination, and sharing study materials. But Maya often loses track of which group shared which file, and she finds it hard to scroll back through old messages to find the last important update from her class rep.

Solution?

Use WhatsApp MCP and get the job done. With tools like **get_last_interaction** and **list_chats**, her assistant can quickly find the most recent messages across all her chats. Need to send her handwritten notes or a voice memo? The assistant uses **send_file** or **send_audio_message**, converting audio to the right format if needed. Now, Maya stays organized and focused—without getting buried in chat clutter.

B. Business Use: Raj's Smart Storefront on WhatsApp

Raj owns a growing electronics shop where most of his customers prefer to place orders and ask questions via WhatsApp. He wants to provide quick support—like sending manuals, tracking orders, and handling returns—but manually replying to every message is slowing him down.

Solution?

WhatsApp MCP turns Raj's WhatsApp into a smart storefront. His assistant can automatically retrieve past messages using **list_messages** and **get_message_context**, find customer chats with **get_contact_chats**, and respond instantly through **send_message**. Need to share a product manual or invoice? It's done in seconds with **send_file**. With WhatsApp MCP handling the routine work, Raj's business runs smoother—and his customers stay happy.

8.2 Step-by-Step Installation

8.2.1 Pre-requisites

Before you dive in, here are a few things you need to set up WhatsApp MCP:

- **Go** – This is a programming language the bridge runs on. (Download it from golang.org.)

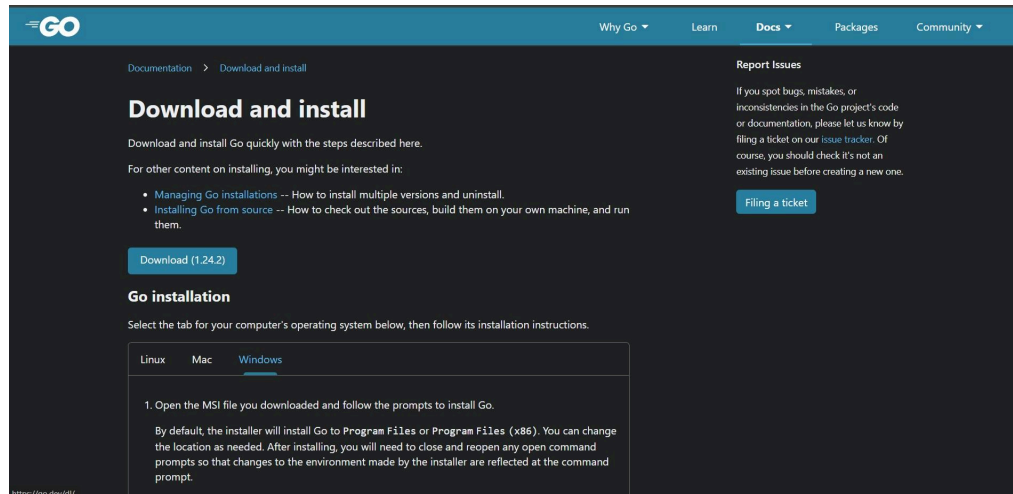


Fig 3.1 - Go installation page

- **FFmpeg (optional)** – Only needed if you want to send audio messages that play like real voice notes in WhatsApp. If you skip this, you can still send audio files—they just won't look like voice messages.

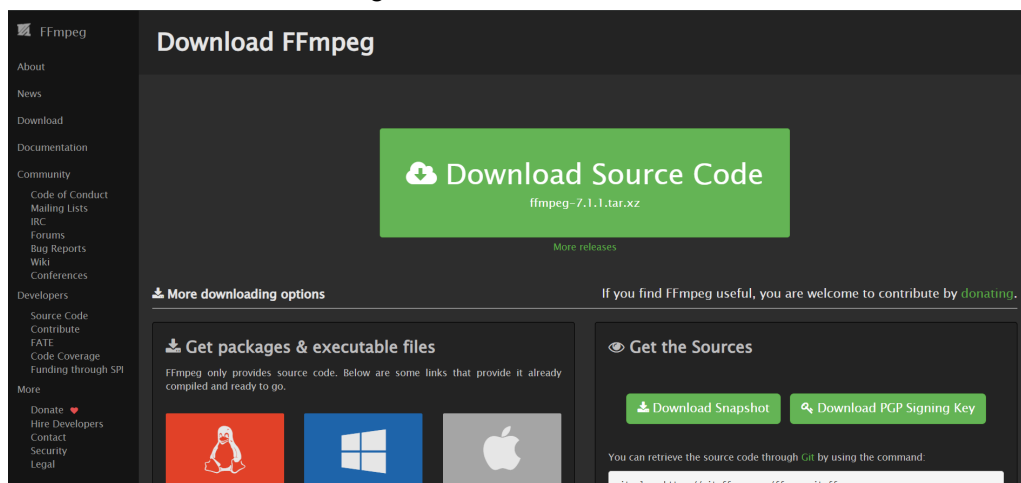


Fig 3.2 - ffmpeg installation page

8.2.2 Steps

1. Download the Server Project. Open your terminal and run:

```
git clone https://github.com/lharries/whatsapp-mcp.git
cd whatsapp-mcp
```

```
User@POINCARE MINGW64 ~/notebooks
$ git clone https://github.com/lharries/whatsapp-mcp.git
Cloning into 'whatsapp-mcp'...
remote: Enumerating objects: 108, done.
remote: Counting objects: 100% (55/55), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 108 (delta 36), reused 30 (delta 30), pack-reused 53 (from 1)
Receiving objects: 100% (108/108), 2.96 MiB | 3.71 MiB/s, done.
Resolving deltas: 100% (62/62), done.

User@POINCARE MINGW64 ~/notebooks
$ cd whatsapp-mcp/

User@POINCARE MINGW64 ~/notebooks/whatsapp-mcp (main)
$
```

Fig 3.3 - Downloading the MCP server

2. **Start the WhatsApp Bridge:** This part connects your WhatsApp to your assistant. In your terminal:

For linux:

```
cd whatsapp-bridge
go run main.go
```

For Windows:

```
cd whatsapp-bridge
go env -w CGO_ENABLED=1
go run main.go
```

3. The first time you run this, it'll show a QR code. Just scan that using the WhatsApp app on your phone to link it up.

Note : You may need to re-scan the QR code every ~20 days to stay connected.

- At this step it is important for us to identify two paths before we proceed - the UV path and the source folder path
- You can get the UV path by running **which uv** in the terminal. We will call this PATH_TO_UV

```
User@POINCARE MINGW64 ~
$ which uv
/c/Users/User/.local/bin/uv
```

uv path

The source folder path is the path of the parent folder containing the whatsapp-mcp project that we just git cloned. We will call this `PATH_TO_SRC`.

4. Now for our final step, we create our `config.json` using these paths.

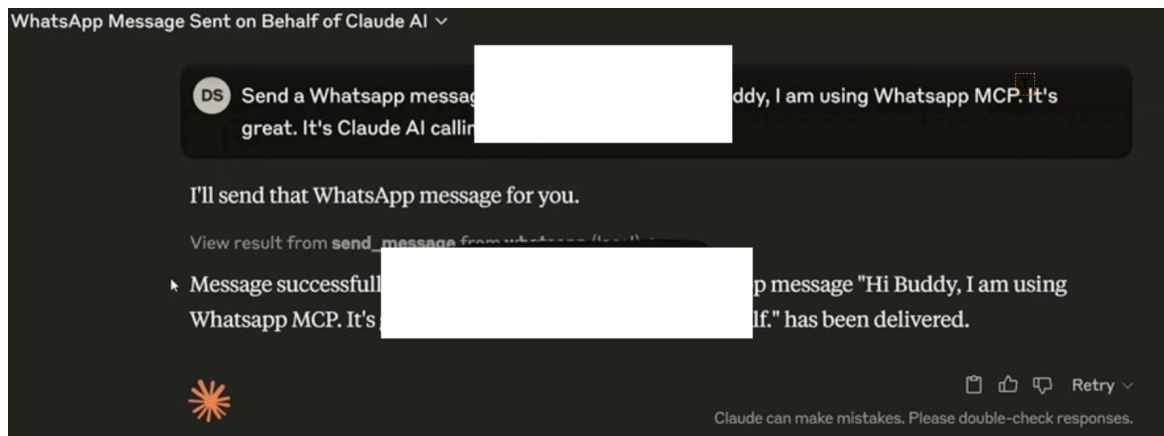
```
{
  "mcpServers": {
    "whatsapp": {
      "command": "{{PATH_TO_UV}}",
      "args": [
        "--directory",
        "{{PATH_TO_SRC}}/whatsapp-mcp/whatsapp-mcp-server",
        "run",
        "main.py"
      ]
    }
  }
}
```

`claude_desktop_config.json`

Finally, save this file:

- **For Claude Desktop:** Save it in your `claude_desktop_config.json`.

And we are done.



If you are a small business manager, a content creator, or maintaining some community, I hope you are now able to manage it using AI on WhatsApp. This looks to be the coolest out of all the MCPs that we have tried. And not just WhatsApp. In this chapter, we are covering some more social media MCPs, so you can have an entire network of your communities with just one prompt to AI.

Chapter 9: YouTube MCP Server

Skip the Scroll — Let AI Crawl YouTube for You!

9.1 Introduction

The **YouTube MCP Server** enables AI to interact with YouTube content using natural language through a structured set of tools powered by the YouTube Data API. With this server, Claude can search for videos, retrieve detailed metadata, and analyze engagement metrics such as views, likes, and comments.

It also supports transcript extraction with language preferences, provides related video suggestions, and offers comprehensive channel analytics. Whether you're researching content trends or building media dashboards, the YouTube MCP Server gives your assistant the power to explore and analyze YouTube at scale.

9.1.1 YouTube MCP Tools

Here's a concise list of tools available in the YouTube MCP server, each designed to help your assistant interact with YouTube content and analytics:

1. **getVideoDetails:** Retrieve detailed metadata, statistics, and content information for one or more YouTube videos.
2. **searchVideos:** Search for YouTube videos based on a keyword or phrase.
3. **getTranscripts:** Retrieve video captions or transcripts, with optional language specification and time-stamped support.
4. **getRelatedVideos:** Fetch videos that are related to a given video ID, based on YouTube's recommendation engine.
5. **getChannelStatistics;** Access metrics for one or more YouTube channels, including subscriber count, total views, and video count.
6. **getChannelTopVideos:** Get a list of top-performing (most viewed) videos from a specific channel.
7. **getVideoEngagementRatio;** Calculate engagement metrics like likes, comments, views, and overall engagement ratio for videos.
8. **getTrendingVideos:** View currently trending videos by region and category.

9. **compareVideos:** Compare performance metrics (views, likes, comments, etc.) across multiple videos.

9.1.2 Real-World Applications

How a Content Strategist Uses AI to Decode YouTube Trends

Meet Ayesha, a content strategist planning a new tech campaign. She needs a snapshot of what's trending and what performs well in her niche on YouTube. Instead of diving into spreadsheets or manually watching dozens of videos, she uses her AI assistant to do the heavy lifting.

For example, she can ask the assistant to:

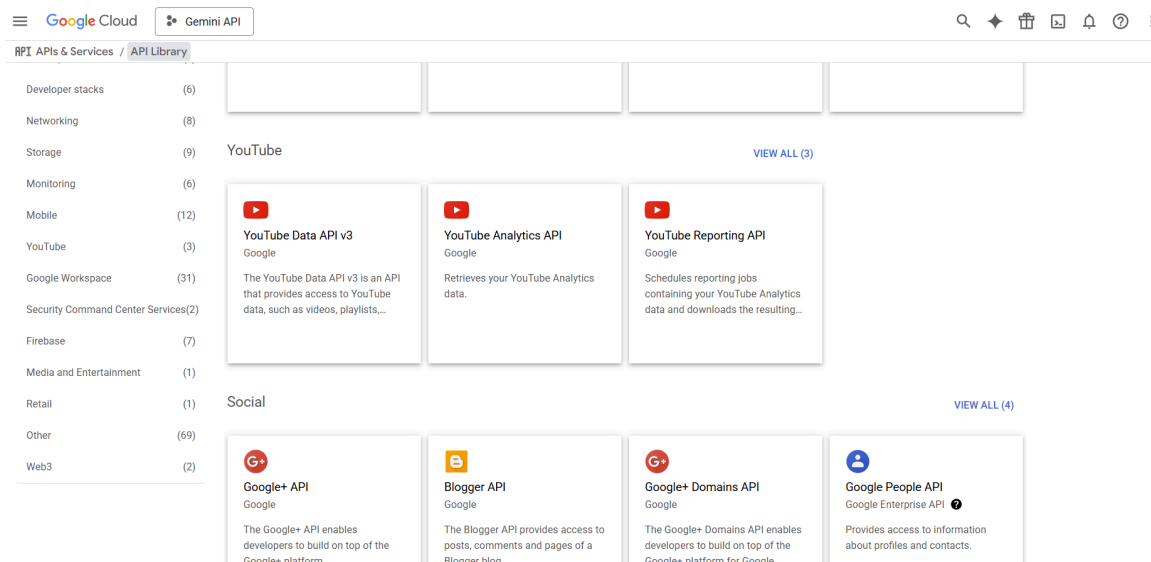
- **Fetch trending videos in India:** "Show me the top 5 trending videos in India."
- **Get channel statistics:** "Fetch the subscriber count and total views for the 'Data Science in Your Pocket' channel."
- **Retrieve video transcripts:** "Give me the transcript for the latest video on machine learning."
- **Compare videos:** "Compare the engagement stats for videos X, Y, and Z."
- **Discover related content:** "Find videos related to the 'Deep Learning Tutorial' video."

9.2 Step-by-Step Installation

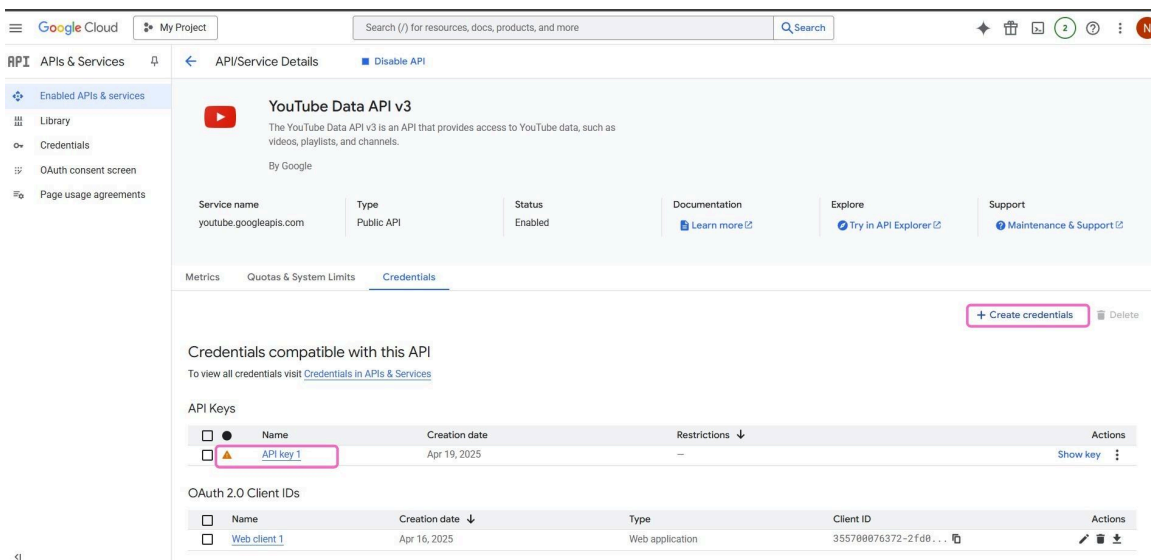
Like the Gmail MCP (GSuite MCP), this integration also requires enabling the YouTube API in Google Cloud. Kindly follow the same steps as we did for Gmail MCP Server.

9.2.1 Pre-requisites

1. Before using the YouTube MCP Server, you'll need a **YouTube Data API** key, available from the Google Cloud Console
2. **Create a project** → Enable *YouTube Data API v3* → Create API credentials (API key)



Youtube API on Google Cloud Platform



Youtube API key

3. You can click on the **Create Credentials** button to create a new api key. This API key will be needed for connecting the MCP to Youtube service.

9.2.2 Steps

1. You have two options to install the YouTube MCP Server::

Option 1: Install via Smithery (Recommended). Smithery a registry and management platform for Model Context Protocol (MCP) servers

```
npx -y @smithery/cli install @icraft2170/youtube-data-mcp-server --client claude
```

Option 2: Clone and install manually

```
git clone https://github.com/icraft2170/youtube-data-mcp-server.git
cd youtube-data-mcp-server
npm install
```

2. Next, add the following block to your Claude configuration JSON file:

For windows:

```
{
  "mcpServers": {
    "youtube": {
      "command": "npx",
      "args": ["-y", "youtube-data-mcp-server"],
      "env": {
        "YOUTUBE_API_KEY": "YOUR_API_KEY_HERE",
        "YOUTUBE_TRANSCRIPT_LANG": "en"
      }
    }
  }
}
```

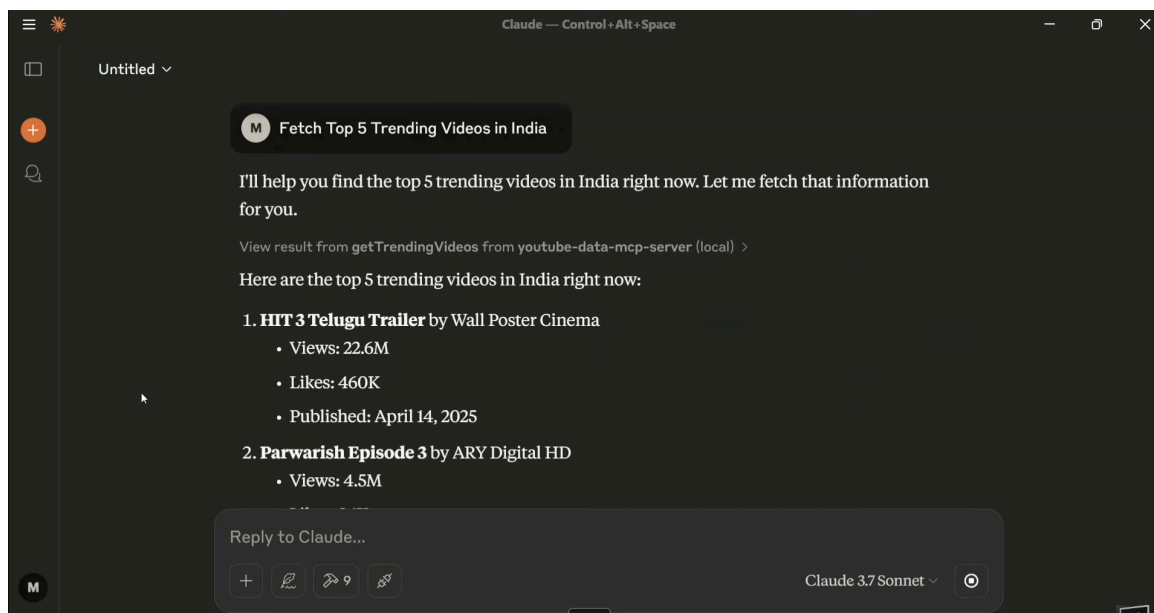
claude_desktop_config.json

For Linux systems:

```
CLAUDE_CONFIG=${HOME}/.config/Claude/claude_desktop_config.json
cat <<EOF > $CLAUDE_CONFIG
{
  "mcpServers": {
    "youtube": {
      "command": "npx",
      "args": ["-y", "youtube-data-mcp-server"],
      "env": {
        "YOUTUBE_API_KEY": "YOUR_API_KEY_HERE",
        "YOUTUBE_TRANSCRIPT_LANG": "en"
      }
    }
  }
}
EOF
cat $CLAUDE_CONFIG
```

claude_desktop_config.json

The tools provided by the YouTube MCP Server make it easy to retrieve and analyze video content, track engagement metrics, and explore trending topics across various categories. Simply ask your assistant to perform tasks like searching for videos, analyzing channels, or comparing video performance to gain valuable insights quickly.



This is so cool. AI handling any software end-to-end without a glitch. Shall we take it to the next level? Let's try **Slack** next.

Chapter 10: Slack MCP Server

Let AI Keep Up with Slack So You Don't Have To

10.1 Introduction

The Slack MCP Server enables AI to interact with Slack in real time using the Model Context Protocol (MCP). Through this integration, the Slack MCP Server can read from, write to, and organize conversations across channels, threads, and users—all using natural language.

It supports posting messages, replying to threads, fetching conversation histories, managing user interactions, and even reacting to specific messages. With this setup, your assistant can monitor key channels, summarize discussions, notify the right people, and streamline collaboration inside your Slack workspace.

10.1.1 Slack MCP Tools

Here's a list of the core capabilities provided by the Slack MCP Server:

1. **slack_list_channels**
List public or predefined channels in the workspace.
2. **slack_post_message**
Post a message to a specific Slack channel.
3. **slack_reply_to_thread**
Reply to a specific thread in a Slack channel.
4. **slack_add_reaction**
React to a message in a channel using an emoji.
5. **slack_get_channel_history**
Retrieve recent messages from a Slack channel.
6. **slack_get_thread_replies**
Get all replies to a specific message thread.
7. **slack_get_users**
List users in the workspace with basic profile information.
8. **slack_get_user_profile**
Get detailed profile information for a specific user.

10.1.2 Real-World Applications

How a Project Manager Tracks Team Discussions with AI in Slack?

Meet Asha, a project manager overseeing several product teams. Each morning, she asks Claude to “Summarize yesterday’s messages in #product-updates.” The assistant scans recent conversations, highlights key decisions and blockers, and shares a concise summary directly in the channel.

Not just that, it can be very handy managing big teams :

- Post updates or announcements: “Send a message to #general: The deployment is complete!”
- Respond to ongoing conversations: “Reply to this thread: We’ll review the PR by tomorrow.”
- Summarize discussions: “Summarize the last 10 messages in #design-feedback.”
- List workspace channels or users: “Show me all the public channels.” or “Who are the members of the #engineering channel?”
- Inspect a user’s profile: “What’s the profile info for @alex?”
- Review channel activity: “Fetch the last 5 messages from #support.” or “Show replies to the thread discussing API rate limits.”

10.2 Step-by-Step Installation

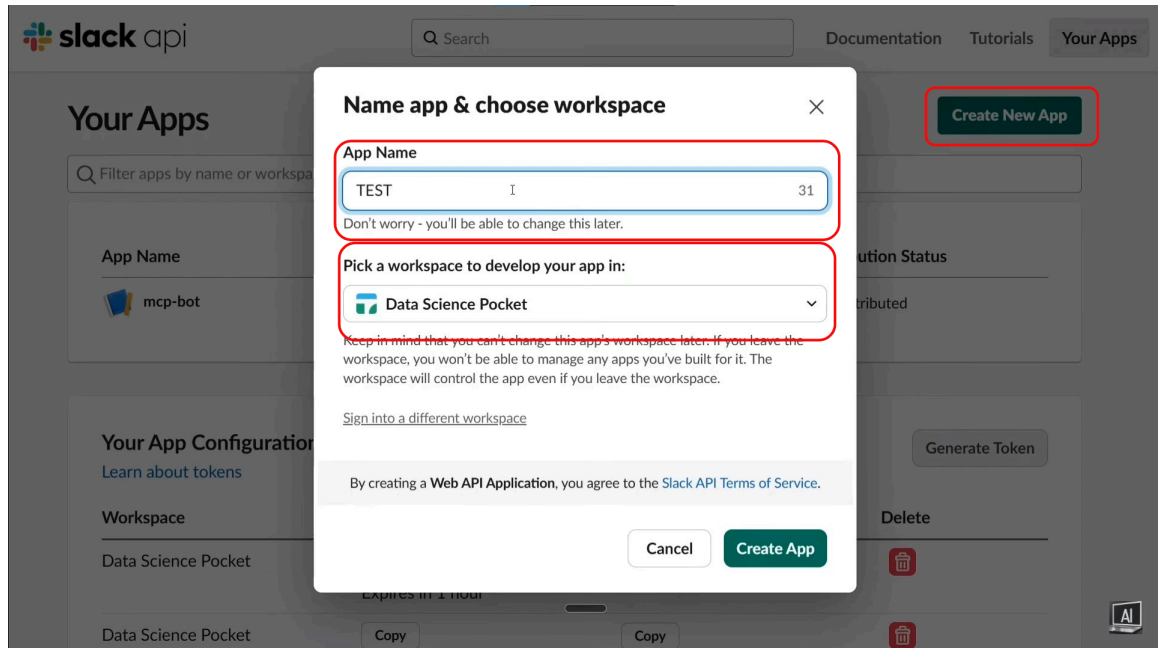
10.2.1 Pre-requisites

1. Ensure you have Slack workspace admin access or appropriate permissions to install apps.
2. Optionally have docker desktop running.

10.2.2 Steps

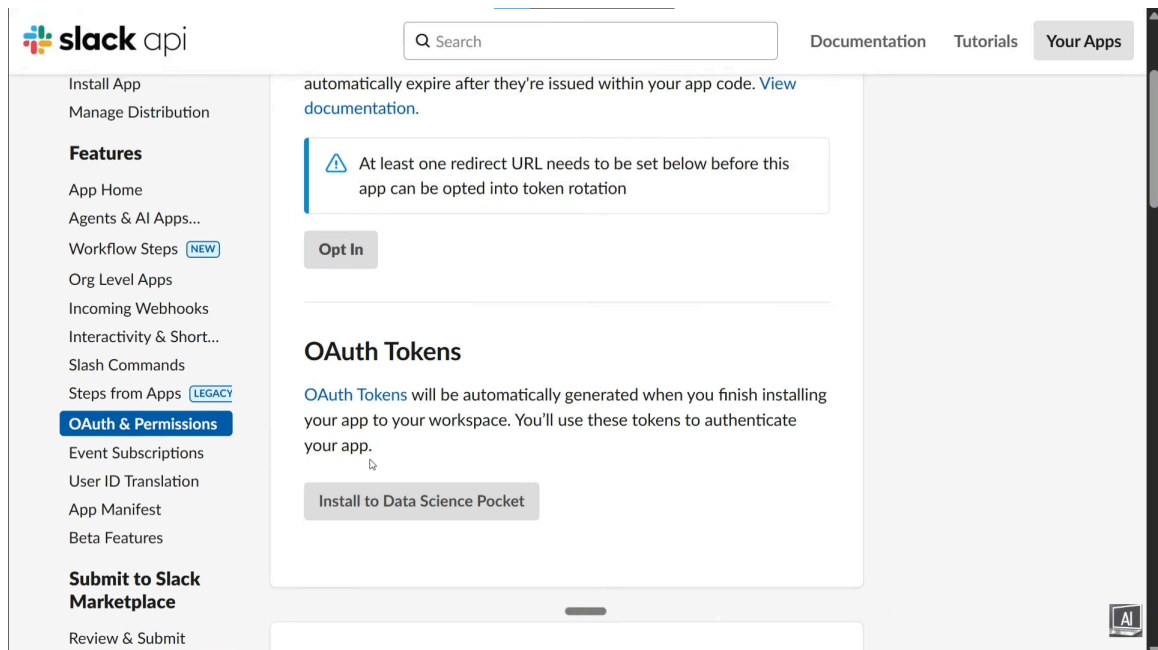
1. Create a Slack app at <https://api.slack.com/apps>.
2. You can create a new test app by clicking on **"Create New App"** - **"From scratch"**

3. Finally providing a name for your app and selecting the workspace where you want to install it.



Set App and select workspace

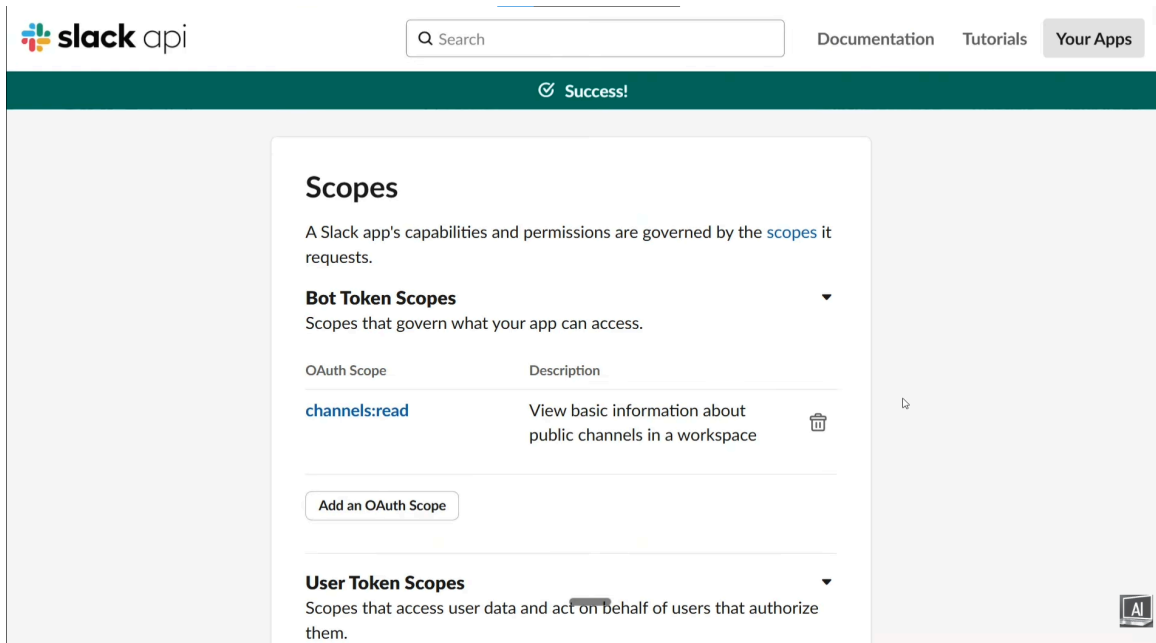
4. **Configure Bot Token Scopes:** Navigate to "OAuth & Permissions" in the left-hand menu.



OAuth & Permissions on left pane

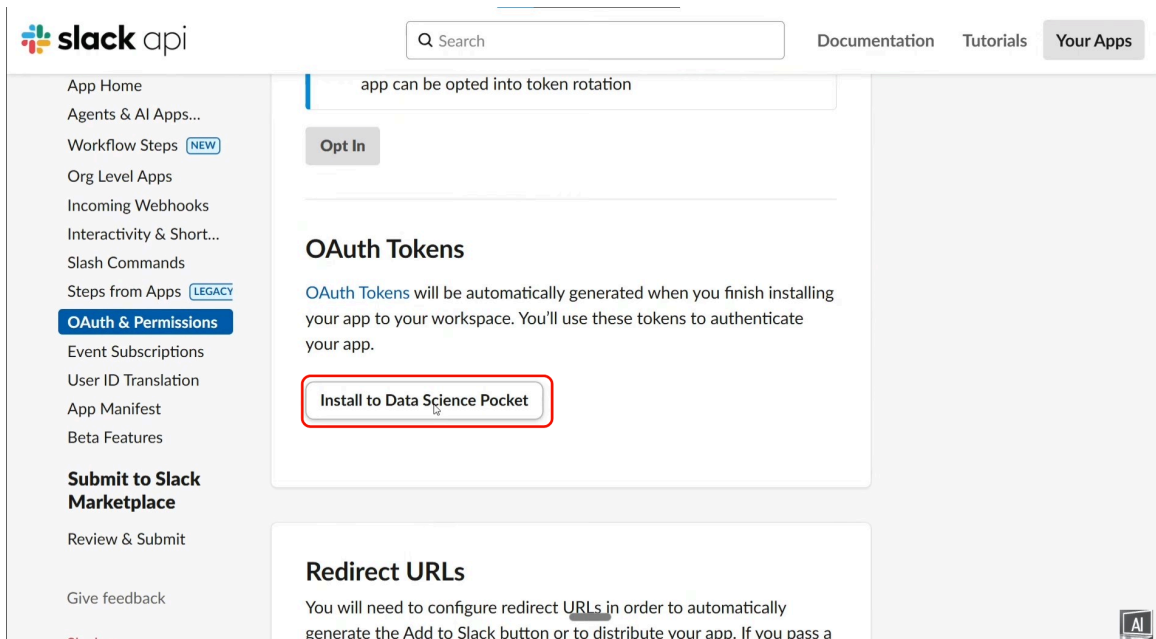
5. Under the "Scopes" section, add the following bot token scopes:
 - channels:history – View messages and other content in public channels.
 - channels:read – View basic channel information.

- chat:write – Send messages as the app.
- reactions:write – Add emoji reactions to messages.
- users:read – View users and their basic information.
- users.profile:read – View detailed profiles about users.



Add scope to Slack app

6. Add the test app to your workspace and save the **"Bot User OAuth Token"** that starts with "xoxb-"



Install the app to your workspace

7. We would also need the **teams id** for the workspace. Follow this guide for the same: <https://slack.com/help/articles/221769328-Locate-your-Slack-URL-or-ID#find-your-work-space-or-org-id>

Your teams id will usually start with a 'T'.



Teams ID

8. For the final step to enable the Slack MCP Server in Claude Desktop, you'll need to update your `claude_desktop_config.json` with one of the following options:

Option 1: Using NPX (Recommended for Quick Start)

```
{
  "mcpServers": {
    "slack": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-slack"
      ],
      "env": {
        "SLACK_BOT_TOKEN": "xoxb-your-bot-token",
        "SLACK_TEAM_ID": "T07TGTSTDFST6RB5",
        "SLACK_CHANNEL_IDS": "C07U1HEHNG1"
      }
    }
  }
}
```

`claude_desktop_config.json`

Option 2: Using Docker (For Isolated Environments)

```
{
  "mcpServers": {
    "slack": {
      "command": "docker",
      "args": [
        "run",
        "-i",
        "--rm",
        "-e",
        "SLACK_BOT_TOKEN",
        "-e",
        "SLACK_TEAM_ID",
        "-e",
        "SLACK_CHANNEL_IDS",
        "mcp/slack"
      ],
      "env": {
        "SLACK_BOT_TOKEN": "xoxb-your-bot-token",
        "SLACK_TEAM_ID": "T07TGTSTDFST6RB5",
        "SLACK_CHANNEL_IDS": "C07U1EHNG1"
      }
    }
  }
}
```

claude_desktop_config.json

Note: You are required to pass on some required environment variables

1. **SLACK_BOT_TOKEN** (Required): Your Slack Bot User OAuth token, which should start with `xoxb-`.
2. **SLACK_TEAM_ID** (Required): The unique ID of your Slack workspace, typically starts with `T`.
3. **SLACK_CHANNEL_IDS** (Optional): A comma-separated list of specific channel IDs you want to allow Claude to access (e.g., `"C01234567, C76543210"`). If omitted, all public channels will be available.

The interactions happen using the channel and space ids. So do copy that for sure. Quite some effort to set up slack compared to others but trust me, this is completely worth it !

M Post a message in the group MCP testing telling that you are having a good time today

I'll post a message to the MCP testing channel for you.

View result from `slack_list_channels` from `slack` (local) ▾

```
{  
  
  [{"type": "text", "text": "{\n\"ok\":true,\n\"channels\":  
[\n  {\n    \"id\": \"C08LDMUA1M\",\n    \"name\": \"mcp-  
test\",\n    \"is_channel\": true,\n    \"is_group\": false,\n    \"is_im\": false,\n    \"is_mpim\": false,\n    \"is_private\": false,\n    \"is_shared\": false,\n    \"is_org_shared\": false,\n    \"is_pending_ext_shared\": false,\n    \"pending_ext_shared\": false,\n    \"context_team_id\": \"T08LWH3E2RY\",\n    \"updated\": 1743778642316,\n    \"parent_conversation\": null  
  ]  
}"}]
```

I found a channel called "mcp-test" that appears to be the MCP testing channel you mentio

Next up we've, similar to Slack, Discord which also enables a bot for posting in Discord communities

Chapter 11: Discord MCP server

Finding it hard to manage your Discord community? AI can help big time

11.1 Introduction

The **Discord MCP server** is a powerful integration that bridges the capabilities of Discord and connects it with AI. This server allows users to seamlessly interact with Discord servers, manage channels, messages, roles, and webhooks—all from within their AI-powered development environment. By leveraging the Discord API, this integration simplifies complex workflows, enabling developers to automate tasks, moderate content, and streamline collaboration without leaving their workspace.

11.1.1 Discord MCP Tools

Discord MCP comes with a lot of useful tools that can help you orchestrate your Discord journeys:

A. Server Information

1. **get_server_info**
Retrieve detailed information about the Discord server, including its name, ID, member count, and more.
2. **list_members**
Fetch a list of all members in the server along with their respective roles.

B. Message Management

1. **send_message**
Send a message to a specific channel within the server.
2. **read_messages**
Access recent message history from a channel.
3. **add_reaction**
Add a reaction (e.g., an emoji) to a specific message.
4. **add_multiple_reactions**
Add multiple reactions to a single message.
5. **remove_reaction**
Remove a reaction from a message.

6. **moderate_message**

Delete inappropriate messages or timeout offending users.

C. Channel Management

1. **create_text_channel**

Create a new text channel within the server.

2. **delete_channel**

Remove an existing channel.

D. Role Management

1. **add_role**

Assign a role to a user.

2. **remove_role**

Remove a role from a user.

E. Webhook Management

1. **create_webhook**

Set up a new webhook for automated messaging.

2. **list_webhooks**

Retrieve a list of all webhooks configured in a channel.

3. **send_webhook_message**

Post messages through a webhook.

4. **modify_webhook**

Update the settings of an existing webhook.

5. **delete_webhook**

Remove a webhook.

11.1.2 Real-World Applications

A. Automating Moderation: Simplifying Community Management

Meet Alex, a community manager overseeing a large Discord server. With hundreds of active members, manually moderating chats becomes overwhelming. Using the Discord MCP server, Alex automates key moderation tasks such as deleting inappropriate messages and timing out rule-breaking users. By integrating these tools into his workflow, Alex ensures the server remains a safe and welcoming space while reducing manual effort.

B. Streamlining Content Creation: Sam's Webhook-Powered Announcements

Sam manages a gaming community on Discord and regularly shares updates with the server. To streamline this process, Sam has set up an automated system that generates a messaging endpoint and uses it to deliver scheduled announcements. This approach eliminates the need for manual posting, saving time while ensuring reliable and consistent communication with the community.

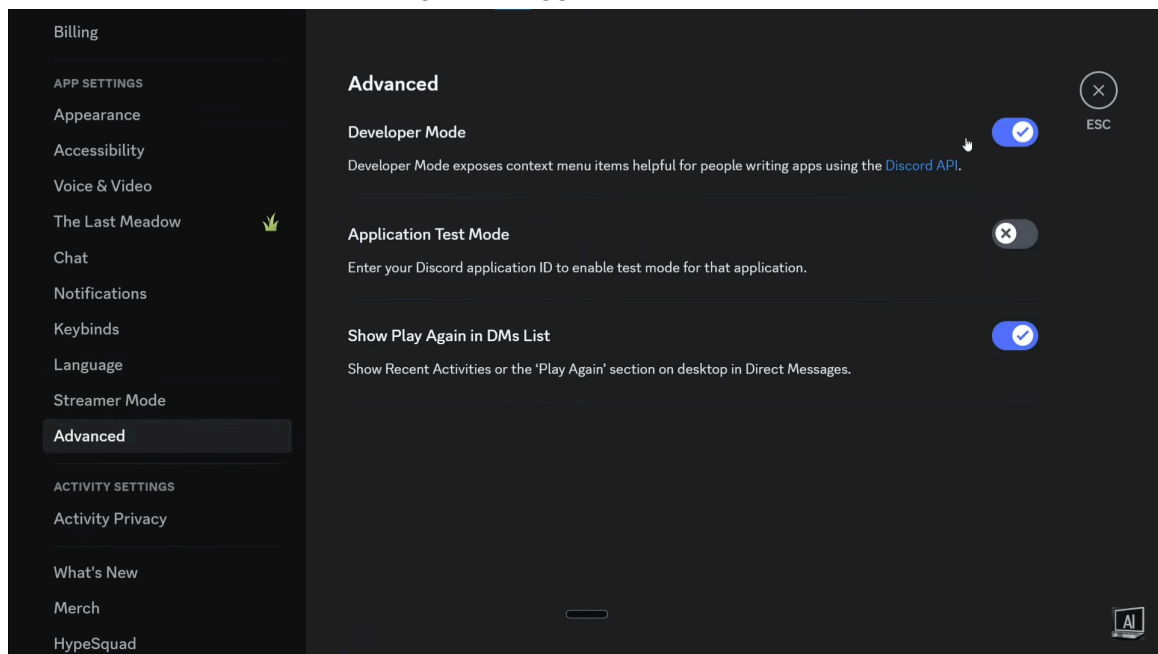
Here are some cool things you can try:

- Post updates or announcements: “Hey Claude, send a message to #general saying ‘Server maintenance is done!’”
- Check out conversations: “Read the last 20 messages in #support.”
- Keep things tidy (moderation): * “Delete the last 5 messages in #off-topic.”
- Organize your server: “Make a new text channel called #event-planning.”
- Manage roles like a boss: “Give @jane_doe the @VIP role.”

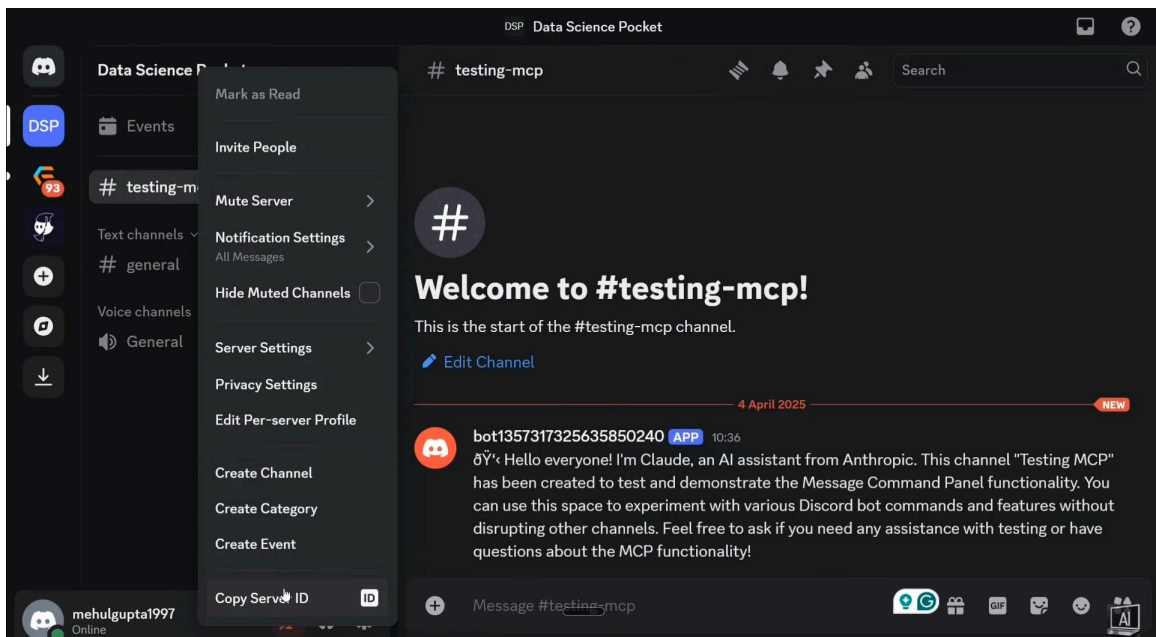
11.2 Step-by-Step Installation

11.2.1 Prerequisites

1. **A Discord channel** where you have administrative privileges and Developer settings enabled. Go to Advanced Settings and toggle **Developer Mode** on.

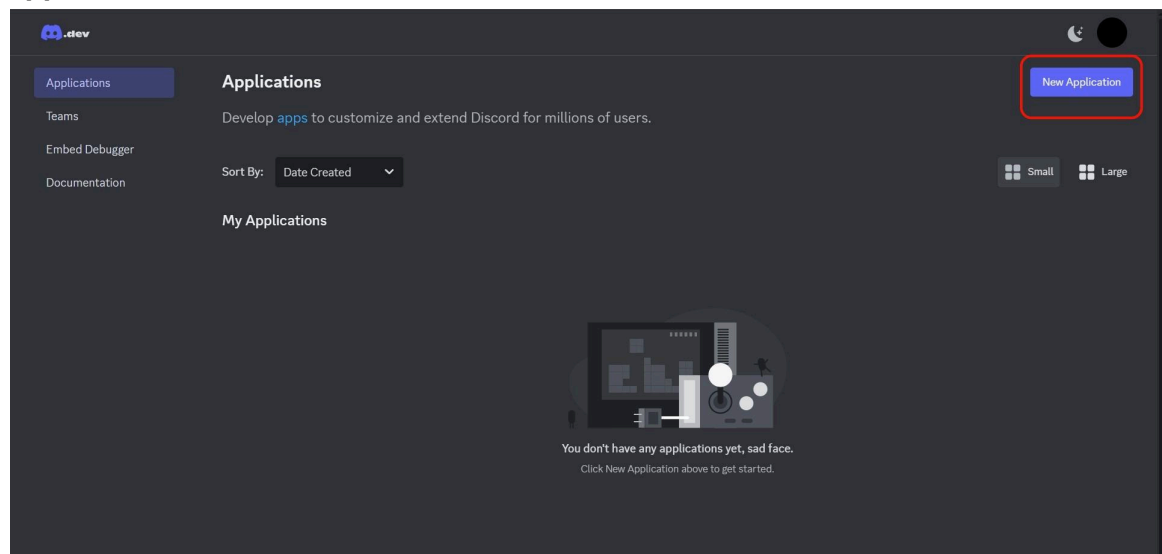


2. You will now be able to see **server ID** from the right click drop down from your Server name as shown in the figure.

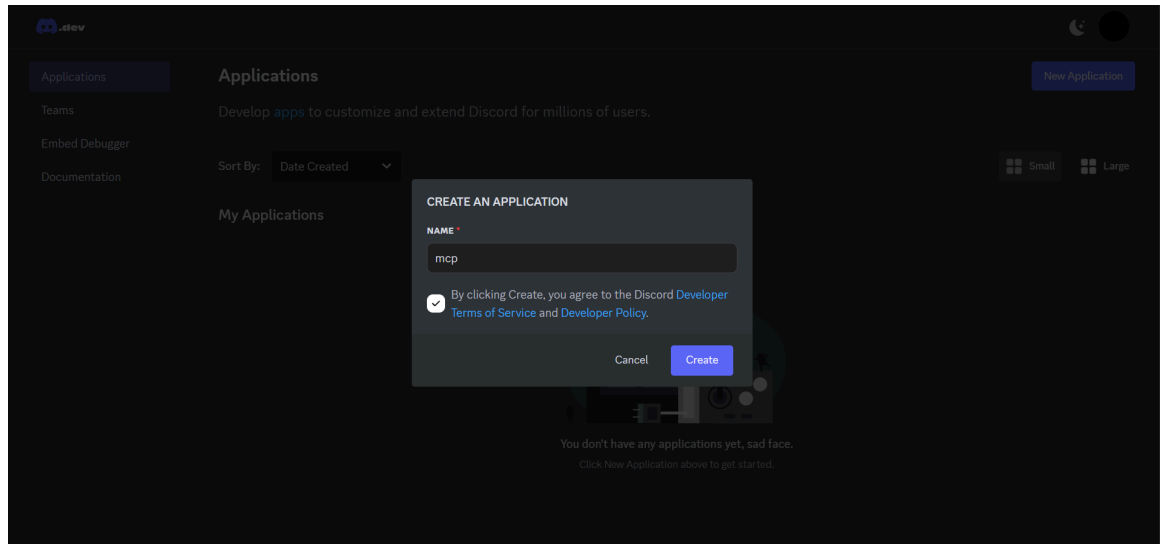


3. A **Discord bot** token generated from the Discord Developer Portal.

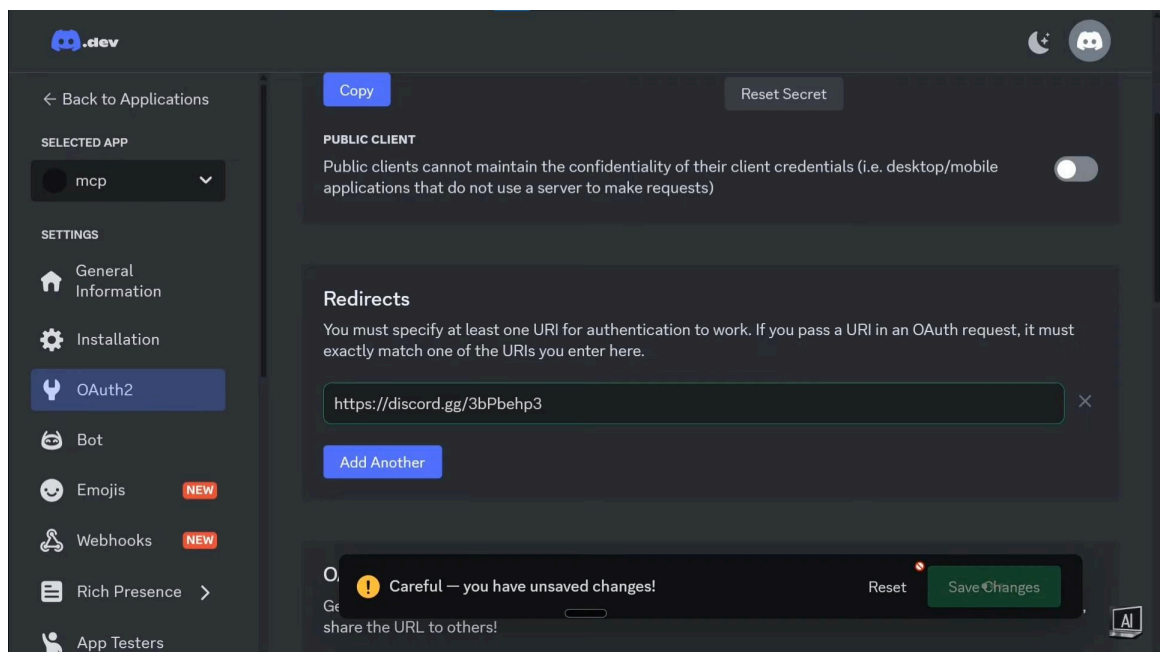
A. To do this first visit <https://discord.com/developers/applications> and click on **New Application**



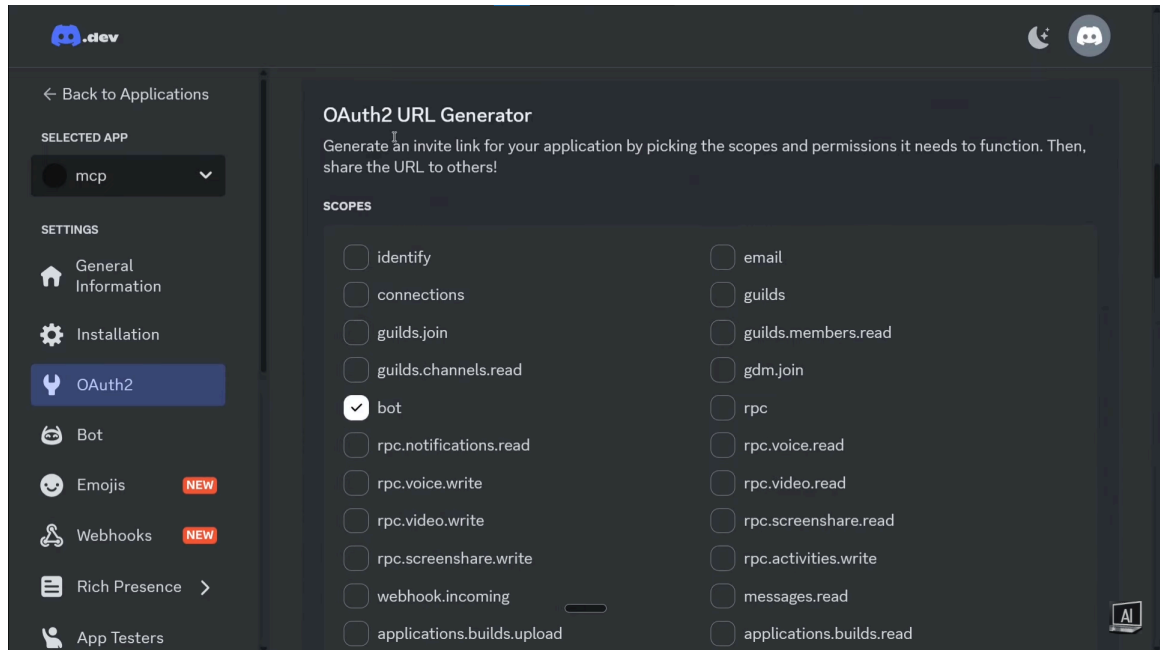
B. Now give a name to MCP bot and create the application.



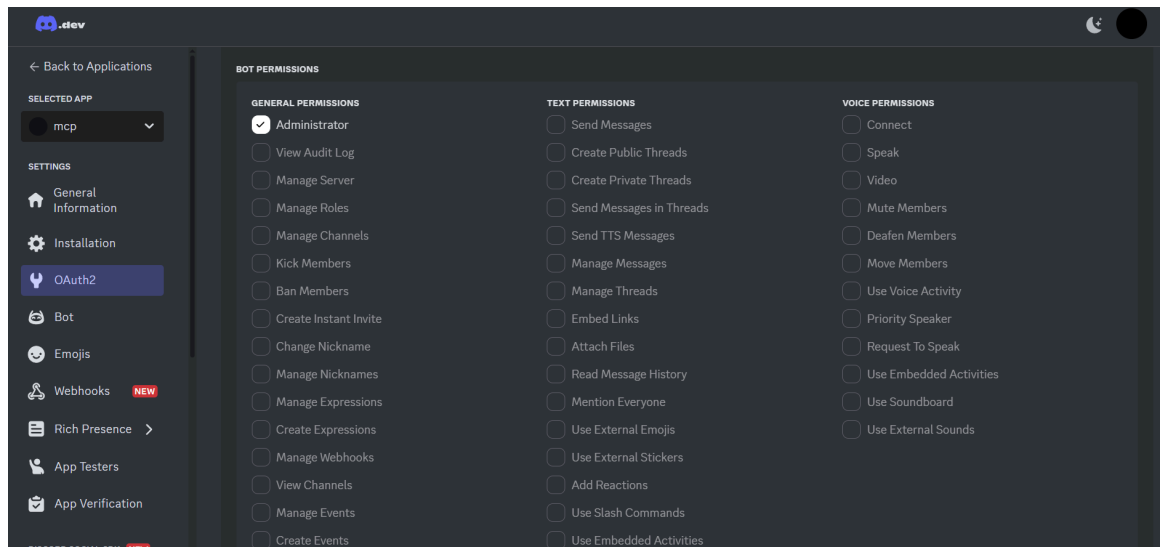
C. Once the bot is created, you can connect it to your server by pasting the server URI at ***Oauth2 -> Redirects*** tab.



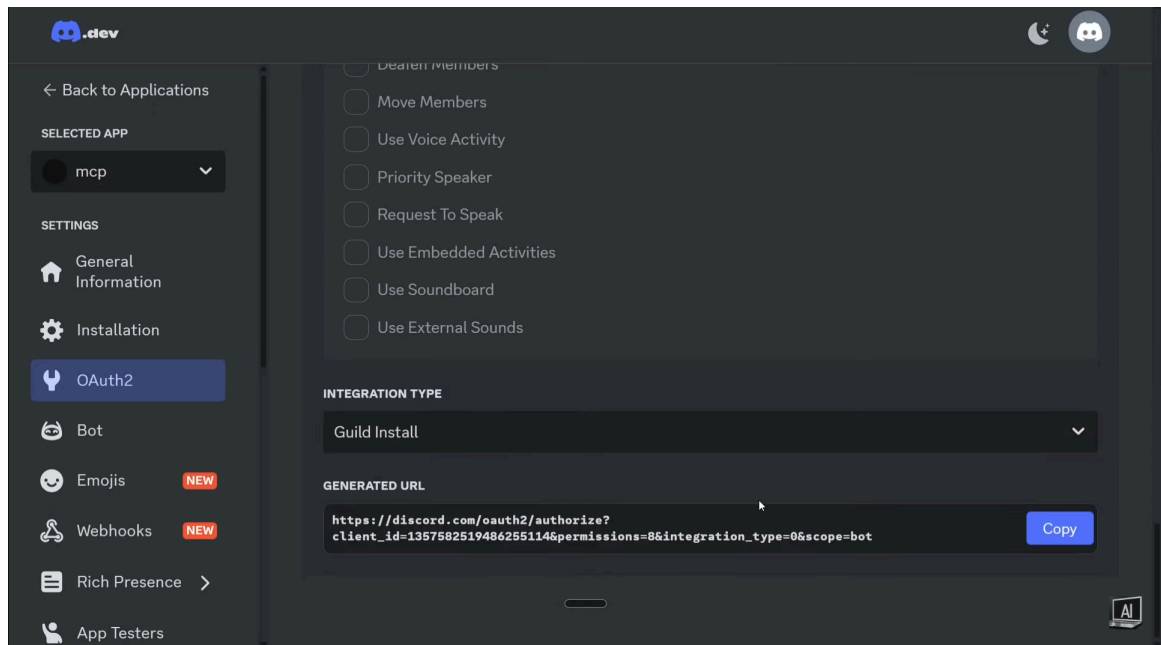
- D. As for final steps of bot creation, select **“bot” checkbox under OAuth2 URL Generator** section.



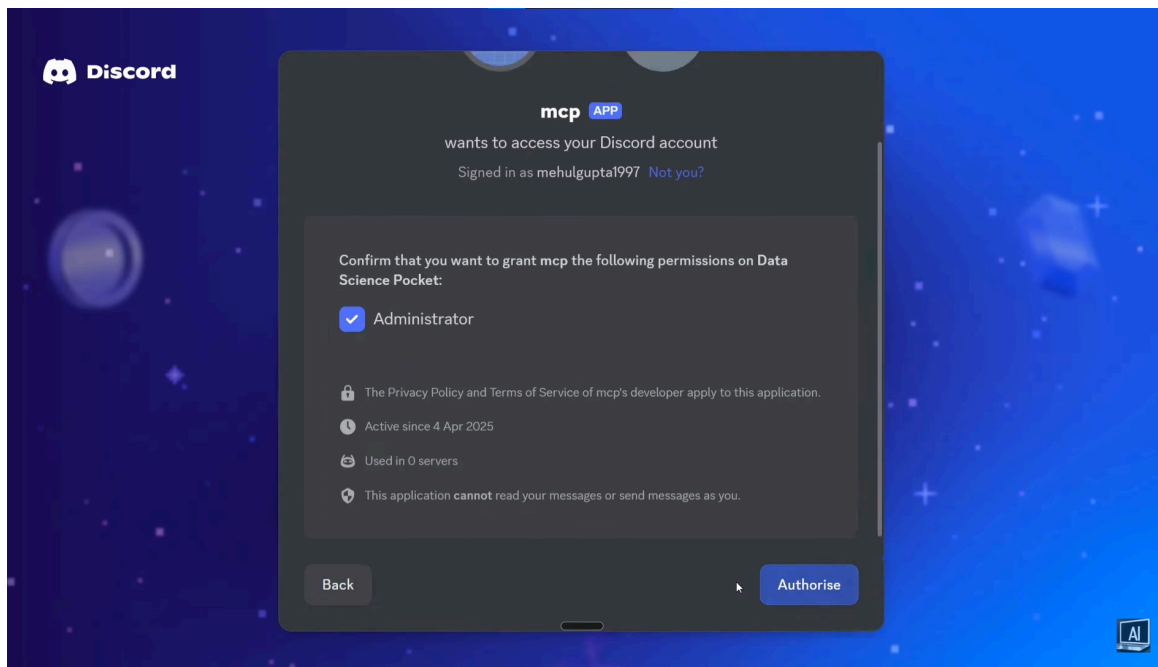
- E. Then select **“Administrator”** under Bot permissions section



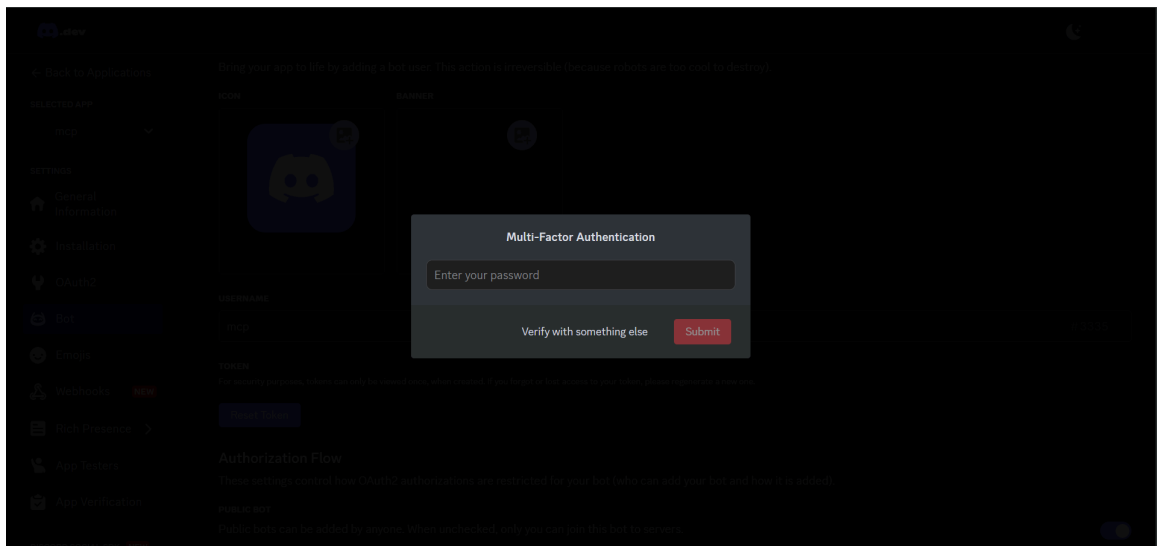
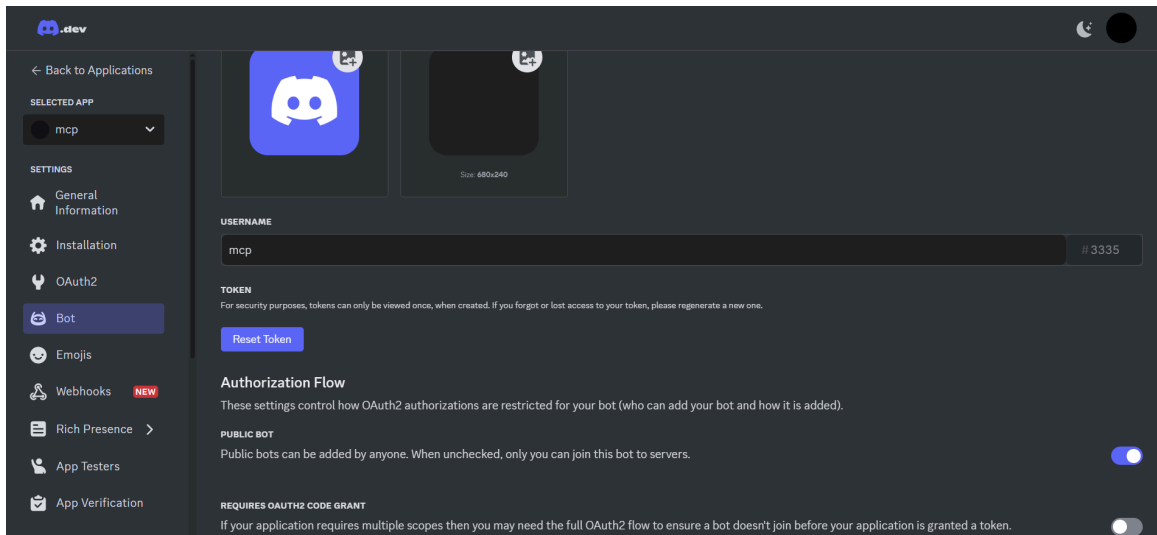
F. Copy the final bot URL at the bottom.



G. Paste this link into your browser, and it will prompt you to authorize the Bot to access Discord Account. Go ahead and authorize the request.



H. Now go back to your Bot page and Click on Reset Token. Go ahead and authorize the reset and remember to copy the token that will be shown.



I. To trigger this bot to be online, you need to keep a discord process running. You can create a python file to create this discord process. Just download the file and run it using **“python discord_1.py”** in cmd. Find the bot file here :

https://github.com/mehulGupta7991/discord_bot/blob/main/discord_1.py

J: Create a .env file and fill in the below details

```
DISCORD_BOT_TOKEN="your bot id"
TESTING_GUILD_ID="your server id"
```

.env file

11.2.2 Steps

1. First we need to clone the MCP server. Clone the server using the git command.

```
git clone https://github.com/hanweg/mcp-discord.git
cd mcp-discord
```

2. Once everything is setup, just edit your claude config json file with the following

```
{
  "mcpServers": {
    "discord": {
      "command": "uv",
      "args": [
        "--directory",
        "C:\\PATH\\TO\\mcp-discord",
        "run",
        "mcp-discord"
      ],
      "env": {
        "DISCORD_TOKEN": "your_bot_token"
      }
    }
  }
}
```

claude_desktop_config.json

And this should be it.

M Introduce yourself in the newly created text channel and tell me what is the purpose of the channel.channel id 1357581978144342067

I'll send an introduction message to the new channel explaining its purpose.

View result from `send_message` from `discord` (local) >

I've sent an introduction message to the new "Testing MCP" channel explaining that it's been created to test and demonstrate the Message Command Panel functionality. The message encourages users to use this space for experimenting with Discord bot commands and features without disrupting other channels.

Is there anything else you'd like me to add to the channel or any other actions you'd like me to take?

Now, let AI control your Discord community and you chill and relax on your recliner. Next up we have Twitter or we should say X MCP server. Very exciting !!

Chapter 12: Twitter MCP Server

AI That Tweets for You? Yep, It's Real (and Kinda Awesome)

12.1 Introduction

The Twitter MCP Server opens up the world of social media to your AI assistant, allowing it to post tweets and search Twitter directly through natural language. Whether you want your assistant to share updates, promote a new product, or track trends across the platform, the Twitter MCP makes it simple. Your assistant can craft tweets, post them instantly, and even search for recent discussions or hashtags—all without you ever needing to break a sweat.

In the previous chapter, we explored how the Discord MCP helps your assistant engage in real-time conversations, manage channels, and support community interactions. Now, we move from chat-based communities to the broader social sphere. With Twitter MCP, your assistant becomes a social media manager—posting on your behalf, tracking brand mentions, and staying up-to-date on public sentiment, all through a few natural prompts.

12.1.1 Twitter MCP Tools

The Twitter MCP server gives your assistant two main capabilities:

1. **post_tweet**: It can publish new tweets, letting you share announcements or thoughts directly through natural language prompts.
2. **search_tweets**: It can search Twitter for recent tweets about any topic, user, or hashtag, helping you keep a finger on the pulse of public conversations.

12.1.2 Real-World Applications

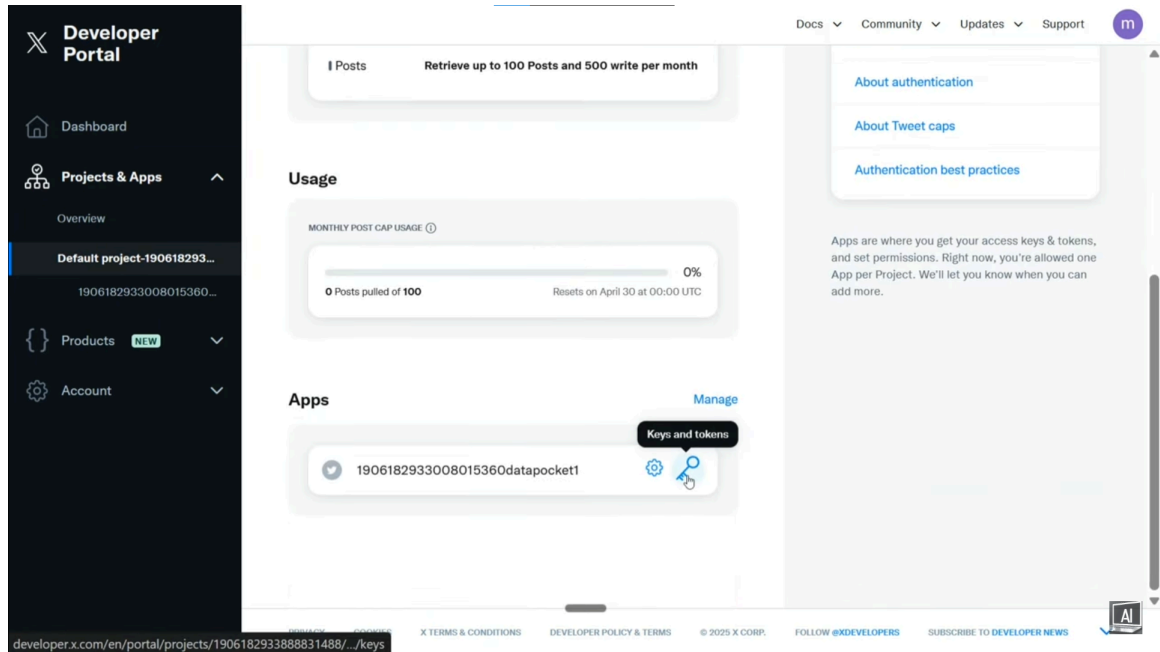
Say hello to Jordan, a freelance digital marketer who manages multiple client accounts. With the Twitter MCP connected, Jordan can now simply instruct Claude: “Post a tweet for @clientbrand: New feature just dropped! Check it out here: [link].” Within seconds, it's live.

Later in the day, Jordan wants to check on how a campaign is trending. Instead of scrolling through Twitter manually, they ask Claude, “Search for tweets mentioning #SummerSale2025.” Claude returns a list of recent posts and replies related to the campaign, saving Jordan time and keeping them informed.

12.2 Step-by-Step Installation

12.2.1 Pre-requisites

1. A **Twitter Developer** account at <https://developer.twitter.com/en/portal/dashboard>
2. **Generate API credentials:** API Key, API Secret Key, Access Token, and Access Token Secret, click on Keys and Tokens under Projects & Apps option.



3. Regenerate Consumer Keys to get your API Key and API Secret Key. Similarly regenerate Access Token and Secret under Authentication Tokens to get your Access Token, and Access Token Secret.

12.2.2 Steps

1. Update your Claude Desktop configuration file with the server details and credentials:

```
{
  "mcpServers": {
    "twitter-mcp": {
      "command": "npx",
      "args": ["-y", "@enescinar/twitter-mcp"],
      "env": {
        "API_KEY": "your_api_key_here",
        "API_SECRET_KEY": "your_api_secret_key_here",
        "ACCESS_TOKEN": "your_access_token_here",
        "ACCESS_TOKEN_SECRET": "your_access_token_secret_here"
      }
    }
  }
}
```

```
}  
}  
}
```

claude_desktop_config.json

Not as tough as others to set up but very effective. To manage an entire Twitter account without man-handling is actually terrific and a boon for social media influencers.

M Post a tweet saying how model context protocol will change the world of AI agents.

I'd be happy to post a tweet about model context protocol and AI agents. Let me do that for you now.

[View result from post_tweet from twitter-mcp \(local\) >](#)

I apologize for the error. My tweet was too long - tweets must be 280 characters or less. Let me post a shorter version:

[View result from post_tweet from twitter-mcp \(local\) >](#)

Great! Your tweet about Model Context Protocol has been posted successfully. The tweet highlights how MCP is creating a standardized foundation for AI agents to better understand their environment and capabilities, which will enable more autonomous and interoperable AI systems.

Chapter 13: LinkedIn Automation using Zapier MCP

Your next big career move starts with smarter workflows powered by AI.

13.1 Introduction

With the LinkedIn MCP powered by Zapier, your AI assistant can now help you engage with your professional network automatically. Whether you're sharing updates, posting about a new role, or promoting content, LinkedIn automation via Claude and Zapier makes it seamless. Building on what we learned in the Twitter MCP chapter, this integration focuses on boosting your presence on LinkedIn using natural language commands.

Zapier acts as a bridge between your AI assistant and LinkedIn, making it possible to trigger LinkedIn actions such as posting updates—just by prompting Claude.

But what is Zapier exactly?

Zapier is a workflow automation tool that connects to hundreds of platforms. When connected via the Zapier MCP server, Claude can issue commands through Zapier. For eg.: to create or update LinkedIn posts automatically.

For example, you can say:

"Post on LinkedIn: Just published a new article on AI in finance. Read more here: [link]"

Claude sends the request through the Zapier MCP, and your LinkedIn account posts the update within seconds.

13.1.1 Zapier MCP Tools (LinkedIn)

The Zapier MCP server gives your assistant multiple capabilities across multiple platforms::

1. **add_actions**: Add new actions to your MCP provider
2. **edit_actions**: Edit your existing MCP provider actions
3. **linkedin_create_share_update** : Post a status update on linkedin platform for sharing some content.

Don't worry, there can be hundreds of such tools that you can use to automate workflows.

13.1.2 Real-World Applications

Meet Priya, a data science student preparing for job applications. She uses Claude with Zapier MCP to stay current on hiring trends and showcase her skills.

One morning she says: "Search LinkedIn for the latest job postings related to data science internships in Bangalore." Claude retrieves a curated list of recent openings. Next, Priya says: "Create a LinkedIn post: Actively looking for data science internships in Bangalore. Passionate about machine learning, statistics, and real-world problem-solving. Let's connect!" Moments later, Claude confirms the post is live. Priya's proactive outreach begins to attract attention from recruiters.

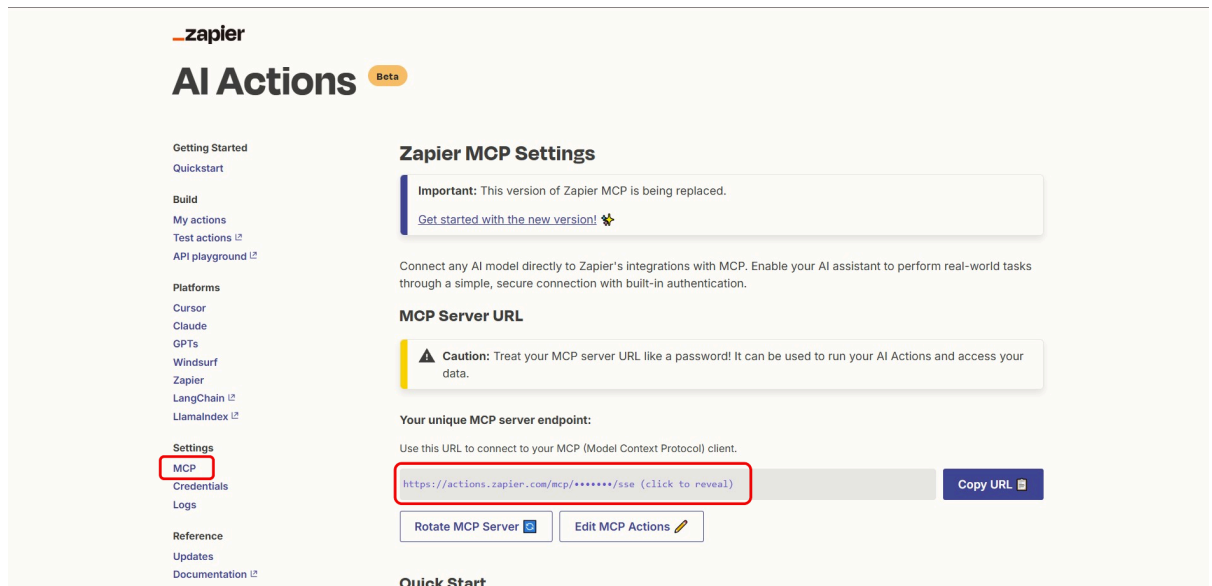
13.2 Step-by-Step Installation

13.2.1 Pre-requisites

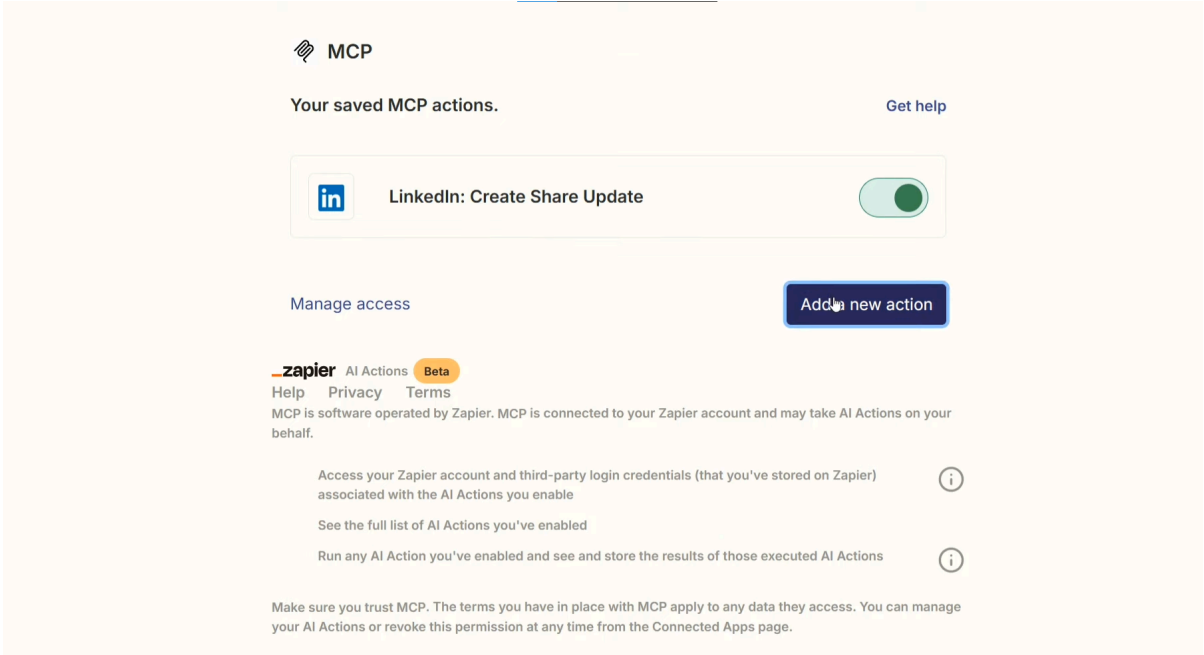
1. A Zapier account at <https://zapier.com/mcp>.

13.2.2 Steps

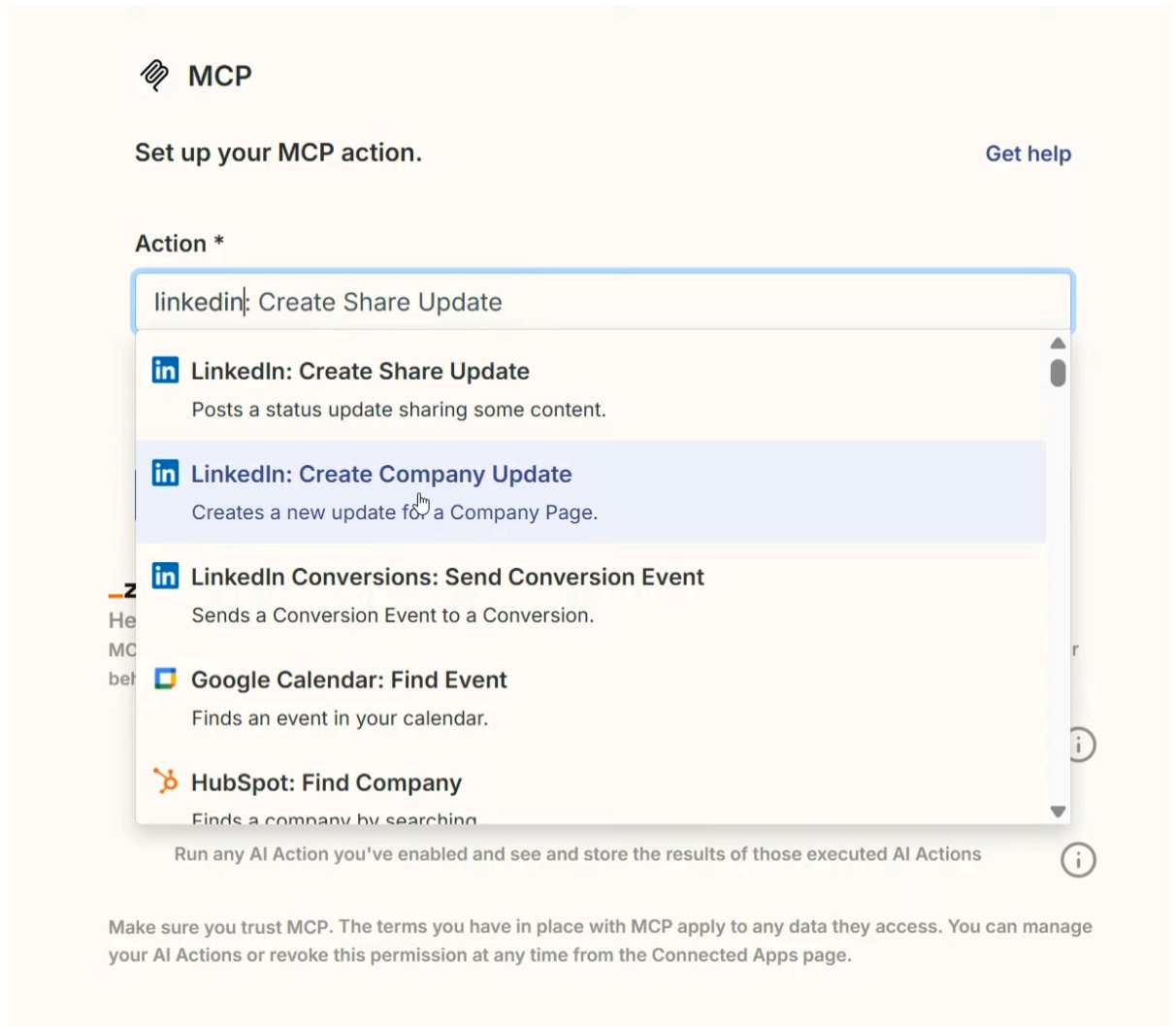
1. Open the Zapier actions page at **actions.zapier.com/settings/mcp/** to start your automation journey. Click on the MCP tab under settings on the left.
2. Copy the MCP server endpoint. This is confidential information so do be careful not to share this endpoint.



3. Once done, click on the **Edit MCP Actions** Button. Here you can see your existing saved MCP actions. To add a new one, **Click Add new section**



4. Start typing in the field for your favourite platform. For us, we will search for LinkedIn.



5. To complete adding actions, you need to provide some simple configuration to your MCP action like LinkedIn Account, commenting permissions and visibility scopes.

Click **Enable action** to finish setting up the MCP.

The screenshot shows the Zapier MCP configuration page. At the top, there's a heading 'MCP' with a gear icon. Below it, the instruction 'Set up your MCP action.' is followed by a 'Get help' link. The 'Action *' section has a dropdown menu set to 'LinkedIn: Create Share Update' and a 'Clear' button. A description below reads 'Posts a status update sharing some content.' The 'LinkedIn Account *' section features a dropdown menu with 'LinkedIn [profile picture]' and buttons for 'Connect new', 'Reconnect', and 'Refresh', along with a 'Manage accounts' link. The 'Comment *' section has a dropdown menu set to 'Have AI guess a value for this field'. The 'Visible To *' section also has a dropdown menu set to 'Have AI guess a value for this field'. Below these, there's a 'Show all options' link and a checkbox for 'Require preview before running AI Action (learn more)'. At the bottom, there are three buttons: '< Back', 'Delete', and 'Enable action'.

6. To let Claude know about the Zapier integration all you need to do is edit your `claude_desktop_config.json` file.

```
{
  "mcpServers": {
    "zapier-mcp": {
      "command": "npx",
      "args": [
        "mcp-remote",
        "<Zapier MCP endpoint>"
      ]
    }
  }
}
```

```
} 
```

claude_desktop_config.json

Simple to set up, smooth to run. LinkedIn workflows without the hassle — a game-changer for career growth and networking.

AI-Generated LinkedIn Post ▾

M

Create a LinkedIn Post stating this post is AI generated and has been posted using Claude AI LinkedIn MCP Server.

I'd be happy to create a LinkedIn post for you. Here's a professional update that acknowledges it's AI-generated and posted through Claude's LinkedIn integration:

[View result from linkedin_create_share_update from zapier-mcp \(local\) >](#)

Your LinkedIn post has been published successfully! You can view it at the URL provided in the results.



 [Retry ▾](#)

Claude can make mistakes. Please double-check responses.

Part 4: Programming MCPs

Chapter 14: SQL MCP server

No more last minute SQL query writing from now on

14.1 Introduction

The SQL Model Context Protocol (MCP) lets your AI assistant interact with databases like a pro—without ever needing to write raw SQL by hand. You can just ask it to pull customer data, update records, create a new table, or even explore the structure of your database, and it'll handle the rest. Whether you're digging into product analytics or logging new business insights from your data, the assistant knows exactly how to navigate and work with the database behind the scenes.

It's not just about running queries either. Your assistant can browse what tables exist, understand the layout of each one, and even add takeaways it uncovers during analysis into a shared insights log. It's like having a database expert who also thinks strategically—ready to help with everything from day-to-day data work to uncovering the story behind the numbers.

14.1.1 SQL MCP Tools

Following are some of the essential utilities provided for executing SQL queries to retrieve, modify, or structure data within the database.

A. Query Tools

1. **read_query**
Run SELECT queries to retrieve data from the database. Results are returned as a list of objects.
2. **write_query**
Run INSERT, UPDATE, or DELETE queries to modify data. Returns a message showing how many rows were affected.
3. **create_table**
Create new tables in the database using a CREATE TABLE SQL command. Returns confirmation that the table was created.

B. Schema Tools

4. **list_tables**
Get a list of all tables in the database.

5. **describe_table**

View column names and data types for a specific table.

C. Analysis Tools

6. **append_insight**

Add new business insights to a memo file. Also updates the memo://insights resource.

With these tools, your assistant can efficiently manage SQL tasks, from data retrieval to report generation, all through simple commands.

14.1.2 Real-World Applications

Lena, a marketing analyst, had to put together a last-minute report on customer trends for an important client meeting. Let's just say, she wasn't looking forward to digging through databases manually. So she opened up her SQL MCP assistant and turned to it for help.

- "Show me all users who signed up in the last 30 days."
- "Add a new order to the orders table with order_id 2045 and status 'shipped'."
- "Create a table for customer support tickets with fields for id, subject, and timestamp."
- "What tables do we have in this database?"
- "What's the structure of the transactions table?"
- "Take a look at the sales data and let me know if there are any trends or insights."

Everything analysis is just a prompt away

14.2 Step-by-Step Installation

14.2.2 Steps

1. Assuming you already have UV installed from previous sections. Set the following config in your claude config json file. Replace “~/test.db” with your SQLite database object. In case of any error, provide full paths for uv or the db.

For Windows platform:

```
# Add the server to your claude_desktop_config.json
"mcpServers": {
  "sqlite": {
    "command": "uv",
    "args": [
      "--directory",
      "parent_of_servers_repo/servers/src/sqlite",
      "run",
      "mcp-server-sqlite",
      "--db-path",
      "~/test.db"
    ]
  }
}
```

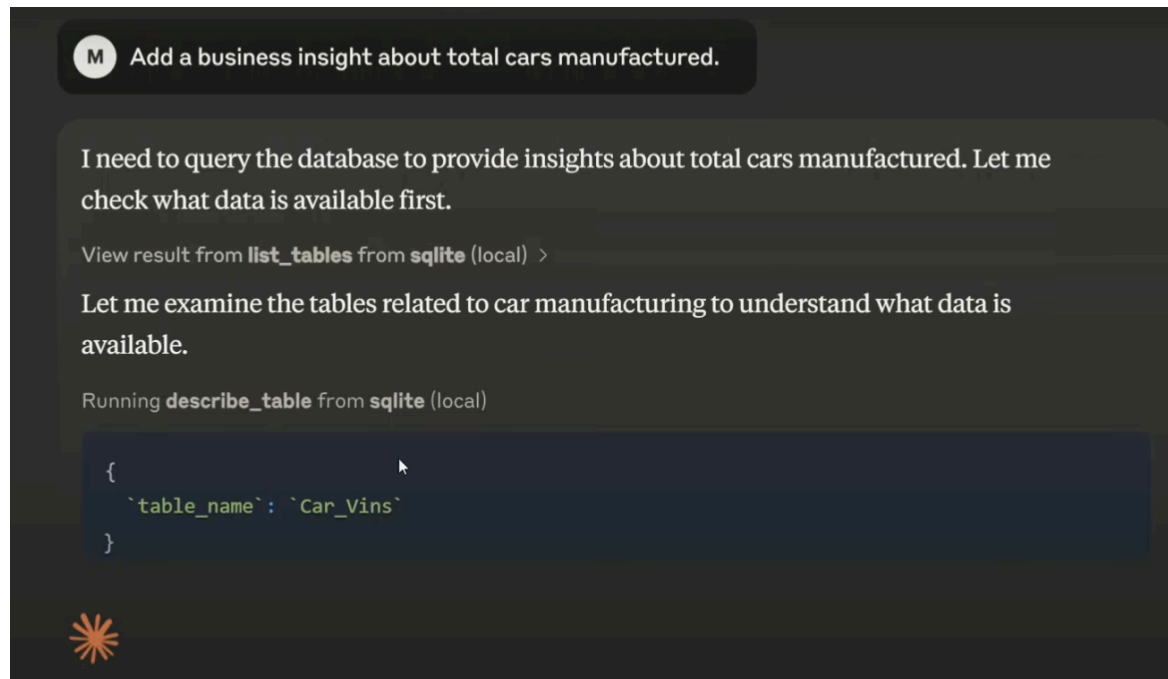
claude_desktop_config.json

For docker :

```
# Add the server to your claude_desktop_config.json
"mcpServers": {
  "sqlite": {
    "command": "docker",
    "args": [
      "run",
      "--rm",
      "-i",
      "-v",
      "mcp-test:/mcp",
      "mcp/sqlite",
      "--db-path",
      "/mcp/test.db"
    ]
  }
}
```

claude_desktop_config.json

Easy peasy



Now once SQL MCP is enabled, your assistant handles all the SQL magic behind the scenes—whether you're reading data, updating records, creating tables, or just trying to get a feel for what's in your database. No need to stress over syntax or commands; just talk like you normally would, and it'll take care of the rest.

Chapter 15: GitHub MCP server

Ever imagined AI will be doing Git pull, Git push?

15.1 Introduction

The **GitHub Model Context Protocol (MCP)** Server bridges your AI assistant with GitHub, enabling seamless interaction with repositories, issues, pull requests, and more through natural language commands. Whether you're managing open-source projects, automating workflows, or reviewing code, GitHub MCP simplifies these tasks, allowing your assistant to handle them effortlessly.

15.1.1 GitHub MCP Tools

Here's a concise list of tools available in the GitHub MCP server, each designed to help your assistant interact with GitHub repositories:

1. **create_or_update_file**
Create or update a single file in a repository.
2. **create_or_update_files**
Create or update multiple files in a repository.
3. **get_file**
Retrieve the contents of a file from a repository.
4. **delete_file**
Delete a file from a repository.
5. **get_repo**
Retrieve metadata about a repository.
6. **create_repo**
Create a new repository under a user or organization.
7. **search_code**
Search for code within repositories.
8. **search_issues_and_prs**
Search for issues and pull requests within repositories.

9. **search_users**

Search for users on GitHub.

These tools enable your assistant to perform a wide range of actions on GitHub, from managing files and repositories to searching for code and users..

15.1.2 Real-World Applications

A. Business Use Case: Priya's Streamlined Project Management

Priya, a project manager at a tech startup, oversees multiple development teams and manages progress across various repositories. With GitHub MCP integrated into her AI assistant, she efficiently monitors and handles development workflows.

The tool allows her to summarize recent commits in specific repositories, list all issues labeled as bugs in designated projects, and merge open pull requests across different codebases. This simplifies tracking updates, identifying critical tasks, and ensuring smooth collaboration among teams

Here are some examples:

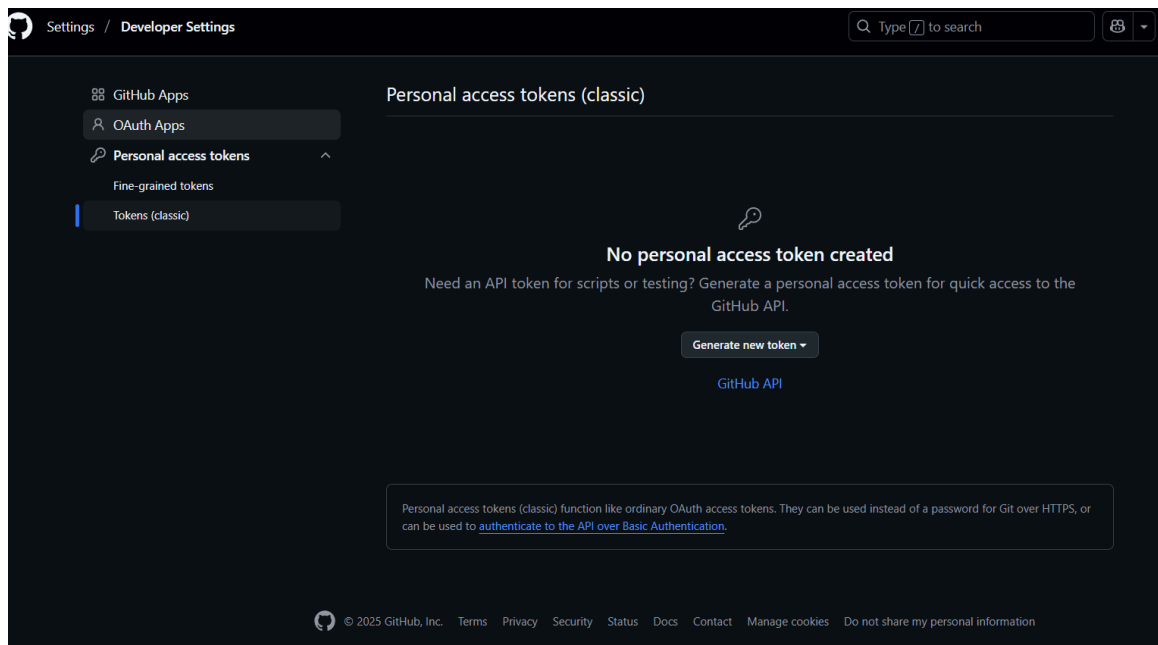
- "Create a new repository called 'project-alpha'."
- "Open an issue in 'project-alpha' titled 'Implement authentication'."
- "List all open pull requests in 'project-alpha'."
- "Merge the pull request titled 'Add user login feature' in 'project-alpha'."

15.2 Step-by-Step Installation

15.2.1 Pre-requisites

Before diving in, ensure you have the following:

1. **GitHub Account:** An active GitHub account.
2. **Personal Access Token:** Generate a token with appropriate permissions (e.g., repo, workflow). You can get the personal Access Token at <https://github.com/settings/tokens>



Personal Token for Github

3. **Node.js and npm:** Required to run the MCP server.

15.2.2 Steps

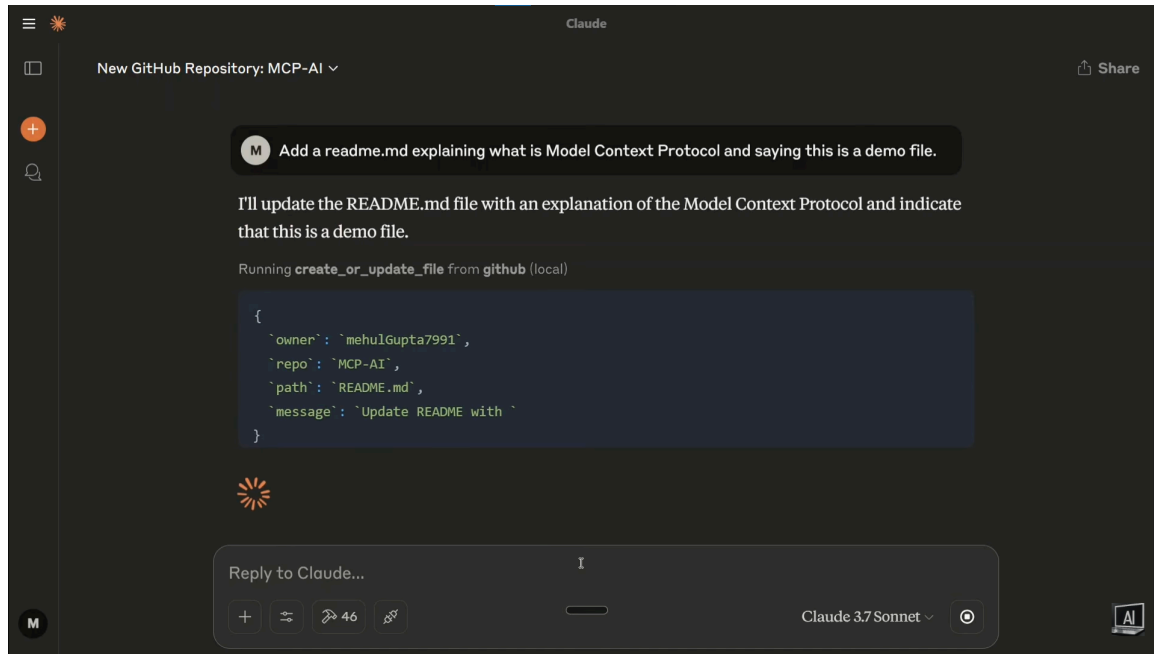
1. Just add the following config to your claude config json file and you will be good to start using your Github MCP server.

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/github-mcp-server"],
      "env": {
        "GITHUB_TOKEN": "your-personal-access-token"
      }
    }
  }
}
```

claude desktop_config json

And we're done. Your assistant will handle the rest—creating repositories, managing issues and pull requests, and more. The below screenshot is just an example of what AI can

achieve now.



Github MCP in action

Chapter 16: Docker MCP server

AI Controlling Docker? Are you kidding me?

16.1 Introduction

The Docker MCP Server integrates the Model Context Protocol (MCP) with Docker, enabling AI assistants to manage containerized environments through natural language commands. This integration allows for seamless execution of tasks such as launching containers, deploying applications, and retrieving logs, all within isolated Docker environments.

By leveraging Docker's capabilities, the Docker MCP Server provides a standardized interface for AI models to interact with various tools and data sources securely and efficiently. This setup is particularly beneficial for developers and DevOps teams seeking to automate workflows and maintain consistency across development environments.

With the Docker MCP Server, AI assistants can perform complex operations, such as managing Docker Compose stacks and monitoring container statuses, enhancing productivity and streamlining development processes.

16.1.1 Docker MCP Tools

Here's a concise list of tools available in the GitHub MCP server, each designed to help your assistant interact with GitHub repositories:

- **create-container**
Create a new standalone Docker container
- **deploy-compose**
Deploy a Docker Compose stack
- **get-logs**
Retrieve the latest logs for a specified Docker container
- **list-containers**
List all Docker containers

16.1.2 Real-World Applications

DevOps Automation: Meet Arjun, a DevOps Engineer

Arjun, a DevOps engineer at a mid-sized tech company, was responsible for managing multiple microservices across various environments. His daily tasks included deploying services, monitoring container health, and managing logs—a routine that was both time-consuming and prone to human error

To streamline his workflow, Arjun integrated Docker MCP into his toolset. Now, instead of manually executing commands, he simply communicates with his AI assistant in natural language.

For instance, he might say, "Deploy the latest version of the authentication service," and the assistant handles the deployment process seamlessly. If he needs to check on a service, he can ask, "What's the status of the payment gateway container?" and receive an immediate update.

This integration not only reduced the manual overhead but also minimized errors, allowing Arjun to focus on more strategic initiatives.

16.2 Step-by-Step Installation

16.2.1 Pre-requisites

You will need a few prerequisites before getting started with Docker MCP.

1. **Docker Engine or Docker Desktop:** Install Docker on your system to manage containers.: <https://www.docker.com/>
2. **uv Package Manager:** Install uv, a fast Python package manager, to handle dependencies efficiently.

16.2.2 Steps

1. All you need to do is to add the Docker MCP to your claude config files.

```
{
  "mcpServers": {
    "docker-mcp": {
      "command": "uvx",
      "args": [
        "docker-mcp"
      ]
    }
  }
}
```

claude_desktop_config.json

Done and Dusted !



Docker is actually a difficult technology to learn and with AI's assistance, many programmers can leverage the power of containerization with just a prompt.

Chapter 17: Jupyter MCP server

Data Scientists are gonna love this one !

17.1 Introduction

The Jupyter MCP Server integrates the Model Context Protocol (MCP) with JupyterLab, enabling AI assistants to interact with Jupyter notebooks through natural language commands. This integration allows for seamless execution of tasks such as running code cells, managing notebook content, and retrieving outputs, all within the Jupyter environment.

By leveraging Jupyter's capabilities, the Jupyter MCP Server provides a standardized interface for AI models to interact with computational notebooks securely and efficiently. This setup is particularly beneficial for data scientists and researchers seeking to automate workflows and maintain consistency across analysis environments.

17.1.1 Jupyter MCP Tools

Here's a concise list of tools available in the Jupyter MCP server, each designed to help your assistant interact with Jupyter notebooks:

1. **add_execute_code_cell**
Add and execute a code cell in a Jupyter notebook.
2. **add_markdown_cell**
Add a markdown cell in a Jupyter notebook.

17.1.2 Real-World Applications

Data Scientist's Agile Analysis During Meetings

Meet Palak, a data scientist at a marketing firm. During a strategy meeting, her team discusses the latest campaign's performance metrics. Instead of postponing the analysis, Palak leverages the Jupyter MCP Server integrated with her AI assistant.

She simply says, "Insert a code cell to calculate the average click-through rate from last month's data." The assistant adds and executes the code cell in her notebook, providing immediate results. Later, she requests, "Add a markdown cell summarizing the key insights," and the assistant inserts a well-formatted summary.

This real-time interaction allows Palak to contribute valuable insights during the meeting without manual coding, enhancing decision-making and collaboration.

17.2 Step-by-Step Installation

17.2.1 Pre-requisites

1. You will need to have Docker Desktop installed before getting started with Jupyter MCP from <https://www.docker.com/products/docker-desktop/>
2. You will also need the Jupyter session token. You will find this in your anaconda prompt or relevant terminal. We will be referring to this as MY_TOKEN from hereon.

```
Anaconda Prompt - jupyter lab
I 2025-04-19 03:16:29.199 ServerApp] notebook_shim | extension was successfully linked.
I 2025-04-19 03:16:29.200 ServerApp] panel.io.jupyter_server_extension | extension was successfully linked.
I 2025-04-19 03:16:29.258 ServerApp] notebook_shim | extension was successfully loaded.
I 2025-04-19 03:16:29.260 ServerApp] jupyter_lsp | extension was successfully loaded.
I 2025-04-19 03:16:29.261 ServerApp] jupyter_server_terminals | extension was successfully loaded.
I 2025-04-19 03:16:29.270 LabApp] JupyterLab extension loaded from C:\Users\User\anaconda3\lib\site-packages\jupyterlab
I 2025-04-19 03:16:29.271 LabApp] JupyterLab application directory is C:\Users\User\anaconda3\share\jupyter\lab
I 2025-04-19 03:16:29.272 LabApp] Extension Manager is 'pypi'.
I 2025-04-19 03:16:29.631 ServerApp] jupyterlab | extension was successfully loaded.
I 2025-04-19 03:16:29.638 ServerApp] notebook | extension was successfully loaded.
I 2025-04-19 03:16:29.639 ServerApp] panel.io.jupyter_server_extension | extension was successfully loaded.
I 2025-04-19 03:16:29.641 ServerApp] Serving notebooks from local directory: C:\Users\User
I 2025-04-19 03:16:29.641 ServerApp] Jupyter Server 2.14.1 is running at:
I 2025-04-19 03:16:29.641 ServerApp] http://localhost:8888/lab?token=d3b492a75211f98d0864e1be283a085b8b3367a8d74ca5da
I 2025-04-19 03:16:29.642 ServerApp] http://127.0.0.1:8888/lab?token=d3b492a75211f98d0864e1be283a085b8b3367a8d74ca5da
I 2025-04-19 03:16:29.642 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
tion).
C 2025-04-19 03:16:29.693 ServerApp]

To access the server, open this file in a browser:
file:///C:/Users/User/AppData/Roaming/jupyter/runtime/jpserver-12640-open.html
Or copy and paste one of these URLs:
http://localhost:8888/lab?token=d3b492a75211f98d0864e1be283a085b8b3367a8d74ca5da
http://127.0.0.1:8888/lab?token=d3b492a75211f98d0864e1be283a085b8b3367a8d74ca5da
W 2025-04-19 03:16:29.806 ServerApp] Could not determine npm prefix: [WinError 2] The system cannot find the file speci
fied
I 2025-04-19 03:16:29.932 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-
```

17.2.2 Steps

1. Install Required Packages in your jupyter environment

```
pip install jupyterlab jupyter-collaboration ipykernel
pip uninstall -y pycrdt datalayer_pycrdt
pip install datalayer_pycrdt
```

2. Start JupyterLab with the provided configurations

```
jupyter lab --port 8888 --IdentityProvider.token MY_TOKEN --ip 0.0.0.0
```

Note: To allow the Docker container to communicate with your locally running JupyterLab instance, you need to make JupyterLab accessible over the network. By setting `--ip 0.0.0.0`, JupyterLab becomes accessible from outside the host machine

3. Simply add the following to your claude config json file

```
{
  "mcpServers": {
    "jupyter": {
      "command": "docker",
      "args": [
        "run",
        "-j",
        "--rm",
        "-e",
        "SERVER_URL",
        "-e",
        "TOKEN",
        "-e",
        "NOTEBOOK_PATH",
        "datalayer/jupyter-mcp-server:latest"
      ],
      "env": {
        "SERVER_URL": "http://host.docker.internal:8888",
        "TOKEN": "MY_TOKEN",
        "NOTEBOOK_PATH": "notebook.ipynb"
      }
    }
  }
}
```

claude_desktop_config.json

Note: In case you are working on linux, use the following configuration instead
CLAUDE_CONFIG=\${HOME}/.config/Claude/claude_desktop_config.json
cat <<EOF > \$CLAUDE_CONFIG

```
{
  "mcpServers": {
    "jupyter": {
      "command": "docker",
      "args": [
        "run",
        "-j",
        "--rm",
        "-e",
        "SERVER_URL",
        "-e",
        "TOKEN",
        "-e",
        "NOTEBOOK_PATH",
```

```

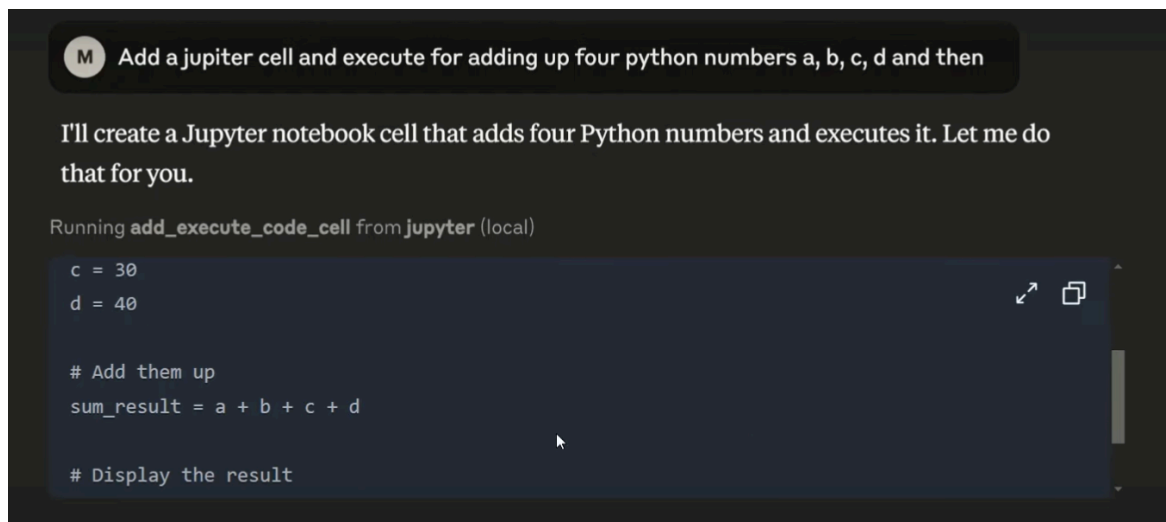
    "--network=host",
    "datalayer/jupyter-mcp-server:latest"
  ],
  "env": {
    "SERVER_URL": "http://localhost:8888",
    "TOKEN": "MY_TOKEN",
    "NOTEBOOK_PATH": "notebook.ipynb"
  }
}
}
}
EOF
cat $CLAUDE_CONFIG

```

claude_desktop_config.json for linux

Also `NOTEBOOK_PATH` will be the relative path to the notebook file from the directory where JupyterLab was launched.

And let the magic begin



Any data scientist out there is probably smiling right now. This is basically their favorite thing in the world—and when AI can pitch in, well... let's just say our tribe is one happy bunch!

Part 5: Other MCPs

Chapter 18: Blender MCP server

Now anyone can be Graphic Designer, thanks to AI

18.1 Introduction

The Blender MCP Server allows Claude to interact with Blender (free and open-source 3D computer graphics software tool set) in real time using the Model Context Protocol (MCP). Through this integration, your assistant can generate, inspect, and manipulate 3D scenes using natural language. Whether you're prototyping game assets, testing visual layouts, or building procedural models, the Blender MCP Server brings creative control into the hands of your assistant.

It supports direct object manipulation, material assignment, scene inspection, and even arbitrary Python execution inside Blender. With this setup, you can prompt Blender to generate 3D models, build environments, or apply shaders—all powered by language.

18.1.1 Blender MCP Tools

Here's a list of the core capabilities provided by the Blender MCP Server:

1. **Get_scene_info**
Get detailed information about the current Blender scene.
2. **Get_object_info**
Get detailed information about a specific object in the Blender scene.
3. **execute_blender_code**
Execute arbitrary Python code in Blender.
4. **get_polyhaven_categories**
Get a list of categories for a specific asset type on Polyhaven.

Just in case you are wondering, Poly Haven is a Blender add-on that offers high-quality HDRI maps, 3D models, and textures that work great with Blender. It's an awesome way to make your scenes look realistic without starting from scratch.

5. **search_polyhaven_assets**
Search for assets on Polyhaven with optional filtering.
6. **download_polyhaven_asset**
Download and import a Polyhaven asset into Blender.

7. **set_texture**

Apply a previously downloaded Polyhaven texture to an object.

8. **get_polyhaven_status**

Check if PolyHaven integration is enabled in Blender.

9. **get_hyper3d_status**

Check if Hyper3D Rodin integration is enabled in Blender.

Again, Hyper3D Rodin is an API add-on that lets you generate high-quality 3D models from simple text or image prompts – no modeling experience needed. It helps you to bring your ideas into 3D, fast.

10. **generate_hyper3d_model_via_text**

Generate 3D assets using Hyper3D by giving a description of the desired asset.

11. **generate_hyper3d_model_via_images**

Generate 3D assets using Hyper3D by giving images of the wanted asset.

12. **poll_rodin_job_status**

Check if the Hyper3D Rodin generation task is completed.

13. **import_generated_asset**

Import the asset generated by Hyper3D Rodin after the generation task is completed.

18.1.2 Real-World Applications

How a Designer Prototypes Game Assets with AI in Blender

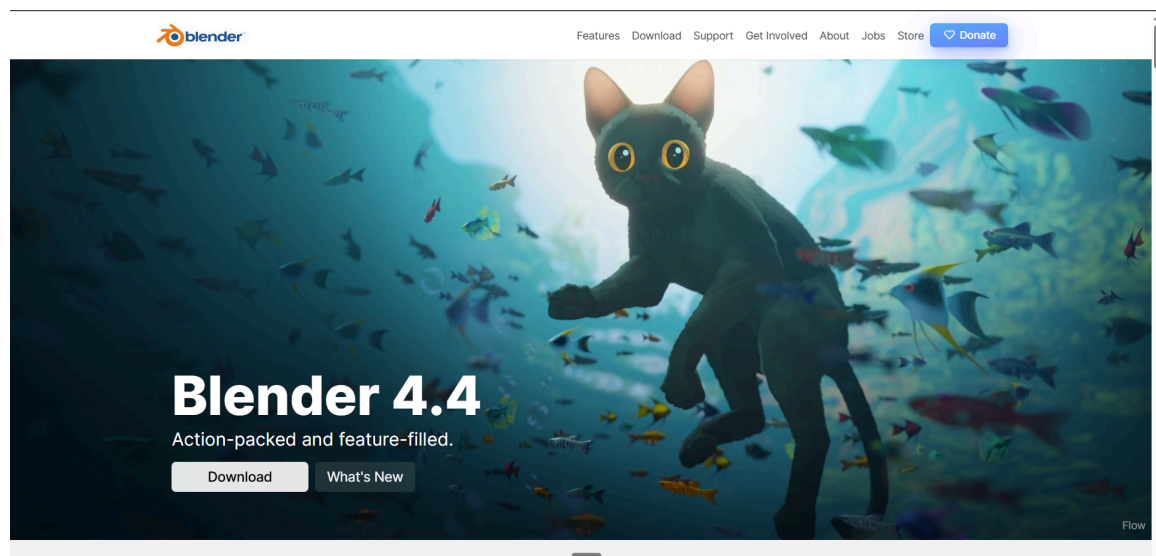
Meet Leo, a 3D artist creating assets for an indie game. Instead of manually modeling placeholder props, he opens Blender and asks Claude to “Create a low-poly campfire with flickering light.” In seconds, the scene is ready. He tweaks the shape and materials by asking Claude to “Make the fire orange and glowing,” and “Scale the logs down.”

When it’s time to populate a scene, he uses Claude to pull free models and HDRIs from Poly Haven, saving hours on setup. The AI handles the tedious setup, so Leo can focus on design.

18.2 Step-by-Step Installation

18.2.1 Pre-requisites

Just download your Blender application with version 3.0 or newer. You can download it from <https://www.blender.org/>



18.2.2 Steps

1. You have two options to install the Blender MCP Server:

A. Option 1: Install via UVX (Recommended)

```
uvx install blender-mcp
```

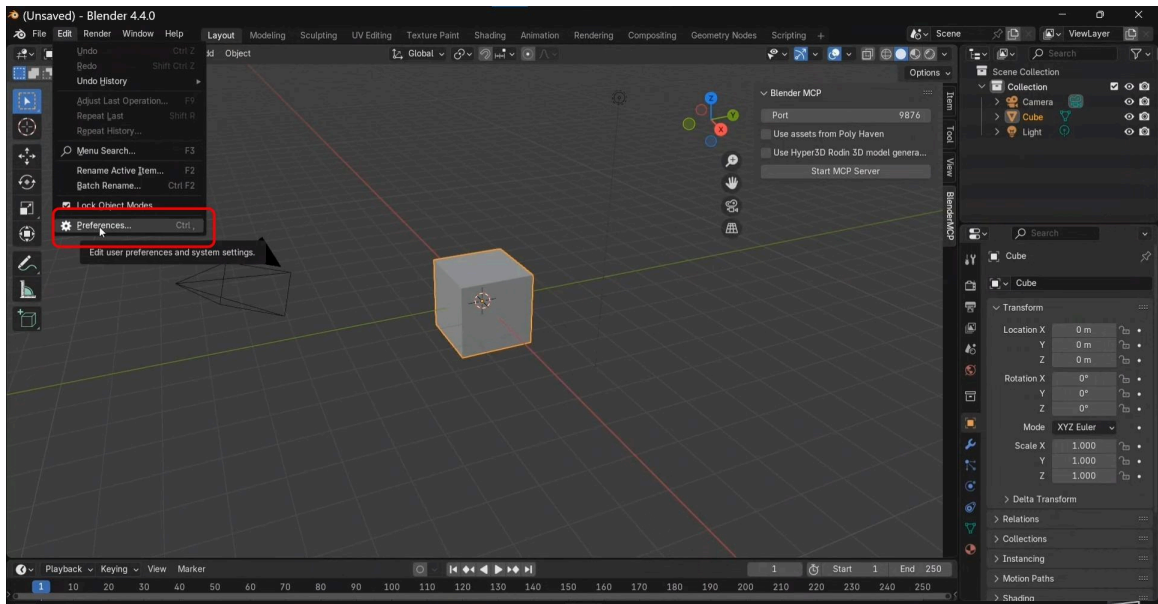
B. Option 2: Manual Setup

```
git clone https://github.com/ahujasid/blender-mcp.git
cd blender-mcp
uv venv
uv pip install -r requirements.txt
```

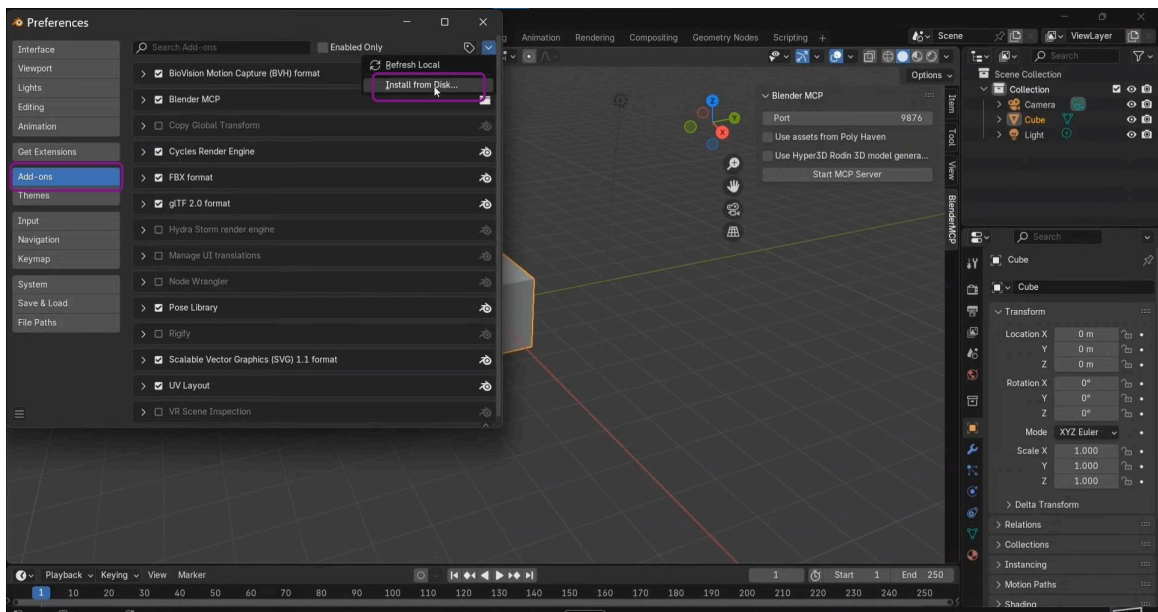
2. Add the following block to your Claude configuration JSON file:

```
{
  "mcpServers": {
    "blender": {
      "command": "uvx",
      "args": [
        "blender-mcp"
      ]
    }
  }
}
```

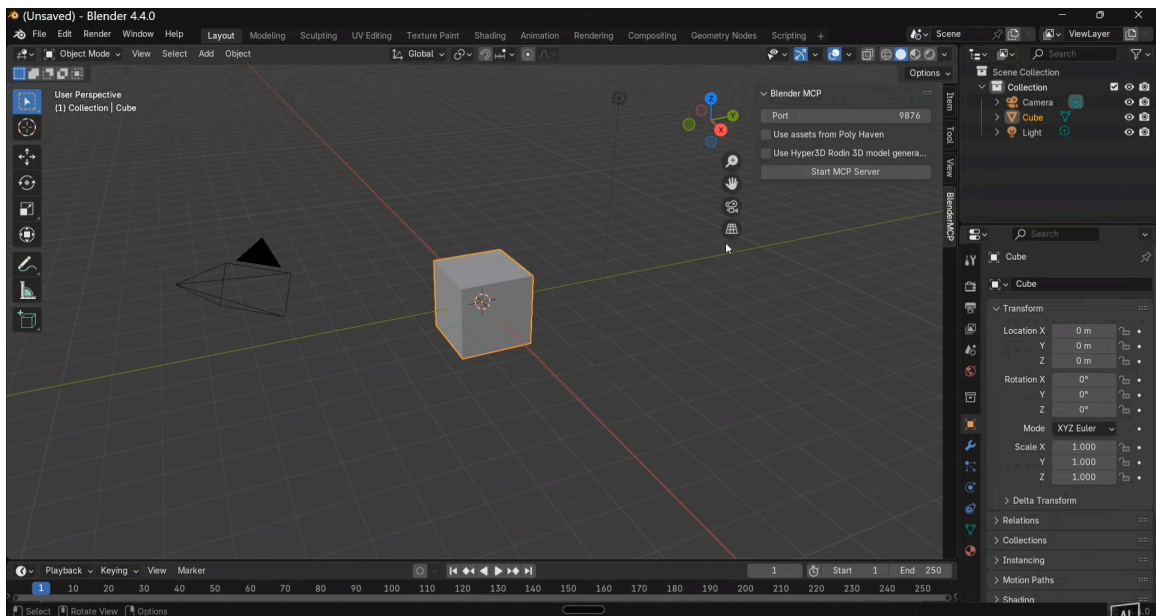

3. **Installing the Blender Addon:** Download addon.py from the Blender MCP repo
<https://github.com/ahujasid/blender-mcp/blob/main/addon.py>
4. **Add the addon file :** Open Blender → Edit → Preferences → Add-ons.



5. Click Install from disk and select the file addon.py



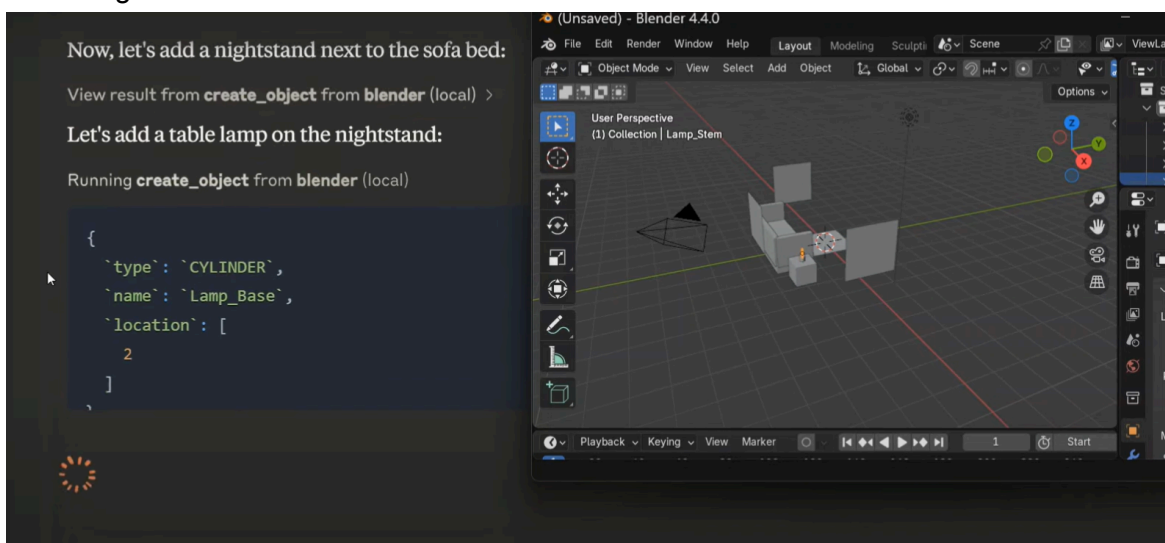
6. **Enable Interface:** Blender MCP by checkboxing it. In the 3D View, open the BlenderMCP tab and click Start MCP server



And you're a Graphic Designer now !

Once the MCP server and Blender Addon are running, you can control Blender with natural commands:

- "Create a red cube and scale it to 2x size."
- "Import an HDRI sky from Poly Haven."
- "Generate a stone archway using Hyper3D Rodin."
- "Inspect the scene and list all objects."
- "Assign a metallic shader to the sword model."



You can use Claude to automate prototyping, quickly visualize environments, or script animations and simulations. These are crazy times to be alive!

Chapter 19: Figma MCP server

What if AI can integrate with Figma?

19.1 Introduction

The Figma MCP Server enables AI to seamlessly interact with your Figma designs using the Model Context Protocol (MCP). This integration allows AI-powered coding tools like Cursor to fetch, interpret, and utilize real-time design data from your Figma files—making implementation of UI elements smoother, faster, and far more accurate than using screenshots or manual specs.

The server distills complex Figma API responses into simplified metadata, prioritizing layout, component structure, and styling information. This lightweight translation improves model focus and response quality, giving your assistant exactly what it needs to turn design intent into production-ready code.

19.1.1 Figma MCP Tools

Figma MCP comes with 2 tools as of now:

1. **Download_figma_images:** This tool helps you get SVG and PNG images (like icons or pictures) from a Figma design file. You just need to know the IDs of the images you want.
2. **Get_figma_data:** If you're not sure which image or part you need (or don't have the ID), this tool can grab the full layout details of the whole Figma file instead.

19.1.2 Real-World Applications

A. How Maya Automates Design-to-Development Workflow with Smart Tools

Meet Maya, a developer building a mobile app. Her teammate Sam, the designer, shares the latest Figma file with updated screens. Instead of asking Sam to manually export each icon and image, Maya uses an internal tool that pulls the right assets—like SVGs and PNGs—directly from the Figma server, using just their IDs.

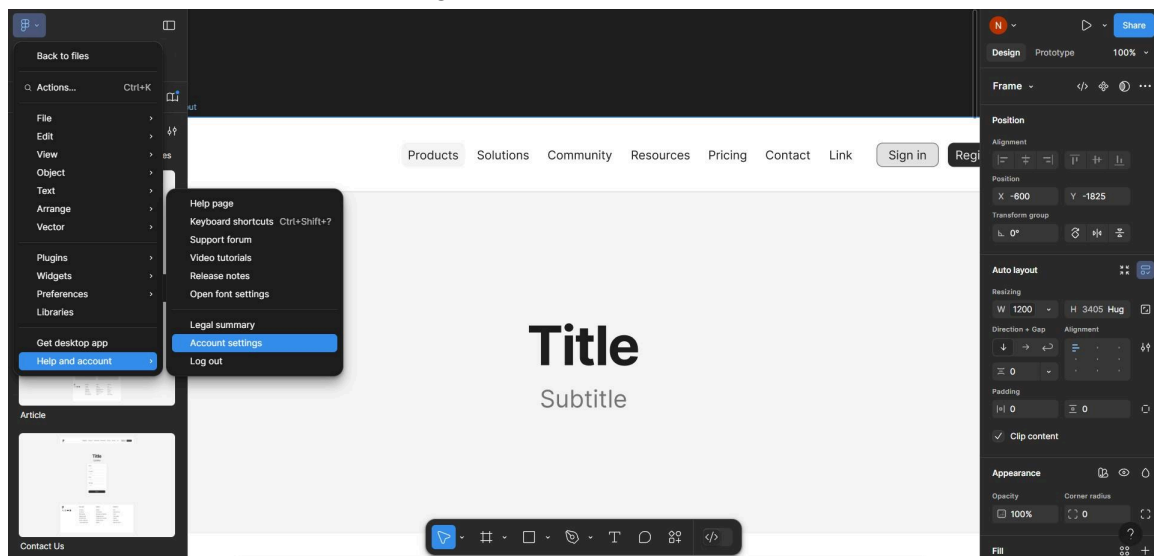
Sometimes Maya doesn't have those IDs. In those cases, she runs another tool that scans the entire design file and gives her a full picture of the layout—components, positions, structure—everything she needs to keep moving without extra coordination.

With these tools, Maya gets what she needs in seconds, avoids unnecessary back-and-forth, and keeps the workflow smooth for both her and Sam.

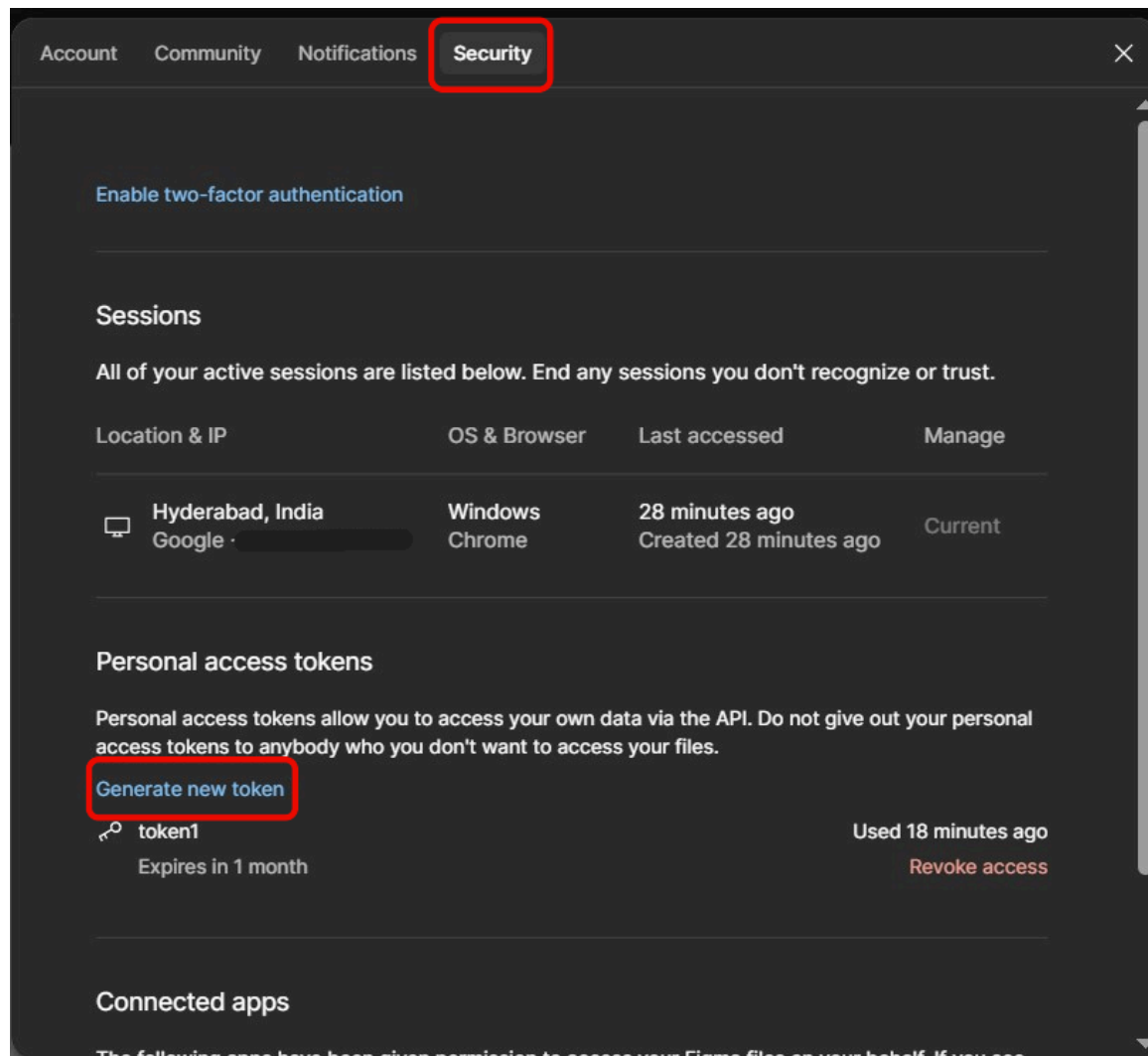
19.2 Step-by-Step Installation

19.2.1 Pre-requisites

1. Ensure you have Figma workspace ready with the reference design.
2. Procure a **personal access token**. You can do that by clicking on Main menu - Help and account - Account settings



3. Next, go to the Security tab and generate a new token. Remember to provide all the necessary accesses.



4. Remember to copy this token as this will be required to create our config file.

19.2.2 Steps

1. Once everything is setup, just edit your claude config json file with the following

A. Windows

```
{
  "mcpServers": {
    "Framelink Figma MCP": {
      "command": "cmd",
      "args": ["/c", "npx", "-y", "figma-developer-mcp", "--figma-api-key=YOUR-KEY",
"--stdio"]
    }
  }
}
```

claude_desktop_config.json

B. Linux/MacOS

```
{
  "mcpServers": {
    "Framelink Figma MCP": {
      "command": "npx",
      "args": ["-y", "figma-developer-mcp", "--figma-api-key=YOUR-KEY", "--stdio"]
    }
  }
}
```

claude_desktop_config.json

M Generate a similar design :

[https://www.figma.com/design/y1WZp60HOVpoRlfwOESsJ0/Untitled?](https://www.figma.com/design/y1WZp60HOVpoRlfwOESsJ0/Untitled?m=dev&t=BVGrbfsVjhisbTyE-1)

[m=dev&t=BVGrbfsVjhisbTyE-1](https://www.figma.com/design/y1WZp60HOVpoRlfwOESsJ0/Untitled?m=dev&t=BVGrbfsVjhisbTyE-1)

I'll help you create a similar design to the one in your Figma link. Let me first examine the Figma file to understand its structure and elements.

Running `get_figma_data` from **Framelink Figma MCP** (local)

```
{
  `fileKey`: `y1WZp60HOVpoRlfwOESsJ0`
}
```



There you go! Your AI assistant can now easily locate files, open them up, and even help organize your folders—like having a smart helper for all your file tasks. Great for getting things done quicker without the hassle of digging through directories.

Chapter 20: Filesystem MCP Server

Can AI clean up my local system?

20.1 Introduction

The Filesystem MCP Server helps AI assistants like AI understand and work with the files on your computer, just like you do. Whether it's opening files, creating new ones, or making changes to existing code, this server makes it easy for AI tools to safely explore and edit your projects—without needing you to do all the work by hand.

Think of it like a friendly helper that connects your AI assistant to your files in a clear and organized way. This means the AI can read your code, find what it's looking for, suggest improvements, or even make small edits for you—all while staying within the folders you've approved.

With this connection, your AI assistant becomes much more powerful and useful, helping you build and manage your projects faster and with fewer mistakes. It's like giving your AI a map and a toolbox so it can really understand your project and help bring your ideas to life.

20.1.1 Filesystem MCP Tools

Filesystem MCP comes with multiple tools:

1. **read_file**
Read complete contents of a file using UTF-8 encoding
2. **read_multiple_files**
Read multiple files simultaneously. Failed reads won't stop the entire operation
3. **write_file**
Create a new file or overwrite an existing one. Use with caution
4. **edit_file**
Make selective edits using advanced pattern matching and formatting. Supports line-based and multi-line changes, indentation preservation, and dry run mode for previewing changes
5. **create_directory**
Create a new directory or ensure it exists. Creates parent directories if needed and succeeds silently if the directory already exists
6. **list_directory**
List directory contents with [FILE] or [DIR] prefixes to indicate file and directory types

7. **move_file**
Move or rename files and directories. Fails if the destination already exists
8. **search_files**
Recursively search for files and directories starting from a given directory. Supports include/exclude patterns and case-insensitive matching
9. **get_file_info**
Retrieve detailed metadata about a file or directory including size, timestamps, type, and permissions
10. **list_allowed_directories**
List all directories that the server is allowed to access for read/write operations

20.1.2 Real-World Applications

A. Effortless Refactoring: Transforming a Large Codebase with Smart Tools

Meet Jordan , a full-stack developer working on a growing web application with a complex codebase spread across dozens of directories. Recently, the team decided to update their component naming conventions to improve readability and maintainability.

Instead of manually searching through hundreds of files or writing fragile scripts, Jordan uses an AI assistant powered by the Filesystem MCP Server . The assistant scans the entire project using the search_files tool to locate relevant files, then previews changes using edit_file in dry-run mode. This lets Jordan see exactly what will change before applying any edits.

With just a few prompts, Jordan automates what would have taken hours of careful editing—ensuring consistency across the codebase while avoiding costly mistakes.

B. Welcome to the Team: Guiding New Developers with Smart Tools

When Alex, a senior developer, brings on new hires, he used to spend hours walking them through folder structures, config files, and setup steps.

Now, he uses a custom tool built on top of the Filesystem MCP Server that runs a guided onboarding script. It uses list_directory and get_file_info to show a visual map of the project structure and highlights important configuration files using read_file.

The new hires can explore the layout themselves, and the AI assistant answers basic questions about where things live and what they do—making the onboarding process faster, more interactive, and less overwhelming.

20.2 Step-by-Step Installation

20.2.1 Steps

1. All you need to do is to just edit your claude config json file

A. Option 1: NPX on Windows

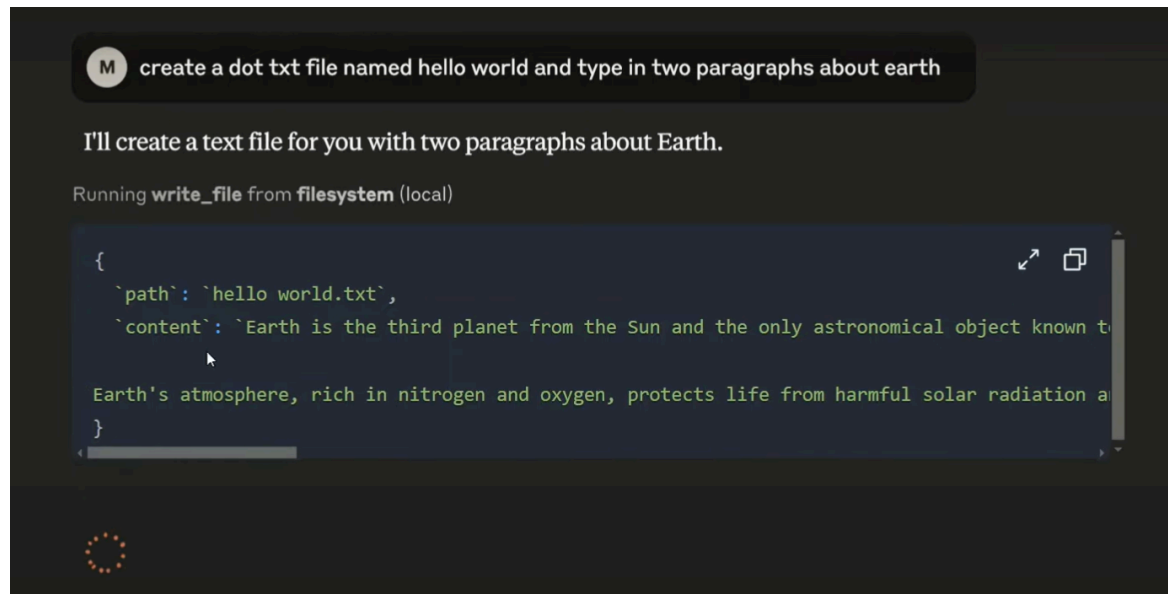
```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/Desktop",
        "/path/to/other/allowed/dir"
      ]
    }
  }
}
```

claude_desktop_config.json

B. Option 2: Docker

You can provide sandboxed access to specific directories or files by mounting them into the Docker container under the `/projects` directory. You may optionally add the `'ro'` (read-only) flag for added safety, preventing unintended modifications.

```
{
  "mcpServers": {
    "filesystem": {
      "command": "docker",
      "args": [
        "run",
        "-i",
        "--rm",
        "--mount", "type=bind,src=/Users/username/Desktop,dst=/projects/Desktop",
        "--mount", "type=bind,src=/path/to/allowed/dir,dst=/projects/other/allowed/dir,ro",
        "--mount", "type=bind,src=/path/to/file.txt,dst=/projects/path/to/file.txt",
        "mcp/filesystem",
        "/projects"
      ]
    }
  }
}
```



And that's it! With just a few simple steps, you can now give your AI assistant seamless access to your file system—making tasks like reading files, navigating directories, and applying smart edits faster and more accurate than ever.

This integration is especially helpful for developers and technical teams who want to streamline workflows, reduce manual effort, and empower AI tools to truly understand and assist with real-world projects.

Whether you're cleaning up code, onboarding new team members, or automating repetitive tasks, the File System MCP Server brings a new level of efficiency and intelligence to your daily work.

Chapter 21: Puppeteer MCP Server

Ordering something from Amazon? Let AI order on your behalf

21.1 Introduction

The Puppeteer MCP Server integrates the Model Context Protocol (MCP) with Puppeteer, enabling AI assistants to control a headless browser (a web browser that operates without a GUI like tabs, buttons, or a URL bar) through natural language commands. This allows assistants to automate browser tasks such as navigation, interaction with page elements, and script execution—empowering users to perform complex web automation seamlessly.

21.1.1 Puppeteer MCP Tools

Here's a concise list of tools available in the Puppeteer MCP server, each designed to help your assistant interact with and control the browser environment:

1. **puppeteer_navigate**
Navigate to any URL in the browser.
2. **puppeteer_screenshot**
Capture full-page or element-specific screenshots.
3. **puppeteer_click**
Click an element on the page using its CSS selector.
4. **puppeteer_hover**
Hover over an element using its CSS selector.
5. **puppeteer_fill**
Fill input fields with specified values.
6. **puppeteer_select**
Select a value from a dropdown.
7. **puppeteer_evaluate**
Execute JavaScript in the browser's console.

21.1.1 Real-World Applications

A. Product Manager's On-Demand UI Check

Meet Rohan, a product manager working on a customer dashboard. During a stakeholder review call, a colleague mentions that a UI component on the staging site seems misaligned. Instead of deferring the issue, Rohan uses the Puppeteer MCP Server via his AI assistant to investigate in real time.

He says, "Navigate to the staging site URL and capture a screenshot of the payment section." The assistant uses `puppeteer_navigate` and `puppeteer_screenshot` tools to load the site and capture the relevant area. Within seconds, Rohan shares the screenshot with the team. Later, he requests, "Run a script to log all console warnings," helping the dev team identify JavaScript issues.

This kind of instant web automation streamlines communication, reduces feedback loops, and enables faster decision-making—all without writing manual Puppeteer scripts.

21.2 Step-by-Step Installation

21.2.1 Prerequisites

To use the Puppeteer MCP Server, you'll need one of the following options installed:

1. **Docker Desktop:** Recommended for headless browser automation. You can download from <https://www.docker.com/products/docker-desktop/>
2. **Node.js & NPX:** If you prefer seeing the browser UI or need more local flexibility, make sure you have Node.js installed.

Depending on your setup and preferences, you can either run Puppeteer MCP via Docker (headless) or NPX (browser UI).

21.2.2 Steps

So, we have two options here, we can use either Docker to trigger a headless browser or we can use npx to open our regular browser.

1. Using Docker (Headless Browser)

This is ideal for automation workflows that don't require visual interaction with the browser. Simply add the following to your Claude config JSON file:

```
{
  "mcpServers": {
    "puppeteer": {
      "command": "docker",
```

```

    "args": ["run", "-i", "--rm", "--init", "-e", "DOCKER_CONTAINER=true",
"mcp/puppeteer"]
  }
}
}

```

If you'd like to pass custom launch options (like disabling headless mode or setting a custom browser path), you can include:

```

{
  "mcpServers": {
    "puppeteer": {
      "command": "docker",
      "args": ["run", "-i", "--rm", "--init", "-e", "DOCKER_CONTAINER=true",
"mcp/puppeteer"],
      "env": {
        "PUPPETEER_LAUNCH_OPTIONS": "{ \"headless\": false, \"args\": [\"--window-size=1280,720\"] }",
        "ALLOW_DANGEROUS": "true"
      }
    }
  }
}

```

2. Using NPX (Browser Window Visible)

This option is useful when you want to visually debug or observe browser behavior.

```

{
  "mcpServers": {
    "puppeteer": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"]
    }
  }
}

```

To pass launch options when using NPX:

```

{
  "mcpServers": {
    "puppeteer": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"],
      "env": {

```

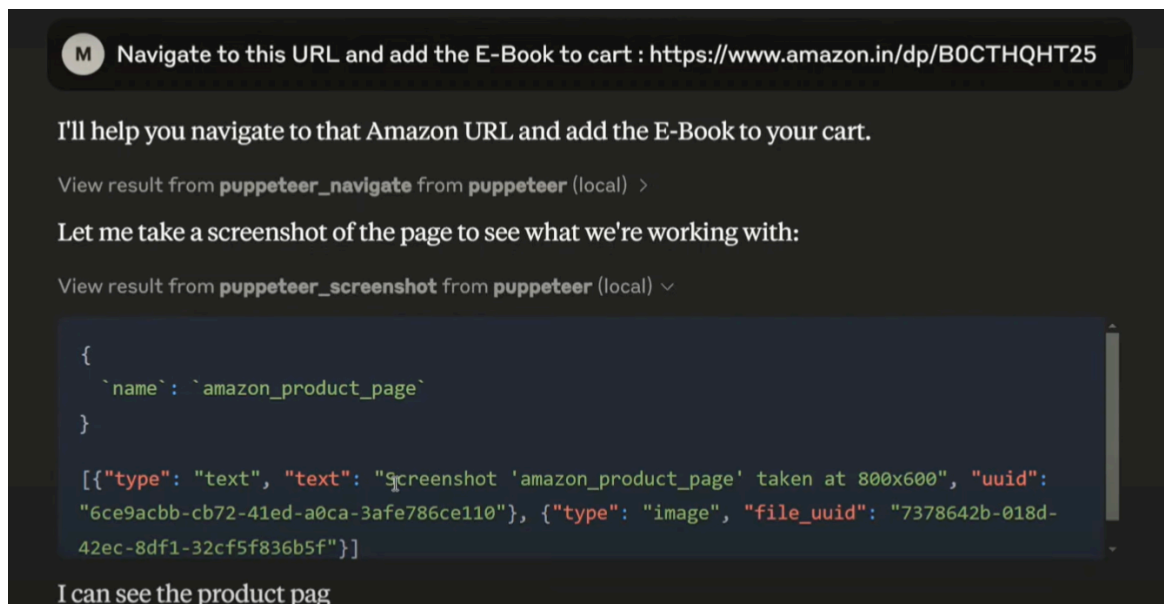
```
"PUPPETEER_LAUNCH_OPTIONS": "{ \"headless\": false, \"executablePath\":  
\"C:/Program Files/Google/Chrome/Application/chrome.exe\" }"  
}  
}  
}  
}
```

Once your Puppeteer MCP Server is configured, you can begin interacting with your browser using natural language through Claude desktop.

For example: “Navigate to this Amazon URL and add the ebook to cart.”

AI will:

- Launch a headless (or visible, if configured) browser instance
- Navigate to the provided product page
- Analyze the HTML and available elements
- Dynamically determine the correct Add to Cart button using selectors
- Click it on your behalf
- Provide confirmation once the book is successfully added



You'll see the entire process unfold in real time — from opening the page to parsing DOM elements and confirming actions via on-screen messages like “Added to eBook cart.”

This isn't just web scraping — it's active browser automation with interactive feedback, useful for:

- Automated testing
- Shopping tasks
- Form submissions

- Any complex web flow requiring real user-like interaction

Anything to be done in the browser, now AI can handle it !

Part 6: Customizations

Chapter 22: MCP using Local LLMs with Ollama

Moving beyond Claude AI & Cursor AI

22.1 Introduction

Want your own personal AI assistant that stays completely local and respects your privacy?

With the Model Context Protocol (MCP) and a local LLM running on Ollama, you can build a snappy little assistant to automate stuff—without sending your data to the cloud. This chapter walks you through setting up a private, offline automation setup using MCPHost and your favorite function-calling model.

Running a local LLM means total control, better privacy, and no annoying API rate limits. MCPHost helps your local model connect with MCP-compatible servers—think web search, databases, or even tools that post to social media—all right from your own machine.

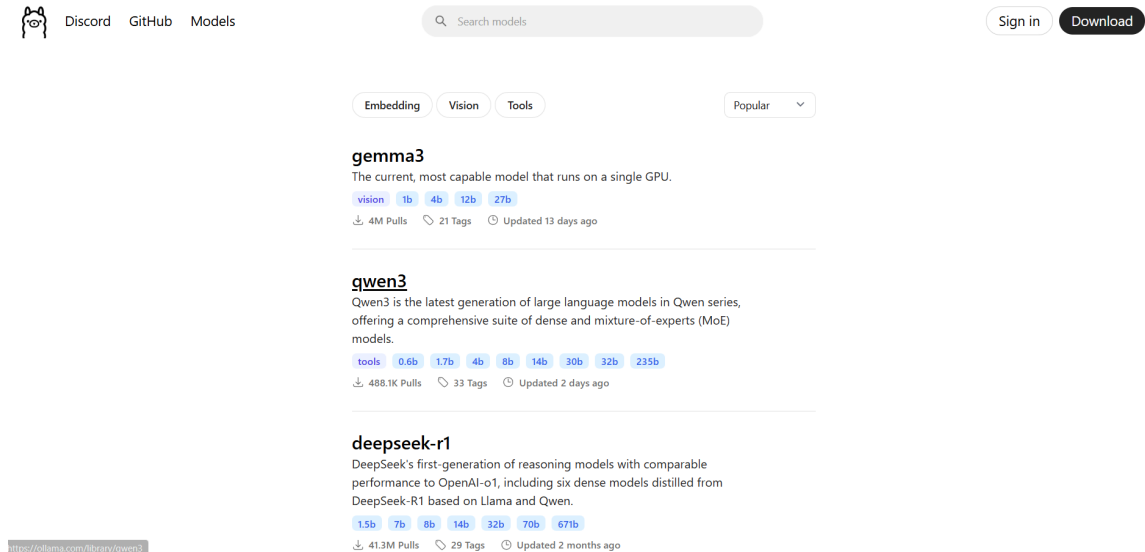
22.1.1 Task Automation with MCPHost

MCPHost is the CLI glue that connects your local AI model to external tools—without handing over your data to the cloud. Picture this: your AI assistant runs on your machine, searches the web through a locally hosted DuckDuckGo wrapper, or posts updates using a tool server you control. Every action flows through the tools you set up, keeping your data right where it belongs: on your computer. It's private automation with zero mystery meat.

22.2 Step-by-Step Installation

22.2.1 Prerequisites

1. **Local function-calling-capable LLMs** from Ollama installed and running. You can refer to the models page to see the models with tool support (see the tools tab).



2. Go 1.23 or later. We already set up Go for WhatsApp MCP if you remember.

22.2.2 Steps

1. Install and run your model

```
ollama pull qwen2.5:3b
ollama serve
```

2. Install MCPHost:

```
go install github.com/mark3labs/mcphost@latest
```

3. Create a config file (like ~/.mcp.json) with the required server configuration:

```
{
  "mcpServers": {
    "sqlite": {
      "command": "uvx",
      "args": [
        "mcp-server-sqlite",
        "--db-path",
        "/tmp/foo.db"
      ]
    }
  },
}
```

```

"filesystem": {
  "command": "npx",
  "args": [
    "-y",
    "@modelcontextprotocol/server-filesystem",
    "/tmp"
  ]
}
}
}

```

4. Finally to call the MCP server via your local LLM, use the following command:

```
mcphost -m ollama:qwen2.5:3b --config ./config.json
```

The `-m` flag specifies which model to call and `--config` directs where to look for the json config. In the below screenshot, you can see we enabled multiple MCP servers including filesystem, sqlite and duckduckgo.

```

(base) C:\Users\datas>mcphost -m ollama:qwen2.5 --config "C:\Users\datas\OneDrive\Desktop\local.json"
2025/03/29 10:47:20 INFO Model loaded provider=ollama model=qwen2.5
2025/03/29 10:47:20 INFO Initializing server... name=sqlite
2025/03/29 10:47:22 INFO Initializing server... name=ddg-search
2025/03/29 10:47:24 INFO Initializing server... name=filesystem
2025/03/29 10:47:27 INFO Server connected name=filesystem
2025/03/29 10:47:27 INFO Server connected name=sqlite
2025/03/29 10:47:27 INFO Server connected name=ddg-search
2025/03/29 10:47:27 INFO Tools loaded server=sqlite count=6
2025/03/29 10:47:27 INFO Tools loaded server=ddg-search count=2
2025/03/29 10:47:27 INFO Tools loaded server=filesystem count=11

You: What MCP tools you can access?
⚡ Thinking...

```

22.3 Usage tips

You've got some handy chat commands to manage your session like a pro:

- **/help**: Lists all the available commands you can use
- **/tools**: Shows which tools are currently connected and ready
- **/servers**: Displays your configured MCP servers
- **/history**: Lets you peek at your chat history with the model
- **/quit**: Cleanly exits the session
- **Ctrl+C**: The universal "I'm outta here" shortcut

These are especially useful for debugging, exploring what your assistant can do, or just keeping things tidy.

Note: *Smaller models might need more explicit and structured instructions—like clearly naming the tool you want it to use or giving step-by-step guidance—since they often struggle to infer complex intent from vague prompts.*

With MCPHost and Ollama, you've got a lean, mean, offline automation machine. It's private, flexible, and honestly, pretty fun. Once your tool servers are wired up, your local AI assistant is ready to help out—with no cloud strings attached.

Chapter 23: MCP tools using LangChain

MCP X LangChain? Interesting

23.1 Introduction

If you have been working in the Generative AI space, you must have heard of LangChain, the most popular framework for building out AI agents.

Hope you heard of “LangChain In Your Pocket” 😊

If you've ever wanted to hook up LangChain with external tools — like your own custom servers or services like Slack and File System — you're in luck! The LangChain MCP Adapter makes it super easy to connect your LangChain agents to any MCP server with just a little bit of setup.

Let's dive in and see how you can get started.

23.2 Step-by-Step Installation

23.2.1 Prerequisites

1. First things first, you'll need a couple of libraries. Here's the quick install:

```
pip install langchain-mcp-adapters
pip install langgraph
pip install langchain-google-genai
```

Note: Remember to install the compatible langchain library for your LLM provider. We are using Gemini for this example hence installing langchain-google-genai. Also, kindly set your respective API key before using any LLM

2. Next, we create a small but powerful MCP server for us (basic calculator)

```
# math_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Math")

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@mcp.tool()
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

math_server.py

Now we are ready to design our Client

23.2.2 Steps

1. You can create a MCP client that will trigger the LLM to use the MCP server. To do this:
 - a. Start the tool server and open a communication channel with it using the `StdioServerParameters` class.
 - b. Initialize the session so the server is ready to receive and respond to commands using `ClientSession` class.
 - c. Loads available tools from the running server

- d. Create a ReAct agent that can reason through and use those tools when needed

Refer to the following script (named `mcp_serve.py`)

```
# Create server parameters for stdio connection
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent

from langchain_google_genai import ChatGoogleGenerativeAI
model=ChatGoogleGenerativeAI( model="gemini-2.0-flash-001")

server_params = StdioServerParameters(
    command="python",
    # Make sure to update to the full absolute path to your math_server.py file
    args=["/path/to/math_server.py"],
)

async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        # Initialize the connection
        await session.initialize()

        # Get tools
        tools = await load_mcp_tools(session)

        # Create and run the agent
        agent = create_react_agent(model, tools)
        agent_response = await agent.ainvoke({"messages": "what's (3 + 5) x 12?"})
```

`mcp_serve.py`

2. Once you run this client in your terminal (command is **`python mcp_serve.py`**), you will get an AI response message in your output.

```
(base) C:\Users\datas\OneDrive\Desktop>python mcp_serve.py
{'messages': [HumanMessage(content='what is (2+3)x4', additional_kwargs={}, response_metadata={}, id='6ed7c947-389a-414a-9fb2-0ead986a48c6'), AIMessage(content='what is (2+3)x4', additional_kwargs={'function_call': {'name': 'add', 'arguments': {'a': 2.0, 'b': 3.0}}}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.0-flash-001', 'safety_ratings': [], id='run-b4c43255-dacd-4409-b29b-5982f31ee409-0', tool_calls=[{'name': 'add', 'args': {'a': 2.0, 'b': 3.0}, 'id': 'e3882b0a-7854-497c-8091-126d5893c5fd', 'type': 'tool_call'}], usage_metadata={'input_tokens': 25, 'output_tokens': 3, 'total_tokens': 28, 'input_token_details': {'cache_read': 0}}), ToolMessage(content='7', name='add', id='fc3f4b3e-3336-4105-8b74-7b374b1b0ee6', tool_call_id='e3882b0a-7854-497c-8091-126d5893c5fd'), AIMessage(content='what is (2+3)x4', additional_kwargs={'function_call': {'name': 'multiply', 'arguments': {'a': 5.0, 'b': 4.0}}}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.0-flash-001', 'safety_ratings': [], id='run-60fd48ca-41d0-496c-b0d7-d1cf637eee46-0', tool_calls=[{'name': 'multiply', 'args': {'a': 5.0, 'b': 4.0}, 'id': 'ae45f6c8-cb1c-4e31-8e16-98eb60048a1a', 'type': 'tool_call'}], usage_metadata={'input_tokens': 30, 'output_tokens': 3, 'total_tokens': 33, 'input_token_details': {'cache_read': 0}}), ToolMessage(content='20', name='multiply', id='f4728ff3-de06-4e05-bf01-a0e14074cd61', tool_call_id='ae45f6c8-cb1c-4e31-8e16-98eb60048a1a'), AIMessage(content='(2+3)x4 is 20.', additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.0-flash-001', 'safety_ratings': [], id='run-339995c4-10a2-4416-9eb1-196606a812b7-0', usage_metadata={'input_tokens': 35, 'output_tokens': 12, 'total_tokens': 47, 'input_token_details': {'cache_read': 0}})]], WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1744620223.413153 2928 init.cc:229] grpc_wait_for_shutdown_with_timeout() timed out.
```


If you notice the output, the LLM has reasoned through the input prompt and decided to perform a function call using the MCP server. You can actually extend this to support calling multiple MCP servers.

3. Simply extend your server file to contain more tools.

```
# math_server.py
...

# weather_server.py
from typing import List
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Weather")

@mcp.tool()
async def get_weather(location: str) -> str:
    """Get weather for location."""
    return "It's always sunny in New York"

if __name__ == "__main__":
    mcp.run(transport="sse")
```

4. Next modify your client file to include all the command line arguments that are necessary to call the servers.

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

from langchain_google_genai import ChatGenerativeAI
model = ChatGenerativeAI(model="gemini-2.0-flash-001")

async with MultiServerMCPClient(
    {
        "math": {
            "command": "python",
            # Make sure to update to the full absolute path to your math_server.py file
            "args": ["/path/to/math_server.py"],
            "transport": "stdio",
        },
        "weather": {
            # make sure you start your weather server on port 8000
            "url": "http://localhost:8000/sse",
            "transport": "sse",
        },
    },
    model=model,
)
```

```
    }  
  }  
) as client:  
    agent = create_react_agent(model, client.get_tools())  
    math_response = await agent.ainvoke({"messages": "what's (3 + 5) x 12?"})  
    weather_response = await agent.ainvoke({"messages": "what is the weather in  
nyc?"})
```

Notice how we have extended the client to include both math and weather server with all the necessary arguments.

It's that easy !

Not just existing MCP Servers, you can even create your own servers and use them.
What a bliss!

Chapter 24: Any LLM - Any MCP

Can I use MCP with Llama or Qwen or GPT? Yes

24.1 Introduction

So you've got a powerful LLM like GPT-4, Claude, or Gemini — and you've also got an MCP server that exposes cool tools like file access, Slack integrations, or even your own custom APIs.

Now what?

Thanks to the **mcp-use python library**, you can easily connect any LLM to any MCP server and let your model make smart tool calls in real time. No fuss, just function calls that work.

Let's walk through how this magic happens.

24.1.1 Features

At its heart, mcp-use is a lightweight Python library designed to seamlessly connect your LLM with any MCP server — and it offers far more than just basic tool call handling.

Here's why you'll love using it:

1. **Incredibly Simple Setup** : Have a working agent connected to an MCP server in just 6 lines of code — no exaggeration.
2. **Compatible with All Major LLMs** : Whether you're working with OpenAI, Claude, Groq, Llama, or any LangChain-supported model that allows tool calls, mcp-use integrates effortlessly.
3. **Built-in HTTP Connectivity** : Just provide the address of an MCP server running on a port, and mcp-use will connect directly.
4. **Build Your Own Agents** : Use LangChain adapters or create custom agents from scratch using the provided tools — total flexibility is built in.
5. **Support for Multiple MCP Servers** : Run and manage multiple MCP servers simultaneously. Your agent can dynamically select the best one for each task.
6. **Smart Server Routing** : If you have a group of MCP servers, your agent can automatically choose the most appropriate one based on what the task requires.

7. **Security-First Design** : You can disable or restrict access to sensitive tools — such as those interacting with the file system or network — keeping your environment secure.
8. **Interactive Code Assistant** : Not sure where to begin? The built-in code builder lets you explore tool schemas and generate starter code in seconds.

It's more than just a connector — it's a full-featured toolkit built for real-world applications.

24.2 Step-by-Step Installation

24.2.1 Pre-requisites

1. You can install mcp-use via pip. Install the compatible langchain library for your LLM provider.:

```
pip install mcp-use  
pip install langchain-google-genai
```

Or clone it from source if you prefer:

```
git clone https://github.com/pietrozullo/mcp-use.git  
cd mcp-use  
pip install -e .
```

2. Don't forget to add your API keys as an environment variable:

```
GOOGLE_API_KEY=your-key-here
```

24.2.2 Steps

1. You can create a MCP client that will trigger the LLM to use the MCP server:
 - a. We need to create a client using a config file that holds connection details for tools like Slack
 - b. Creates an MCP agent that connects the model with the client
 - c. Sends a natural language instruction asking the agent to send a message in a Slack channel
 - d. It waits for the agent to figure out and perform the task

All in all, this would look something like this:

```
import asyncio  
import os  
from dotenv import load_dotenv  
from langchain_google_genai import ChatGoogleGenerativeAI  
from mcp_use import MCPAgent, MCPClient
```

```

async def main():
    # Load environment variables
    load_dotenv()

    # Create MCPClient from config file
    client = MCPClient.from_config_file(
        r"C:\Users\User\AppData\Roaming\Claude\claude_desktop_config.json"
    )

    # Create LLM
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash-001")

    # Create agent with the client
    agent = MCPAgent(llm=llm, client=client, max_steps=30)

    # Run the query
    result = await agent.run(
        "Send a message Hi I am Gemini to slacks channel mcp-test",
        max_steps=30,
    )
    print(f"\nResult: {result}")

if __name__ == "__main__":
    asyncio.run(main())

```

mcp1.py

2. Run this file in your terminal (command is **python mcp1.py**). This will essentially help the Gemini model to access the Slacks MCP server that we set up in earlier lessons and send a message to the aforementioned Slacks channel.

You can actually define the config directly in the python file instead of referring to a json file like this:

```

import asyncio
import os
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from mcp_use import MCPAgent, MCPClient

async def main():
    # Load environment variables
    load_dotenv()

    # Create configuration dictionary
    config = {

```

```

"mcpServers": {
  "playwright": {
    "command": "npx",
    "args": ["@playwright/mcp@latest"],
    "env": {
      "DISPLAY": ":1"
    }
  }
}
}

# Create MCPClient from configuration dictionary
client = MCPClient.from_dict(config)

# Create LLM
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash-001")

# Create agent with the client
agent = MCPAgent(llm=llm, client=client, max_steps=30)

# Run the query
result = await agent.run(
    "Find the best restaurant in San Francisco",
)
print(f"\nResult: {result}")

if __name__ == "__main__":
    asyncio.run(main())

```

You can notice how we have mentioned the arguments required for calling the playwright MCP server by using npx directly in the python file.

That's it — your agent is now connected and ready to use tools through MCP!

Chapter 25: Building Your Own MCP Server

Because your LLM deserves a personalized touch!

25.1 Introduction

Imagine you're working with a powerful LLM like Claude or Qwen — but now you want it to interact with your own tools, APIs, or custom logic.

That's where Custom MCP servers come in. In this chapter, you'll learn how to build your own custom MCP server using the Python SDK and expose tools as functions. No paid subscription is needed. The tools are free, the framework is open, and the result is a locally running extension your LLM can call live.

25.1.1 Features

Everything you need to integrate, extend, and operate with the Model Context Protocol.

1. **Works Both Ways:** Create clients that talk to any MCP server, or build your own server that shares tools, data, and prompts.
2. **Flexible Connection Options:** Supports different ways to connect, like stdio, SSE, or streaming over HTTP — all built in.
3. **Easy Tool & Data Sharing:** Add tools that do things or resources that share information, all with simple Python functions.
4. **Built-in App Context:** Access things like progress updates or shared state across tools using helpful context objects.
5. **Support for Auth & Images:** Add optional login with OAuth2, and work with images right out of the box.

25.2 Step-by-Step Installation

25.2.1 Pre-requisites

1. You can initialize your MCP project either using *pip* or *uv*.

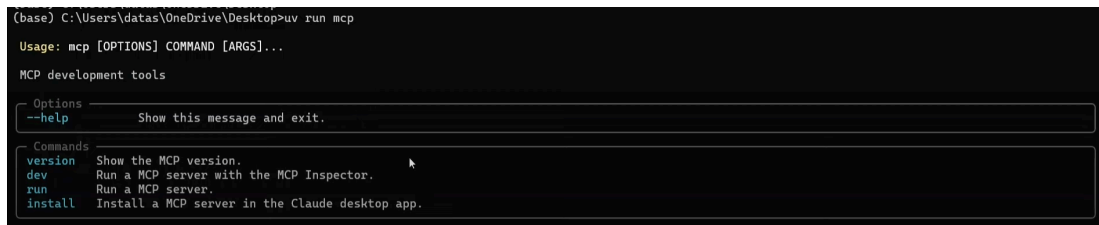
For *uv managed project*:

```
uv init my-mcp-server  
cd my-mcp-server
```

Follow up by adding mcp to your project. This will be your magic ingredient!

```
uv add "mcp[cli]"
```

You can verify the installation by running: *uv run mcp*



```
(base) C:\Users\datas\OneDrive\Desktop>uv run mcp  
Usage: mcp [OPTIONS] COMMAND [ARGS]...  
  
MCP development tools  
  
Options  
  --help  Show this message and exit.  
  
Commands  
  version  Show the MCP version.  
  dev      Run a MCP server with the MCP Inspector.  
  run      Run a MCP server.  
  install  Install a MCP server in the Claude desktop app.
```

Output of *uv run mcp*

25.2.2 Steps

1. Now that we got the setup done, it's time for us to create a small server for us that will have some tools so that our LLM can use them.

```
from mcp.server.fastmcp import FastMCP  
  
# Create MCP Server named "Demo"  
mcp = FastMCP("Demo")  
  
# Tool to add two numbers  
@mcp.tool()  
def add(a: int, b: int) -> int:  
    """Add two numbers"""  
    return a + b  
  
@mcp.tool()  
def subtract(a: int, b: int) -> int:  
    """Subtract b from a"""  
    return a - b
```

server.py

2. To register your MCP server with Claude (or any MCP-compatible LLM), run:

```
mcp install server.py
```


This will automatically add the tool definitions into your Claude Desktop's configuration (or your default MCP config)

```
(base) C:\Users\datas\OneDrive\Desktop>mcp install server.py
[03/29/25 17:45:59] INFO      Added server 'Demo' to Claude config
                        INFO      Successfully installed Demo in Claude app

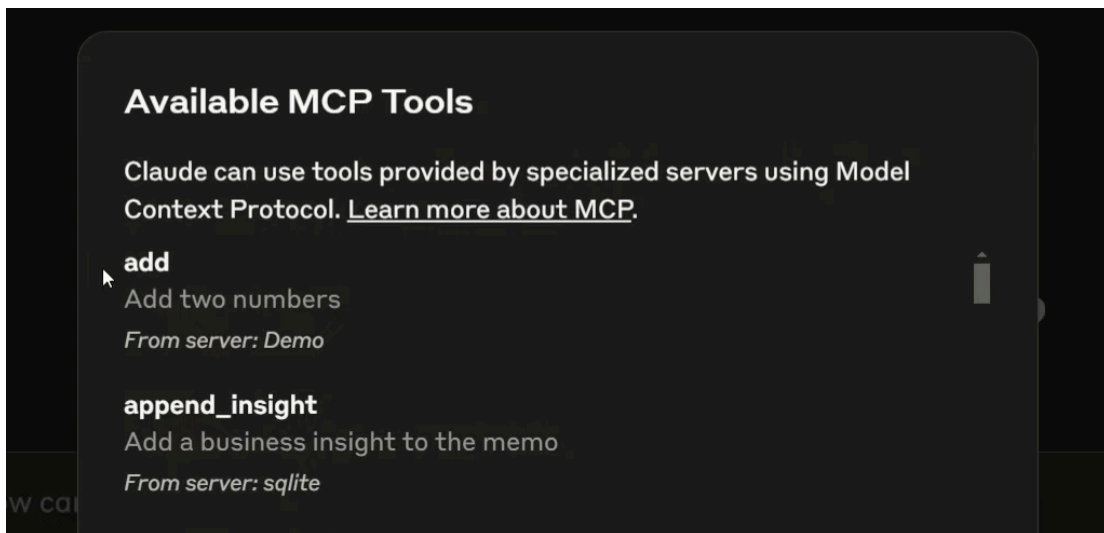
(base) C:\Users\datas\OneDrive\Desktop>
```

Custom MCP server installation

Attention Windows users: MCP sometimes doesn't auto-detect uv path. You may need to edit the config manually and point to the full uv.exe path in your `claude_desktop_config.json` file:

```
"command": "C:\\Users\\datas\\anaconda3\\Scripts\\uv.exe"
```

As for the final step, restart your Claude and check if the tools are listed in your MCP dropdown.



Custom MCP tools

Here are some more MCP customizations that might come in handy as your app grows. Whether it's exposing data, running tools, creating prompts, or handling images and auth — these examples show how flexible MCP can be with just a few lines of code.

- **Resources:** Makes static data (like config) available to the LLM.

```
@mcp.resource("config://app")
def get_config() -> str:
    return "App configuration here"
```

- **Prompts:** A simple text template to ask the model to review code.

```
@mcp.prompt()
def review_code(code: str) -> str:
    return f"Please review this code:\n\n{code}"
```

- **Context:** The Context object gives your tools and resources access to MCP capabilities:

```
from mcp.server.fastmcp import Context

@mcp.tool()
async def long_task(files: list[str], ctx: Context) -> str:
    for i, file in enumerate(files):
        ctx.info(f"Processing {file}")
        await ctx.report_progress(i, len(files))
        data, mime_type = await ctx.read_resource(f"file://{file}")
    return "Processing complete"
```

- **Authentication:** Adds basic authentication to your server using OAuth.

```
mcp = FastMCP("My App",
    auth_provider=MyOAuthServerProvider(),
    auth=AuthSettings(
        issuer_url="https://myapp.com",
        required_scopes=["myscope"],
    ),
)
```

For more details in Authentication you can refer to the OAuth 2.0 server interface provided through `mcp.server.auth` at github.com/modelcontextprotocol/python-sdk/blob/main/src/mcp/server/auth/provider.py.

Your custom MCP setup is now live and ready to serve — whether it's tools, data, or prompts, you're in full control. Go ahead and shape it to fit your app's unique needs!

M Add 37 with 64

I'll add those numbers for you.

View result from **add** from **Demo** (local) ▾

```
{
  `a`: 37,
  `b`: 64
}

101
```

The sum of 37 and 64 is 101. ▸



   Retry ▾

Claude can make mistakes. Please double-check responses.

Chapter 26: MCP might be risky

All that glitters is not gold

The Model Context Protocol is having a moment. Developers and businesses are using it to streamline AI workflows, automate tasks, and connect models to real-world tools. But as with any powerful technology, MCP isn't without its flaws—especially when it comes to security. While the protocol's flexibility and ease of use are selling points, these same features can create openings for attackers. Let's break down the risks, explain why they matter, and how to stay safe—with real-world examples and actionable fixes.

26.1 Why are MCP Servers unsafe?

Many reasons, not one

A. Hidden Malicious Instructions (Tool Poisoning)

Example: Imagine a tool named `GenerateCustomerReport` that's supposed to pull data from a database and format it into a PDF. But hidden in the code is a script that copies sensitive customer emails and uploads them to a hacker's server. The AI client sees only "Generate report," but the tool's backend executes the theft.

Precaution:

1. **Code Audits:** Require developers to review tool source code before deployment.
2. **Sandbox Testing:** Run tools in isolated environments to catch hidden behavior.
3. **Static Analysis Tools:** Use automated scanners to flag suspicious code patterns (e.g., unexpected file access).

B. Users Can't See Hidden Risks

Example:

A tool listed as "Organize files" might delete critical documents if its code lacks safeguards. For instance, a script that deletes files older than 30 days could accidentally target important data if the AI misinterprets dates. Users only see "organize," not the underlying logic.

Precaution:

1. **User-Facing Documentation:** Require tools to include plain-language explanations of what they *actually* do.
2. **Dry-Run Mode:** Force tools to simulate actions before executing them (e.g., "This will delete files X, Y, Z. Proceed?").
3. **Permission Limits:** Restrict tools to read-only access unless write/delete actions are explicitly approved.

C. Fake Updates After Approval (MCP Rug Pulls)

Example:

A trusted calculator tool (MathHelper_v1) is approved for use. Later, hackers compromise its server and push “v2,” which logs every number entered into the tool. If users approve updates automatically, their financial data gets stolen.

Precaution:

1. Version Pinning: Lock tools to specific versions (e.g., only use v1 until v2 is manually verified).
2. Checksum Verification: Teach users to compare file hashes before updating tools.
3. Update Notifications: Flag updates with warnings like “This update requires manual approval.”

D. Hijacking Trusted Tools (Cross-Server Attacks)

Example:

A hacked MCP server overrides the SendEmail tool. Even if you type “john@company.com,” the tool silently forwards copies to hacker@scam.com. The user assumes emails are secure because the tool was vetted—until sensitive data leaks.

Precaution:

1. Tool Signing: Digitally sign tools so MCP servers can verify authenticity.
2. Network Segmentation: Run critical tools (e.g., email) on separate MCP instances to limit cross-server damage.
3. Input Sanitization: Add checks like “Confirm recipient address matches saved contacts.”

E. Real-World Exploits That Worked

Example 1:

1. Cursor AI Exploit: A tool named add() pretended to log user preferences but injected code to steal passwords from a local config file. When the AI ran add(“theme=dark”), it copied credentials to a remote server.
2. Fix: Audit tools for unexpected file access.

Example 2:

1. Fake Email Tool: A tool named BulkEmailSender claimed to send newsletters but redirected output to a hacker’s inbox. Users lost trust in all email tools.
2. Fix: Use tools with audit trails (e.g., logs showing exactly what each tool did).

F. MCP’s Built-In Weaknesses

Example:

If a hacker compromises FileReader_v1, they could use its permissions to access DatabaseConnector_v2. Once inside the database, they pivot to other tools like BackupSystem_v3, encrypting files for ransom.

Precaution:

1. Zero Trust Architecture: Treat every tool as untrusted. Require re-authentication for cross-tool actions.
2. Network Firewalls: Restrict MCP servers from communicating with non-approved services.

G. Weak Protections Today**Example:**

A tool called AutoSave lacks version control. Hackers replace it with AutoSave_v2, which deletes unsaved work. Since users don't check for updates, hours of progress vanish.

Precaution:

1. Immutable Backups: Automatically save tool outputs to a secure location (e.g., write-only cloud storage).
2. Automated Alerts: Notify users when tools are modified. Example: Slack/Teams alerts like "The 'AutoSave' tool was updated at 2 PM—review changes?"

26.2 How to Stay Safe

1. For Developers:

- a. Use sandboxed testing environments.
- b. Reject tools that don't include code audits.

2. For Users:

- a. Treat MCP tools like browser extensions—only install trusted ones.
- b. Enable multi-factor authentication (MFA) for critical workflows.

3. For Everyone:

- a. Run periodic "red team" drills to test MCP security.
- b. Assume every tool will be hacked—plan backups and failsafes.

The Bottom Line

MCP is powerful, but it's not magic. Its risks stem from its simplicity—attackers exploit trust, automation, and invisibility. By treating MCP tools with the same skepticism as public Wi-Fi or email attachments, you can mitigate threats.

Key Takeaway:

1. **Default to Paranoia:** Verify tools, isolate workflows, and assume the worst.
2. **Security is a Team Sport:** Developers, users, and admins must collaborate to harden MCP.

Until MCP adds built-in safeguards (like mandatory code signing), proceed with extreme caution. Think of it as skydiving without a backup parachute—fun until the main chute fails.

Chapter 27 : The Future of MCP and AI Agents

This was one hell of a ride! We just now covered one of the latest advancements made in Generative AI in the short book.

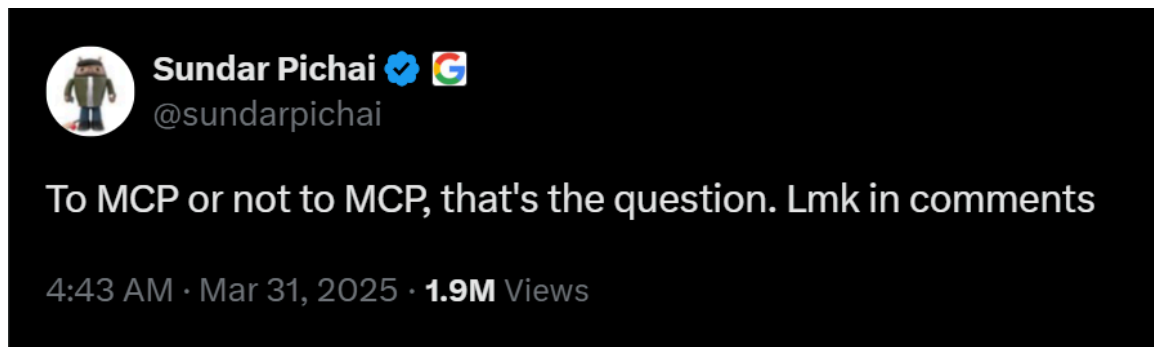
Congratulations

I hope this was enjoyable for you, and you are now equally excited to explore model context protocols to delve into the world of AI Agents 2.0.

The Model Context Protocol isn't just a clever spec for connecting LLMs with tools and data—it's a full-on platform shift. In the same way that HTTP transformed how humans interact with information, MCP is transforming how **AI interacts with the world**. And this shift is happening *faster* than most people realize.

A year ago, building an AI agent meant gluing together prompts, APIs, JSON schemas, and praying it didn't break. Today, with MCP servers, anyone can wire up their personal or professional tools—**zero DevOps, zero cost, and zero lock-in**. That's not just a technical win; it's a cultural one. MCP is making **AI agents accessible to everyone**—developers, creators, marketers, analysts, and even your mom if she wants to automate her garden schedule.

A.A Global Wave of Adoption



Sundar Pichai, Head of Google, making this tweet is no joke my friends. The MCP wave is real and you need to get on board ASAP.

Across GitHub, Hacker News, and even enterprise Slack channels, we're seeing a Cambrian explosion of MCP servers and tools. People are spinning up:

1. Personal productivity agents that fetch files, summarize emails, and automate calendars.

2. Enterprise automation suites that integrate CRMs, databases, and ticketing systems.
3. Creative MCPs that remix media, generate storyboards in Blender, or collaborate on Figma.

Open-source MCP servers are popping up for everything from **SQL access** to **3D modeling**, and platforms like Cursor, Claude, and WindSurf are embracing it natively. The vibe is clear: *If your tool doesn't speak MCP, it's missing out.*

B. The World is Becoming Agent-Ready

As the infrastructure around AI agents matures, MCP is laying the foundation for a **world where AI becomes a universal interface**. You won't "open apps" anymore—you'll just ask your AI assistant to "book a flight," "edit that document," or "pull the latest sales numbers," and it'll know what to do because the protocols are in place.

In this world, **context is king**, and MCP is the protocol that delivers it—rich, structured, and dynamically scoped to the task at hand. No more dumb AI assistants guessing what you mean. With MCP, they *know*.

But as with every powerful tech, there are trade-offs and responsibilities:

- **Security:** An agent that can access your database and automate tasks can also delete all your records if not sandboxed properly. We covered this in Part 4, but it's worth repeating—**MCP requires careful permissioning and access controls**.
- **Privacy:** Running agents locally? Awesome. But if you're exposing an MCP server publicly, be aware of what data is accessible and who can call what tools.
- **Tool Bloat:** As the ecosystem grows, it's tempting to hook your agent up to everything. Be intentional. Not every agent needs access to your kitchen sink.
- **Emerging Standards:** MCP is still young. Expect changes, upgrades, and debates. But if you're building now, you're ahead of the curve—and helping shape the protocol's future.

C. Where We're Headed

The next few years will likely bring:

1. **MCP-native platforms:** Like how browsers run HTML, we'll have agent runtimes that "speak MCP" out of the box.

2. **Marketplace of MCP servers:** Publicly available plug-and-play resources and tools, sharable across companies and communities.
3. **Personal OS-level agents:** Your laptop will come with a built-in agent that can read your filesystem, interact with apps, and run tasks—all via MCP.

MCP is not a passing trend. It's laying the groundwork for **AI-native computing**, where your agents don't just live in the cloud—they live in your workflow, your desktop, and your daily routine.

EndNotes

As we close out *Model Context Protocol: Advanced AI Agents for Beginners*, it's clear that MCP is more than a technical spec—it's a shift in how we build and use AI. What began as simple tool calling has evolved into a powerful framework for enabling agents to interact with live systems, data, and services in a way that's secure, scalable, and accessible. By abstracting away the messy, custom glue code and replacing it with a universal protocol, MCP has opened the door to a new class of applications where AI agents can truly operate like digital coworkers.

What makes MCP so impactful is how approachable it's become. With just a well-structured `config.json`, even solo developers and hobbyists can spin up local AI agents that talk to GitHub, Notion, Gmail, or any API you can think of. No longer is agent development locked behind full-stack wizardry or enterprise-level engineering teams. Now, anyone can experiment, automate, and build intelligent systems right from their laptop. MCP isn't just enabling developers—it's empowering a new wave of creators.

Looking ahead, MCP will only grow in importance as AI agents become more autonomous and multi-agent orchestration becomes mainstream. Expect to see more open-source servers, standardized resource libraries, and integrations with everything from low-code tools to robotics. But with that growth comes responsibility: security, transparency, and ethical use must be baked into every setup. If you've made it this far, you're not just keeping up with the future—you're helping shape it. And the best part? This is just the beginning.

About the Authors

Mehul Gupta and **Niladri Sen** are the co-authors of *Model Context Protocol : Advanced AI Agents for Beginners*, a book aimed at making next-gen AI agent development accessible to everyone—from tech-savvy builders to curious newcomers.

Mehul Gupta

Mehul is a Data Scientist and Researcher from India with 6+ years of experience in AI and Data Science. A Computer Science graduate from IIITDM Jabalpur, he specializes in Generative AI, NLP, Graph Analytics, Reinforcement Learning, and Time Series Forecasting—building real-world solutions across healthcare, finance, and enterprise AI.

He's the author of the national bestseller on Generative AI **"LangChain In Your Pocket"**, published with Packt, and a prolific content creator with **400+ technical blogs** (3M+ views) and **1,000+ YouTube tutorials (3M+ views)** under *Data Science in your Pocket*. His work is known for making complex AI concepts beginner-friendly and practical.

Connect with Mehul:

Medium: https://medium.com/@mehulgupta_7991

LinkedIn: <https://www.linkedin.com/in/mehulgupta7991/>

YouTube: <https://www.youtube.com/@datascienceinyourpocket>

Email: mehulgupta2016154@gmail.com

Niladri Sen

Niladri Sen is a Data Scientist and AI enthusiast with over 5 years of experience building scalable ML and GenAI solutions in the BFSI sector, spanning B2B Fintech and B2C BankTech. He holds USPTO-published patents in Document AI and is currently advancing Generative AI initiatives like agentic AI frameworks tailored for finance.

Known for his ability in simplifying technical concepts and bringing them to life, he actively mentors data science practitioners through hands-on learning. He brings a practical, tool-driven mindset to the MCP ecosystem, contributing not just to this book but also to the growing community of developers adopting the protocol. A B.Tech graduate in Computer Science from KIIT University, he blends academic rigor with real-world impact.

Connect with Niladri:

LinkedIn: <https://www.linkedin.com/in/niladri-sen-ns2707/>

Email: me.niladri.sen@gmail.com

Together, Mehul and Niladri are on a mission to demystify the world of AI agents and empower builders everywhere to take part in the future of intelligent automation.